ChatGPT 5.2 Thinking

Share

Ich bin fasziniert von GraalVM. Was könnte ich im Kontext von INTERLIS mit dem Truffle Framework machen?

Thought for 58s

Spannende Kombination. **Ja, da gibt es wirklich interessante Möglichkeiten** — aber der Trick ist, **nicht INTERLIS komplett neu zu bauen**, sondern gezielt die Teile zu nehmen, bei denen Truffle seine Stärken ausspielt.

INTERLIS ist eine **formale Modellierungs-/Austauschsprache** (software- und systemunabhängig; heute meist INTERLIS 2.3) und rundherum gibt es bereits etablierte Tools wie **ili2c** (Compiler/Prüfer für INTERLIS-Modelle). interlis.ch +1
Truffle ist dagegen ideal, wenn du eine **Sprache oder ein Tooling auf AST-Basis** bauen willst (eigene Sprache via `TruffleLanguage`, Instrumentierung via `TruffleInstrument`, Polyglot-Einbettung über die GraalVM Polyglot API). GraalVM +2

Was du konkret im INTERLIS-Kontext mit Truffle machen könntest

1) INTERLIS-Regel-Engine als Truffle-Sprache

Baue **keine komplette INTERLIS-Implementierung**, sondern eine **Teil-Sprache für Regeln**:

- Constraints / Plausibilitätsregeln
- Ausdrucksauswertung
- Filter- und Selektionslogik

- Ableitungen (derived values)

Nutzen:
Du bekommst ein performantes, instrumentierbares Laufzeitsystem für Regeln auf Datensätzen (z. B. XTF/DB-Objekte), plus Debug/Tracing.

Warum Truffle passt:
Regeln sind AST-lastig, profitieren von Spezialisierung/JIT, und lassen sich gut erklären ("welche Regel hat warum gefeuert?").

---

2) Polyglotte INTERLIS-Query-/Transformations-DSL

Erstelle eine kleine DSL wie:

- `SELECT ... FROM ... WHERE ...`
- Mapping-/Transformationsregeln zwischen Modellen
- Datenqualitätschecks

Dann kannst du sie in Java (oder anderen Graal-Sprachen) einbetten. GraalVM unterstützt genau dieses Polyglot-Zusammenspiel über Truffle. GraalVM +1

Beispiel-Idee:
Ein ETL-Tool lädt INTERLIS-Daten in Java, ruft dann benutzerdefinierte Regeln in deiner Truffle-DSL auf.

---

3) "Explainable Validator" mit TruffleInstrument

Statt nur *valid/invalid* zu sagen, baust du ein Tool, das zeigt:

- welche Regeln ausgeführt wurden

- welche Felder gelesen wurden
- warum eine Verletzung entstand
- Coverage der Regelbasis (welche Regeln wurden nie getroffen?)

Truffle erlaubt explizit das Bauen von **sprachunabhängigen Tools** (Profiler, Debugger, Instrumentierung) über `TruffleInstrument`. GraalVM

Das wäre im INTERLIS-Umfeld sehr wertvoll für:

- Abnahmetests
- Fachstellen / Modellpflege
- Auditierbarkeit

---

4) INTERLIS-Migrations- und Mapping-DSL

Ein sehr nützliches Produkt wäre eine DSL für:

- **Mapping von Modellversion A → B**
- Attribut-Renaming
- Strukturumbauten
- Wertetransformationen
- Prüfregeln für Migrationsqualität

Truffle hilft dir, daraus eine echte Laufzeit zu machen (inkl. Profiling, Debugging, REPL, Tests).

---

5) REPL / “Model Playground” für INTERLIS

Eine interaktive Shell, z. B.:

- Modell laden
- Klassen/Attribute inspizieren
- Testausdrücke ausführen
- Beispielobjekte validieren
- Regeln live ändern

Das ist gerade für Ausbildung, Prototyping und Modellreviews super.

Was ich nicht als erstes machen würde

Kompletten INTERLIS-Compiler in Truffle neu implementieren

Das wäre ein großes Projekt (Syntax, Semantik, Metamodell, Referenzen, Transferregeln etc.). Für den Einstieg ist das viel Aufwand und bringt wenig gegenüber bestehender Toolchain wie **ili2c**. interlis.ch

Besserer Ansatz:

- INTERLIS-Modell mit bestehendem Compiler parsen/prüfen
- relevante Teile (z. B. Regeln/Ausdrücke) in deine Truffle-AST überführen
- Ausführung/Analyse in Truffle machen

Das kombiniert das Beste aus beiden Welten.

Ein realistisches erstes Projekt (MVP)

Projektidee: "INTERLIS Constraint Runner"

1. **Input**

- `.ili` -Modell
- Beispieldaten (Objekte / XTF-Import über bestehende Java-Toolchain)

2. **Frontend**
   - Parse/Extrahiere Constraints aus dem Modell (anfangs auch nur manuell/teilautomatisch)
3. **Truffle-Language**
   - AST-Knoten für:
     - Vergleich
     - boolesche Operatoren
     - Zugriff auf Attribute
     - einfache Funktionen
     - Mengen-/Null-Checks
4. **Runtime**
   - Ausführung pro Objekt / pro Klasse
   - Fehlerreport mit Pfad + Regel-ID
5. **TruffleInstrument**
   - Regel-Coverage
   - Hot rules
   - Trace “welche Daten führten zum Fehler”

Das wäre schon ein starkes Werkzeug — und deutlich überschaubarer als eine vollständige INTERLIS-Neuimplementierung.

Warum das strategisch interessant ist

Du würdest INTERLIS “modernisieren”, ohne den Standard zu verändern:

- **bessere Developer Experience**
- **erklärbare Validierung**
- **polyglotte Einbettung** in moderne JVM/Tool-Stacks
- potenziell **hohe Performance** bei komplexen Regelmengen durch Truffle/Graal-Optimierungen.

GraalVM +1

Wenn du willst, skizziere ich dir als Nächstes eine **konkrete Architektur** (Pakete/Klassen, AST-Knoten, Datenschnittstellen) für so einen *INTERLIS Constraint Runner* auf GraalVM/Truffle — in Java.

Sources

"Mapping von Modellversion A → B" finde ich super interessant.
Erkläre mir das vertiefert und konrketer.

Thought for 1m 18s

Ja — **das ist genau einer der spannendsten Use Cases**.
Und zwar, weil „**Modellversion A → B**“ bei INTERLIS selten nur ein simples Umbenennen ist, sondern oft eine **semantische Migration**:

- Klassen / Topics werden verschoben
- Attribute werden umbenannt, gesplittet oder zusammengeführt
- Wertebereiche (Enums, Domains) ändern sich
- Kardinalitäten werden strenger

- Beziehungen werden expliziter modelliert
- neue Pflichtfelder kommen dazu

INTERLIS ist dabei genau auf langfristige Modellkompatibilität ausgelegt (modellbasiert, systemunabhängig; auf der Doku-Seite sind auch Tools wie ili2c, ilivalidator, ili2db gelistet), und die heute häufig genutzte Version ist weiterhin 2.3; gleichzeitig weist die offizielle Seite darauf hin, dass die englische 2.4-Referenz veraltet ist und die aktuelle Fassung (2024) auf DE/FR verfügbar ist. interlis.ch
Truffle passt dazu, weil es explizit für Sprachimplementierungen (Interpreter auf AST-Basis) und sprachunabhängige Tools wie Debugger/Profiler gedacht ist. docs.oracle.com +1

Was „Mapping A → B“ im INTERLIS-Kontext konkret bedeutet

Stell dir vor:

- **Modell A** = altes Fachmodell (Produktivdaten liegen darin vor)
- **Modell B** = neue Version (neue Struktur / neue Regeln)
- Ziel: Daten **verlustarm, nachvollziehbar, reproduzierbar** migrieren

Das ist nicht nur ein ETL-Job, sondern im Kern ein **regelgetriebenes Migrationsprogramm**.

Die 5 Ebenen eines Mappings

1. **Struktur-Mapping**
   - `TopicA.ClassX → TopicB.ClassY`
   - Klassensplit oder Klassenmerge
2. **Attribut-Mapping**
   - `name → bezeichnung`
   - `strasse + hausnummer → adresse.text`

- `vollname → vorname, nachname`

3. **Wert-Mapping**
   - Enum-Werte umcodieren
   - Einheiten umrechnen (m ↔ cm)
   - Textformate normalisieren
   - Datentyp-Konversionen
4. **Beziehungs-/Identitäts-Mapping**
   - OID/TID übernehmen oder neu generieren
   - Referenzen neu auflösen
   - 1:n ↔ n:m Strukturänderungen
5. **Qualitäts-/Validierungsregeln**
   - Pflichtfelder auffüllen
   - Plausibilitätschecks
   - Warnungen statt harte Fehler bei Altlasten

Warum Truffle hier wirklich nützlich ist (und nicht nur „nice to have“)

Mit Truffle baust du eine **Migrations-DSL** (Domain-Specific Language), die wie eine kleine Sprache für Mapping-Regeln funktioniert.

Der Mehrwert gegenüber „einfach Java-Code schreiben“

Mit Truffle bekommst du:

- **AST-basierte Ausführung** (Regeln sind strukturierte Knoten, nicht nur String-Skripte)
- **Tooling** (Tracing, Debugging, Coverage, Profiling) via Instrumentation
- **Polyglot-Einbettung** (Java-Hauptpipeline + DSL für Fachregeln)

- **Explainability** (welche Regel hat welches Zielattribut gesetzt?)
- potenziell gute Laufzeit bei vielen gleichartigen Objekten durch Truffle/Graal-Optimierungen.

docs.oracle.com +1

Für Migrationsprojekte ist das Gold wert, weil du am Ende nicht nur „es lief durch“ willst, sondern:

- **Was wurde wie transformiert?**
- **Wo gab es Defaults?**
- **Welche Regeln wurden nie benutzt?**
- **Welche Altwerte führten zu Warnungen?**

Wie so eine Mapping-DSL konkret aussehen könnte

Wichtig: Das hier ist **deine eigene DSL auf Truffle**, nicht INTERLIS-Syntax.

Beispiel (vereinfacht)

Plain text

```
MAP "DM_A_1_0" TO "DM_B_2_0" {

  CLASS A.Gebaeude -> B.Building KEY source.tid {

    target.tid := source.tid

    target.name := normalize_text(source.bezeichnung)

    target.hausnummer := parse_house_no(source.adresse_text)

    target.nutzung := MAP_ENUM(source.nutzung) {
      "Wohnen"   -> "RESIDENTIAL"
```

```
      "Gewerbe"  -> "COMMERCIAL"
      "Misch"    -> "MIXED"
      DEFAULT    -> "UNKNOWN" WARN "Unbekannte Nutzung"
    }

    target.flaeche_m2 := TO_M2(source.flaeche, source.flaeche_einheit)

    WHEN source.status != "GELOESCHT"

    ASSERT target.flaeche_m2 >= 0 ERROR "Negative Fläche"
  }

  CLASS A.Eigentuemer -> B.Owner KEY source.tid {
    target.tid := source.tid
    target.vorname := split_name(source.vollname).first
    target.nachname := split_name(source.vollname).last
  }

  LINK A.Gebaeude.eigentuemer_ref -> B.Building.owner_ref
    USING IDMAP "A.Eigentuemer" -> "B.Owner"
}
```

Was daran stark ist

- **fachlich lesbar**
- **testbar**
- **versionierbar**
- **instrumentierbar**
- nicht hart in Java vergraben

Ein realistisches Migrationsmodell: nicht alles deklarativ, sondern 80/20

In echten Projekten brauchst du meist beides:

1) Deklarative Regeln (für 80 %)

Perfekt für:

- Rename
- Enum-Mapping
- Defaults
- einfache Konversionen
- Filter (`WHEN`)
- Assertions

2) Imperative/Helper-Funktionen (für 20 %)

Für komplexe Fälle:

- Geometrieoperationen
- Gruppierungen / Merges
- fuzzy matching
- externe Lookup-Tabellen
- Spezialfälle aus Altdaten

Diese Helper-Funktionen kannst du als **Java-Funktionen** bereitstellen und in der Truffle-DSL aufrufen (Polyglot/Host-Interop-Ansatz).

Architekturvorschlag für einen „INTERLIS Migration Runner“ (Truffle)

A. Vorverarbeitung (INTERLIS-seitig)

Nutze bestehende Toolchain, um Modelle zu verstehen (nicht neu parsen):

- Modell A laden
- Modell B laden
- Metadaten extrahieren (Topics, Klassen, Attribute, Domains, Enums, Kardinalitäten)
- optional: Deltas (A vs. B) berechnen

Output: ein internes **Schema-Metamodell** (Java-Objekte), auf das die DSL zugreifen kann.

B. Mapping-Compiler (deine DSL → Truffle AST)

Parser (ANTLR o.ä.) + AST-Builder, der Regeln in Truffle-Nodes übersetzt.

Typische Nodes:

- `ReadSourceAttributeNode`
- `WriteTargetAttributeNode`
- `EnumMappingNode`
- `FunctionCallNode`
- `WhenFilterNode`
- `AssertNode`
- `CreateTargetObjectNode`
- `ResolveReferenceNode`

C. Runtime / Ausführung

Eine Laufzeit, die pro Source-Objekt arbeitet:

- Source-Objekt lesen
- passende `CLASS` -Regel wählen
- Target-Objekt erzeugen
- Attribute setzen
- Referenzen auflösen (ggf. zweiphasig)
- Ergebnis + Report erzeugen

Wichtig: Für Referenzen brauchst du fast immer eine **ID-Mapping-Tabelle**:

- `sourceOID -> targetOID`
- ggf. pro Klasse getrennt

D. Instrumentierung (TruffleInstrument)

Hier wird es richtig stark:

- **Rule Trace:** welche Regel lief auf welchem Objekt?
- **Attribute Provenance:** Zielattribut `x` wurde aus `source.a + source.b` berechnet
- **Warnings/Errors mit Kontext**
- **Coverage:** welche Mapping-Regeln wurden nie angewendet?
- **Hotspots:** welche Funktionen kosten Zeit?

Das ist der Unterschied zwischen einem Skript und einem **Produktionswerkzeug**.

Der eigentliche Kern: Mapping ist ein *Plan*, nicht nur Code

Ich würde das System intern als **Migration Plan** modellieren.

1. Plan-Phasen

Phase 1 – Analyse

- Modelle A/B laden
- Delta erkennen (rename? split? type change?)

Phase 2 – Kompilierung

- Mapping-DSL prüfen
- Typcheck gegen A/B
- Enum-Abdeckung prüfen
- Kardinalitäten prüfen

Phase 3 – Dry Run

- nur zählen / simulieren
- potenzielle Fehler früh erkennen

Phase 4 – Execution

- Daten transformieren
- ID-Mapping pflegen
- Referenzen finalisieren

Phase 5 – Validation

- Zielmodell validieren (externe Validator-Toolchain)
- Migrationsreport erstellen

Was der Typchecker deiner DSL konkret prüfen sollte

Das macht dein Tool seriös.

Beispiele für statische Checks

- `source.xyz` existiert wirklich in Modell A?
- `target.abc` existiert in Modell B?
- String → Zahl-Konversion erlaubt?
- Enum-Mapping vollständig?
- Pflichtattribute im Ziel immer gesetzt?
- `WHEN` kann dazu führen, dass Zielobjekt fehlt – ist das erlaubt?
- Referenzzielklasse passt?
- Kardinalitätskonflikt (z. B. A: optional → B: mandatory)?

Wenn du das **vor der Ausführung** meldest, sparst du in Projekten massiv Zeit.

Typische Migrationsfälle (und wie du sie modellierst)

1) Einfaches Rename

```
bezeichnung -> name
```

DSL: direkte Zuweisung

Truffle: `ReadNode + WriteNode`

---

2) Enum-Refactoring

A:

- `Wohnen`, `Gewerbe`, `Misch`

B:

- `Residential`, `Commercial`, `Mixed`, `Unknown`

DSL: `MAP_ENUM` mit `DEFAULT` + Warnung

Zusatznutzen: Coverage zeigt, welche Altwerte tatsächlich vorkamen

---

3) Attribut-Split

A:

- `vollname: TEXT`

B:

- `vorname`, `nachname`

DSL: `split_name(...)`

Risiko: Namen sind schmutzig → Warning-Level statt hard fail

---

4) Klassen-Split

A:

- `Objekt`

B:

- `ObjektCore + ObjektExt`

Ansatz:

- eine Source-Klasse erzeugt mehrere Target-Objekte
- Shared OID oder referenzierte OIDs

- Reihenfolge/Referenzauflösung wichtig

---

5) Klassen-Merge

A:

- `Adresse, PLZOrt`

B:

- `Address`

Ansatz:

- Join über Schlüssel / Referenz
- ggf. gruppierte Vorverarbeitung
- eher „Batch-/Index“-Node als rein zeilenweise

---

6) Beziehung wird explizit

A:

- FK-Attribut

B:

- eigene Assoziationsklasse / Relation

Ansatz:

- erst Zielobjekte erzeugen
- dann Link-Phase (zweiter Pass)

Warum eine zweiphasige Ausführung fast immer nötig ist

Gerade bei Referenzen und Splits/Merges:

Pass 1: Objekte erzeugen + IDs mappen

- Zielobjekte anlegen
- `sourceOID -> targetOID` speichern

Pass 2: Referenzen setzen

- Links auflösen
- forward references behandeln
- fehlende Referenzen protokollieren

Das verhindert viele Reihenfolgeprobleme.

Ein konkreter MVP, der schnell echten Nutzen bringt

Wenn du starten willst, würde ich **nicht** mit Split/Merge anfangen.

MVP-Scope (sehr gut machbar)

- 1:1 Klassen-Mapping
- Attribute-Rename
- Defaults
- Enum-Mapping
- `WHEN` -Filter
- `ASSERT`
- Report (errors/warnings/counts)

- Rule Trace (welche Regel lief)

Danach Phase 2

- Referenzauflösung (IDMAP)
- zweiphasige Ausführung
- einfache Attribut-Splits

Phase 3

- Klassen-Split/-Merge
- Gruppierung/Join
- Geometrie-/CRS-Transformationen
- UI/Explainability

Wie Truffle-Klassen dafür aussehen könnten (grob)

Nur als Denkmodell:

- `InterlisMapLanguage extends TruffleLanguage<MigrationContext>`
- `MigrationRootNode`
- `ClassRuleNode`
- `AssignmentNode`
- `EnumMapNode`
- `CallHelperNode`
- `AssertNode`
- `CreateObjectNode`
- `ResolveRefNode`

Plus Runtime:

- `MigrationContext` (Model A/B metadata, helper funcs, config)
- `ObjectStore` (Source/Target records)
- `IdMapRegistry`
- `MigrationReport`

Instrumentierung:

- `MigrationInstrument`
- Events/Tags für RuleStart, RuleEnd, Assignment, Warning, Error

---

Wichtigster Designentscheid

Deine DSL sollte fachlich stabil sein, auch wenn sich die Runtime ändert

Das heißt:

- Fachregeln in DSL-Dateien
- Java/Truffle nur als Engine
- Modelle A/B extern geladen
- Reports reproduzierbar

Dann kannst du dieselbe Migration:

- lokal testen
- im Batch laufen lassen
- in CI prüfen
- auditieren

Wo das im INTERLIS-Umfeld besonders stark wäre

- **Modellpflege über Versionen**
- **kantonale/organisationsspezifische Zielmodelle**
- **Nachvollziehbarkeit für Fachstellen**
- **Migrationen mit vielen Sonderfällen**
- **Regressionstests für Modelländerungen**

Wenn du möchtest, kann ich dir als Nächstes ein **konkretes Mini-Design** machen für:

1. **DSL-Grammatik (ANTLR)**
2. **AST-Nodes in Java/Truffle**
3. **Execution-Plan inkl. ID-Map / 2-Pass-Referenzauflösung**
4. **Beispiel-Migration (A→B) mit Enum-, Split- und Referenz-Mapping**

Sources

Gerne.

Thought for 23s ›

Perfekt — dann machen wir es **konkret**.

Ich skizziere dir ein **Mini-Design für einen „INTERLIS Migration Runner“ auf Truffle**, mit Fokus auf **Mapping von Modellversion A → B**:

1. **DSL-Grammatik (ANTLR, kompakt)**
2. **AST-/Node-Design in Java/Truffle**
3. **Execution-Plan inkl. ID-Map + 2-Pass-Referenzauflösung**
4. **Beispielmigration (Enum + Split + Referenzmapping)**
5. **Was du als MVP wirklich zuerst bauen solltest**

Dabei nutze ich bewusst den Ansatz: **INTERLIS-Modelle mit bestehender Toolchain (z. B. ili2c) verstehen, Migration aber in eigener DSL + Truffle ausführen**. ili2c ist der INTERLIS-Compiler und prüft `.ili` -Modelle; Truffle ist das Framework für Sprachimplementierungen und Instrumentierung (`TruffleLanguage` , `RootNode` , `TruffleInstrument`). docs.interlis.ch +5

---

1) Zielbild in einem Satz

Du baust eine **fachlich lesbare Migrationssprache**, die aus Regeln wie „Klasse X → Y“, „Enum umcodieren“, „Referenz später auflösen“, „Warnung statt Fehler“ besteht — und Truffle liefert dir dafür **Ausführung, Debugbarkeit und Explainability**.

---

2) DSL-Design (konkrete Syntax)

Beispiel-DSL (kompakt, aber realistisch)

Plain text

```
MIGRATION "DM_A_1_0" -> "DM_B_2_0" {

  OPTIONS {
    onMissingReference = WARN;
```

```
  onEnumUnmapped     = WARN;
  stopOnError        = false;
}

CLASS A.Gebaeude -> B.Building
KEY source.tid AS target.tid
WHEN source.status != "GELOESCHT" {

  SET target.name = normalize_text(source.bezeichnung);

  SET target.address.street = parse_street(source.adresse_text);
  SET target.address.number = parse_house_no(source.adresse_text);

  SET target.usage = MAP_ENUM(source.nutzung) {
    "Wohnen"  -> "Residential";
    "Gewerbe" -> "Commercial";
    "Misch"   -> "Mixed";
    DEFAULT   -> "Unknown" WARN "Unbekannte Nutzung in A.Gebaeude.nutzung";
  };

  SET target.area_m2 = to_m2(source.flaeche, source.flaeche_einheit);

  ASSERT target.area_m2 >= 0 ERROR "Fläche darf nicht negativ sein";

  REF target.owner_ref = RESOLVE A.Eigentuemer BY source.eigentuemer_ref;
}

CLASS A.Eigentuemer -> B.Owner
KEY source.tid AS target.tid {

  LET nameParts = split_name(source.vollname);

  SET target.first_name = nameParts.first;
  SET target.last_name  = nameParts.last;
```

```
    SET target.kind = MAP_ENUM(source.typ) {
      "Privatperson" -> "Person";
      "Firma"        -> "Organisation";
      DEFAULT        -> "Unknown";
    };
  }
}
```

Warum diese Syntax gut funktioniert

- **lesbar** für Fach-/Technikteam
- **explizit** (KEY, WHEN, ASSERT, REF)
- **erweiterbar** (z. B. später `EMIT`, `JOIN`, `GROUP BY`)
- lässt sich sauber in AST-Knoten übersetzen

---

3) Mini-Grammatik (ANTLR-Startpunkt)

Nicht vollständig, aber direkt brauchbar als Basis.

antlr

```
grammar InterlisMap;

migration
  : 'MIGRATION' STRING '->' STRING '{' migrationBody* '}'
  ;

migrationBody
  : optionsBlock
  | classRule
  ;
```

```
optionsBlock
  : 'OPTIONS' '{' optionStmt* '}'
  ;

optionStmt
  : IDENT '=' IDENT ';'
  ;

classRule
  : 'CLASS' qname '->' qname
    keyClause
    whenClause?
    '{' classStmt* '}'
  ;

keyClause
  : 'KEY' 'source' '.' IDENT 'AS' 'target' '.' path
  ;

whenClause
  : 'WHEN' expr
  ;

classStmt
  : setStmt
  | letStmt
  | assertStmt
  | refStmt
  ;

setStmt
  : 'SET' 'target' '.' path '=' expr ';'
  ;

letStmt
```

```
  : 'LET' IDENT '=' expr ';'
  ;

assertStmt
  : 'ASSERT' expr severity STRING ';'
  ;

severity
  : 'ERROR'
  | 'WARN'
  ;

refStmt
  : 'REF' 'target' '.' path '=' 'RESOLVE' qname 'BY' sourceRef ';'
  ;

sourceRef
  : 'source' '.' path
  ;

expr
  : expr ('*'|'/') expr                # MulDivExpr
  | expr ('+'|'-') expr                # AddSubExpr
  | expr ('=='|'!='|'<'|'>'|'<='|'>=') expr # CmpExpr
  | expr ('AND'|'OR') expr             # BoolExpr
  | 'NOT' expr                         # NotExpr
  | funcCall                           # FuncCallExpr
  | mapEnumExpr                        # MapEnumExpr
  | 'source' '.' path                  # SourcePathExpr
  | 'target' '.' path                  # TargetPathExpr
  | IDENT                              # VarExpr
  | STRING                             # StringExpr
  | NUMBER                             # NumberExpr
  | 'true'                             # TrueExpr
  | 'false'                            # FalseExpr
```

```
  | '(' expr ')'                        # ParenExpr
  ;

funcCall
  : IDENT '(' (expr (',' expr)*)? ')'
  ;

mapEnumExpr
  : 'MAP_ENUM' '(' expr ')' '{' enumMapEntry+ enumDefault? '}'
  ;

enumMapEntry
  : STRING '->' STRING ';'
  ;

enumDefault
  : 'DEFAULT' '->' STRING ('WARN' STRING)? ';'
  ;

path
  : IDENT ('.' IDENT)*
  ;

qname
  : IDENT ('.' IDENT)+
  ;

IDENT  : [A-Za-z_][A-Za-z0-9_]* ;
STRING : '"' (~["\\] | '\\' .)* '"' ;
NUMBER : '-'? [0-9]+ ('.' [0-9]+)? ;
WS     : [ \t\r\n]+ -> skip ;
COMMENT: '//' ~[\r\n]* -> skip ;
```

Design-Tipp

Für den Start **keine vollständige Ausdrucksgrammatik mit perfekter Präzedenz** bauen, wenn du schnell vorankommen willst. Du kannst auch zuerst:

- nur Funktionsaufrufe
- Pfade
- Literale
- einfache Vergleiche

unterstützen.

---

4) AST- / Node-Design in Truffle (Java)

Truffle arbeitet AST-basiert; `RootNode` ist der Einstiegspunkt einer ausführbaren Einheit. GraalVM +1

Für dich sind die wichtigsten Ebenen:

A. Sprache + Kontext

Java

```java
@TruffleLanguage.Registration(
    id = "ilimap",
    name = "INTERLIS Mapping DSL",
    version = "0.1",
    characterMimeTypes = "application/x-ilimap"
)
public final class IliMapLanguage extends TruffleLanguage<MigrationContext> {

    @Override
    protected MigrationContext createContext(Env env) {
        return new MigrationContext(env);
    }
```

```
    @Override
    protected CallTarget parse(ParsingRequest request) throws Exception {
        String src = request.getSource().getCharacters().toString();

        // 1) ANTLR parse -> AST model (plain Java)
        MappingAst ast = MappingParserFacade.parse(src);

        // 2) Semantic checks against schema registry (A/B)
        // 3) Build Truffle nodes
        MigrationRootNode root = MigrationNodeFactory.build(this, ast, request.getSource(

        return root.getCallTarget();
    }
}
```

B. RootNode + Ausführungsrahmen

Java

```
public final class MigrationRootNode extends RootNode {
    @Child private ProgramNode programNode;

    public MigrationRootNode(TruffleLanguage<?> lang, FrameDescriptor fd, ProgramNode pro
        super(lang, fd);
        this.programNode = programNode;
    }

    @Override
    public Object execute(VirtualFrame frame) {
        MigrationContext ctx = lookupContextReference(IliMapLanguage.class).get();
        return programNode.execute(frame, ctx);
    }
}
```

5) Node-Hierarchie (konkret)

Basis-Klassen

Java

```java
public abstract class IliMapNode extends Node {
    public SourceSection getSourceSection() { ... } // wichtig für Debug/Reports
}

public abstract class ExprNode extends IliMapNode {
    public abstract Object executeGeneric(VirtualFrame frame, EvalContext ctx);

    public boolean executeBoolean(...) { ... }
    public String executeString(...) { ... }
    public BigDecimal executeNumber(...) { ... }
}

public abstract class StmtNode extends IliMapNode {
    public abstract void executeVoid(VirtualFrame frame, EvalContext ctx);
}
```

Programm- und Regel-Nodes

Java

```java
public final class ProgramNode extends IliMapNode {
    @Children private final ClassRuleNode[] classRules;

    public MigrationResult execute(VirtualFrame frame, MigrationContext ctx) { ... }
}
```

```java
public final class ClassRuleNode extends IliMapNode {
    private final String sourceClassQName;
    private final String targetClassQName;

    @Child private ExprNode whenExpr;                // optional
    @Children private final StmtNode[] statements;
    @Child private KeyMappingNode keyMapping;

    public void executeForObject(VirtualFrame frame, EvalContext ectx, SourceObject src)
}
```

Zuweisung / Assertions / Referenzen

Java

```java
public final class SetStmtNode extends StmtNode {
    private final TargetPath targetPath;
    @Child private ExprNode valueExpr;

    @Override
    public void executeVoid(VirtualFrame frame, EvalContext ctx) {
        Object value = valueExpr.executeGeneric(frame, ctx);
        ctx.currentTarget().write(targetPath, value);
        ctx.report().traceAssignment(targetPath, value, this);
    }
}

public final class AssertStmtNode extends StmtNode {
    @Child private ExprNode condition;
    private final Severity severity;
    private final String message;

    @Override
    public void executeVoid(VirtualFrame frame, EvalContext ctx) {
```

```
        if (!condition.executeBoolean(frame, ctx)) {
            ctx.report().issue(severity, message, ctx.currentSource(), this);
        }
    }
}

public final class RefStmtNode extends StmtNode {
    private final TargetPath targetRefPath;
    private final String targetSourceClassForLookup; // A.Eigentuemer
    @Child private ExprNode sourceRefExpr; // source.eigentuemer_ref

    @Override
    public void executeVoid(VirtualFrame frame, EvalContext ctx) {
        Object sourceRef = sourceRefExpr.executeGeneric(frame, ctx);

        // NICHT sofort setzen -> deferred resolution
        ctx.deferredRefs().add(new DeferredRef(
            ctx.currentTargetObjectId(),
            targetRefPath,
            targetSourceClassForLookup,
            String.valueOf(sourceRef),
            this
        ));
    }
}
```

Expressions (Auszug)

- `SourcePathReadNode`
- `TargetPathReadNode`
- `LiteralNode`
- `BinaryOpNode`
- `FunctionCallNode`

- `MapEnumExprNode`
- `VarReadNode` (für `LET` -Variablen)

---

6) Runtime-Datenmodell (entscheidend für Robustheit)

Hier passiert der eigentliche Zauber. Ich würde das **strictly getrennt** halten von Truffle-Nodes.

`MigrationContext` (global, Engine-lifetime)

- `SchemaRegistry schemaA, schemaB`
- `HelperFunctionRegistry helpers`
- `MigrationOptions options`
- `MigrationReport report`
- `ObjectFactory targetFactory`
- `IdMapRegistry idMaps`
- `ReferenceResolver referenceResolver`

`EvalContext` (pro Ausführung / pro Objekt)

- `SourceRecord currentSource`
- `TargetRecord currentTarget`
- `VariableScope vars` (`LET`)
- `MigrationReport report`
- Zugriff auf `MigrationContext`

`IdMapRegistry` (kritisch)

Enthält die Abbildung:

- `Map<ClassKey, Map<String /*sourceTid*/, String /*targetTid*/>>`

Beispiel:

- `("A.Eigentuemer" -> "B.Owner"): "OID123" -> "OID123"`
- bei Neugenerierung evtl. `"OID123" -> "OWN-000987"`

`DeferredRefStore`

Liste von Referenzen, die erst in Pass 2 gesetzt werden:

```java
record DeferredRef(
    String targetObjectTid,
    TargetPath targetRefPath,
    String sourceClassQNameForLookup,
    String sourceRefTid,
    Node originNode
) {}
```

---

7) Execution-Plan (2-Pass, konkret)

Das ist der Kern für referenzsichere Migrationen.

Pass 0 — Vorbereitung

- Modelle A/B laden (via bestehender Toolchain / Metamodell-Export)
- DSL parsen
- **Semantikcheck**
  - Klassen/Attribute vorhanden?

- Datentypen kompatibel?
- Enum-Mapping plausibel?
- Referenzzielklasse existiert?
- Pflichtfelder potenziell gesetzt?

Pass 1 — Objekte erzeugen + Attribute setzen + ID-Maps aufbauen

Für jede `CLASS` -Regel:

1. Source-Objekte der Klasse aus A lesen
2. `WHEN` prüfen
3. Target-Objekt erzeugen
4. KEY-Regel anwenden (`source.tid -> target.tid`)
5. ID-Map registrieren
6. `SET` , `LET` , `ASSERT` ausführen
7. `REF` **`...`** `RESOLVE` **`...`** **nur als DeferredRef vormerken**
8. Target-Objekt persistieren (oder Buffer)

Pass 2 — Referenzen auflösen

Für jede `DeferredRef` :

1. `sourceRefTid` aus Sourcewelt nehmen
2. In passender ID-Map nach Ziel-TID suchen
3. gefunden → `targetRefPath` setzen
4. nicht gefunden → Error/Warn gemäß `OPTIONS`

Pass 3 — Abschluss / Validierung / Reports

- Zählungen (migriert, gefiltert, Warnungen, Fehler)

- Coverage (welche Regeln/Branches trafen?)
- optional: Export XTF / DB
- optional: externer Validator gegen Zielmodell

---

8) Semantik-Checks (statisch, vor Ausführung)

Das macht dein Tool „enterprise-tauglich“.

Beispiele (sehr nützlich)

- `CLASS A.Gebaeude -> B.Building` : Klassen existieren?
- `target.address.street` : Pfad im Zielmodell gültig?
- `source.bezeichnung` : Quellpfad gültig?
- `MAP_ENUM` : sind gemappte Zielwerte in Ziel-Domain erlaubt?
- `ASSERT target.area_m2 >= 0` : Typen numerisch?
- `REF ... RESOLVE A.Eigentuemer BY source.eigentuemer_ref` : Referenzquelle typ-kompatibel?
- Ziel-Pflichtattribute: gibt es Pfade, die nie gesetzt werden können?

Bonus-Check (sehr stark)

Enum-Abdeckungsprüfung gegen Domain aus Modell A:

- Falls Quell-Enum vollständig bekannt ist, prüfe, ob alle Werte im `MAP_ENUM` abgedeckt sind.
- Sonst markiere Regel als „DEFAULT-abhängig“.

---

9) Instrumentierung mit Truffle (Explainability)

Truffle bietet Instrumentierung für Tools/Debugger/Profiler über `TruffleInstrument`. GraalVM +2

Für dein Produkt heißt das: **Migration nachvollziehbar machen**.

Was du instrumentieren solltest

- **RuleStart / RuleEnd**
- **Assignment**
- **AssertFailed**
- **EnumFallbackUsed**
- **DeferredRefCreated**
- **DeferredRefResolved / Missing**

Beispiel-Nutzen

Für ein Zielobjekt `B.Building:TID=4711` kannst du ausgeben:

- `name` gesetzt durch Regel `SET target.name = normalize_text(source.bezeichnung)`
- `usage` gesetzt via `MAP_ENUM`, Branch `"Misch" -> "Mixed"`
- `owner_ref` in Pass 2 aufgelöst aus `A.Eigentuemer:OID123`
- Warnung: unbekannter Enumwert → `Unknown`

Das ist **Auditfähigkeit**, nicht nur Logging.

---

10) Beispiel-Ende-zu-Ende (A → B) inkl. Enum, Split, Referenz

Modell A (vereinfacht, konzeptuell)

- `A.Gebaeude`
  - `tid`

    - bezeichnung
    - adresse_text (z. B. "Bahnhofstrasse 12")
    - nutzung (Wohnen|Gewerbe|Misch)
    - flaeche
    - flaeche_einheit (m2|ha)
    - eigentuemer_ref (Ref auf A.Eigentuemer)
    - status
- A.Eigentuemer
    - tid
    - vollname
    - typ

Modell B

- B.Building
    - tid
    - name
    - address.street
    - address.number
    - usage
    - area_m2
    - owner_ref (Ref auf B.Owner)
- B.Owner
    - tid
    - first_name
    - last_name

- kind

Beispiel-Daten A

- A.Eigentuemer
  - tid=E1 , vollname="Max Muster" , typ="Privatperson"
- A.Gebaeude
  - tid=G1 , bezeichnung="Haus Nord" , adresse_text="Bahnhofstrasse 12" , nutzung="Misch" , flaeche=0.15 , flaeche_einheit="ha" , eigentuemer_ref="E1" , status="AKTIV"

Ergebnis nach Migration

- B.Owner
  - tid=E1
  - first_name="Max"
  - last_name="Muster"
  - kind="Person"
- B.Building
  - tid=G1
  - name="Haus Nord"
  - address.street="Bahnhofstrasse"
  - address.number="12"
  - usage="Mixed"
  - area_m2=1500
  - owner_ref="E1"

---

11) Konkretes Java-Datenmodell für Source/Target-Records

Damit du nicht direkt an XTF/DB-Objekte gekoppelt bist, empfehle ich ein neutrales Record-Modell:

```java
public interface SourceRecord {
    String classQName();     // z.B. A.Gebaeude
    String tid();
    Object read(Path path);  // "bezeichnung", "adresse_text", ...
}

public interface TargetRecord {
    String classQName();
    String tid();
    void setTid(String tid);
    Object read(Path path);
    void write(Path path, Object value);
}
```

Vorteil

- gleiche Engine für:
  - XTF-Input
  - DB-Input
  - In-Memory Tests
- Adapter-Layer tauschen, DSL bleibt gleich

---

12) Helper-Funktionen (der pragmatische 20%-Teil)

Diese Funktionen stellst du als Java-Registry bereit, und die DSL ruft sie auf:

- `normalize_text(string)`
- `parse_street(string)`
- `parse_house_no(string)`
- `split_name(string)`
- `to_m2(value, unit)`

Beispiel-Registry

Java

```java
public final class HelperFunctionRegistry {
    private final Map<String, MappingFunction> funcs = new HashMap<>();

    public HelperFunctionRegistry register(String name, MappingFunction fn) { ... }
    public Object call(String name, List<Object> args, EvalContext ctx) { ... }
}
```

Wichtig: Helper-Funktionen sollten

- deterministic sein
- sauber Fehler/Warnungen melden
- idealerweise testbar ohne Truffle sein

---

13) Fehler-/Warnmodell (sehr wichtig für echte Projekte)

Nicht alles darf hart abbrechen.

Vorschlag

- **ERROR**: semantisch falsch / Zielmodell verletzt / nicht auflösbare Pflichtreferenz

- **WARN**: unmappbarer Enumwert mit Default, unvollständiger Name, fehlende optionale Referenz
- **INFO**: Filter angewendet, Default gesetzt, Normalisierung durchgeführt

Report-Struktur

- global summary
- pro Klasse counts
- pro Regel hits
- Liste der Issues mit:
  - Severity
  - Message
  - Source class/tid
  - Zielpfad (falls relevant)
  - SourceSection (DSL Zeile/Spalte)

Dank `SourceSection` in Nodes kannst du direkt auf die DSL-Zeile verweisen (wichtig für Debuggability). Truffle-Tools bauen auf solchen Source-Infos auf. GraalVM +1

---

14) MVP in 3 Iterationen (realistisch)

Iteration 1 (2–4 Wochen, je nach Umfang)

- Parser (ANTLR)
- 1:1 Klassenmapping
- `SET`, `LET`, `ASSERT`
- Funktionsaufrufe
- `MAP_ENUM`

- In-Memory Source/Target
- Report

➡️ Schon nützlich für Rename/Enum/Defaults/Split von Attributen.

Iteration 2

- `REF ... RESOLVE ...`
- ID-Map-Registry
- DeferredRef / Pass 2
- Fehlerstrategien (`WARN/ERROR`)

➡️ Jetzt werden echte Modelle mit Beziehungen spannend.

Iteration 3

- Instrumentierung / Rule Trace / Coverage
- bessere Typchecks
- CLI + Dry Run
- XTF-/DB-Adapter

➡️ Aus „Prototype“ wird ein Werkzeug.

---

15) Was ich bewusst noch nicht in den MVP nehmen würde

- Klassen-Merge mit Joins/Gruppierung
- Geometrie-/CRS-Transformationen
- Streaming mit Backpressure
- parallele Ausführung
- UI

Nicht weil es unwichtig ist — sondern weil du zuerst die **Regelsprache + ID/Referenz-Kernlogik** stabil brauchst.

---

16) Nächster sinnvoller Schritt (wenn du willst)

Ich kann dir als Nächstes direkt ein **Mini-Repository-Layout** skizzieren (Gradle/Maven), z. B.:

- `ilimap-parser` (ANTLR + AST)
- `ilimap-semantics` (Typcheck gegen Schema-Metamodell)
- `ilimap-runtime` (Records, IDMaps, Report)
- `ilimap-truffle` (Nodes, Language, Instrument)
- `ilimap-cli` (Dry Run / Execute)

…plus **ein konkretes Java-Skelett** für:

- `ClassRuleNode.executeForObject(...)`
- `MapEnumExprNode`
- `DeferredRefResolver`

Wenn du magst, gehe ich im nächsten Schritt **kompilierbar** runter (Java-Klassen mit Signaturen und Beispieltests).

Sources

Gerne

Thought for 1m 37s ›

Perfekt — dann gehen wir auf **Repo-/Code-Ebene** runter.

Ich gebe dir jetzt ein **konkretes Mini-Repository-Layout** plus **Java-Skelette** für die Kernteile (inkl. Tests), so dass du daraus wirklich einen MVP aufbauen kannst.

Truffle-Sprachen werden über `TruffleLanguage` (inkl. `@TruffleLanguage.Registration`) registriert; Truffle-Tools/Instrumentation analog über `TruffleInstrument` und `@TruffleInstrument.Registration`. Das passt genau zu deinem Vorhaben (DSL + Explainability).

graalvm.org +2

Für INTERLIS bleibt der pragmatische Weg: **Modelle mit bestehender Toolchain (z. B. ili2c) prüfen/auswerten**, Migration in deiner DSL ausführen. ili2c ist der INTERLIS-Compiler. docs.interlis.ch +1

Zielbild für den MVP

- **Parser/AST** für deine Mapping-DSL
- **Semantikcheck** gegen A/B-Schema-Metamodell
- **Runtime** mit:
  - `SET`, `LET`, `ASSERT`
  - `MAP_ENUM`
  - `REF ... RESOLVE ...`
  - **ID-Map + Deferred Reference Resolution (Pass 2)**
- **Truffle-Engine** für Ausführung
- **CLI** für `dry-run` / `run`
- **JUnit-Tests** für Kernlogik

1) Repository-Layout (Gradle Multi-Module)

Plain text

```
ilimap/
├─ settings.gradle.kts
├─ build.gradle.kts
├─ gradle.properties
│
├─ ilimap-model/                # Schema-Metamodell (A/B), unabhängig von Truffle
│  └─ src/main/java/...
│
├─ ilimap-parser/               # ANTLR + AST (Plain Java AST)
│  ├─ src/main/antlr/InterlisMap.g4
│  └─ src/main/java/...
│
├─ ilimap-semantics/            # Typchecks / Regelchecks gegen Schema-Metamodell
│  └─ src/main/java/...
│
├─ ilimap-runtime/              # Records, IDMaps, Report, Resolver, Helper-Funktionen
│  ├─ src/main/java/...
│  └─ src/test/java/...
│
├─ ilimap-truffle/              # TruffleLanguage + Nodes + Instrument
│  ├─ src/main/java/...
│  └─ src/test/java/...
│
├─ ilimap-adapter-interlis/     # Adapter: ili2c/IMD/XTF/DB -> Schema/Records (später)
│  └─ src/main/java/...
│
└─ ilimap-cli/                  # Kommandozeile
   └─ src/main/java/...
```

2) Gradle-Grundgerüst (kompakt, aber brauchbar)

`settings.gradle.kts`

Kotlin

```kotlin
rootProject.name = "ilimap"

include(
    "ilimap-model",
    "ilimap-parser",
    "ilimap-semantics",
    "ilimap-runtime",
    "ilimap-truffle",
    "ilimap-adapter-interlis",
    "ilimap-cli"
)
```

`build.gradle.kts` (Root)

Kotlin

```kotlin
plugins {
    base
}

allprojects {
    group = "ch.example.ilimap"
    version = "0.1.0-SNAPSHOT"
}

subprojects {
```

```
    apply(plugin = "java-library")

    repositories {
        mavenCentral()
    }

    java {
        toolchain {
            languageVersion.set(JavaLanguageVersion.of(21))
        }
    }

    tasks.withType<Test> {
        useJUnitPlatform()
    }

    dependencies {
        testImplementation("org.junit.jupiter:junit-jupiter:5.10.2")
        testRuntimeOnly("org.junit.platform:junit-platform-launcher")
    }
}
```

Beispiel `ilimap-truffle/build.gradle.kts`

Kotlin

```
dependencies {
    api(project(":ilimap-runtime"))
    api(project(":ilimap-semantics"))
    implementation(project(":ilimap-parser"))
    implementation(project(":ilimap-model"))

    implementation("org.graalvm.truffle:truffle-api:25.0.2")
    compileOnly("org.graalvm.truffle:truffle-dsl-processor:25.0.2")
```

```
    annotationProcessor("org.graalvm.truffle:truffle-dsl-processor:25.0.2")
}
```

Beispiel `ilimap-parser/build.gradle.kts`

Kotlin

```kotlin
plugins {
    antlr
}

dependencies {
    implementation(project(":ilimap-model"))
    antlr("org.antlr:antlr4:4.13.1")
    implementation("org.antlr:antlr4-runtime:4.13.1")
}

tasks.generateGrammarSource {
    maxHeapSize = "128m"
    arguments += listOf("-visitor", "-long-messages")
}
```

Hinweis: Versions-Pinning für Truffle/GraalVM solltest du mit deiner lokalen GraalVM-Version synchron halten.

---

3) Kern-Domänenmodell (schema-agnostisch)

`ilimap-model` : Schema-Metamodell (minimal)

Java

```java
package ch.example.ilimap.model;

import java.util.*;

public final class SchemaRegistry {
    private final Map<String, ClassSchema> classes = new HashMap<>();

    public void register(ClassSchema schema) {
        classes.put(schema.qName(), schema);
    }

    public Optional<ClassSchema> findClass(String qName) {
        return Optional.ofNullable(classes.get(qName));
    }

    public ClassSchema requireClass(String qName) {
        return findClass(qName).orElseThrow(() -> new IllegalArgumentException("Unknown c
    }
}
```

Java

```java
package ch.example.ilimap.model;

import java.util.*;

public record ClassSchema(
        String qName,
        Map<String, AttributeSchema> attributes,
        Set<String> requiredAttributes
) {
    public Optional<AttributeSchema> findAttribute(String path) {
```

```
        return Optional.ofNullable(attributes.get(path));
    }
}
```

‹/› Java

```java
package ch.example.ilimap.model;

import java.util.Set;

public record AttributeSchema(
        String path,
        ValueType type,
        Set<String> enumValues,     // nur falls type == ENUM
        boolean reference,
        String referenceTargetClass // nur falls reference=true
) {
}
```

‹/› Java

```java
package ch.example.ilimap.model;

public enum ValueType {
    STRING,
    NUMBER,
    BOOLEAN,
    ENUM,
    REFERENCE,
    OBJECT,
    NULLABLE_STRING,
    NULLABLE_NUMBER
}
```

4) Runtime (das Rückgrat deines Migrationsrunners)

`Path`, `SourceRecord`, `TargetRecord`

Java

```java
package ch.example.ilimap.runtime;

import java.util.List;

public record Path(List<String> parts) {
    public static Path of(String dotted) {
        return new Path(List.of(dotted.split("\\.")));
    }

    @Override
    public String toString() {
        return String.join(".", parts);
    }
}
```

Java

```java
package ch.example.ilimap.runtime;

public interface SourceRecord {
    String classQName();
    String tid();
    Object read(Path path);
}
```

Java

```java
package ch.example.ilimap.runtime;

public interface TargetRecord {
    String classQName();
    String tid();
    void setTid(String tid);
    Object read(Path path);
    void write(Path path, Object value);
}
```

In-Memory-Implementierung (für Tests + MVP)

Java

```java
package ch.example.ilimap.runtime;

import java.util.HashMap;
import java.util.Map;

public final class MapRecord implements SourceRecord, TargetRecord {
    private final String classQName;
    private String tid;
    private final Map<String, Object> values = new HashMap<>();

    public MapRecord(String classQName, String tid) {
        this.classQName = classQName;
        this.tid = tid;
    }

    public MapRecord put(String path, Object value) {
        values.put(path, value);
        return this;
```

```
    }

    @Override
    public String classQName() { return classQName; }

    @Override
    public String tid() { return tid; }

    @Override
    public void setTid(String tid) { this.tid = tid; }

    @Override
    public Object read(Path path) {
        return values.get(path.toString());
    }

    @Override
    public void write(Path path, Object value) {
        values.put(path.toString(), value);
    }

    public Map<String, Object> snapshot() {
        return Map.copyOf(values);
    }
}
```

ID-Map-Registry (kritisch)

Java

```
package ch.example.ilimap.runtime;
```

```
import java.util.HashMap;
import java.util.Map;
import java.util.Optional;

public final class IdMapRegistry {

    private final Map<String, Map<String, String>> maps = new HashMap<>();
    // key = "A.Eigentuemer->B.Owner"

    private static String key(String sourceClass, String targetClass) {
        return sourceClass + "->" + targetClass;
    }

    public void put(String sourceClass, String targetClass, String sourceTid, String targ
        maps.computeIfAbsent(key(sourceClass, targetClass), k -> new HashMap<>())
            .put(sourceTid, targetTid);
    }

    public Optional<String> resolve(String sourceClass, String targetClass, String source
        var byClass = maps.get(key(sourceClass, targetClass));
        if (byClass == null) return Optional.empty();
        return Optional.ofNullable(byClass.get(sourceTid));
    }
}
```

Deferred References + Resolver (Pass 2)

Java

```
package ch.example.ilimap.runtime;

public record DeferredRef(
```

```
        String targetObjectClass,
        String targetObjectTid,
        Path targetRefPath,
        String sourceRefClass,
        String targetRefClass,
        String sourceRefTid
) {}
```

‹/› Java

```java
package ch.example.ilimap.runtime;

import java.util.ArrayList;
import java.util.List;

public final class DeferredRefStore {
    private final List<DeferredRef> refs = new ArrayList<>();

    public void add(DeferredRef ref) {
        refs.add(ref);
    }

    public List<DeferredRef> all() {
        return List.copyOf(refs);
    }

    public void clear() {
        refs.clear();
    }
}
```

‹/› Java

```
package ch.example.ilimap.runtime;

import java.util.Map;

public final class DeferredRefResolver {

    private final IdMapRegistry idMaps;
    private final RecordRepository targetRepo;
    private final MigrationReport report;
    private final MissingReferencePolicy policy;

    public DeferredRefResolver(IdMapRegistry idMaps,
                               RecordRepository targetRepo,
                               MigrationReport report,
                               MissingReferencePolicy policy) {
        this.idMaps = idMaps;
        this.targetRepo = targetRepo;
        this.report = report;
        this.policy = policy;
    }

    public void resolveAll(Iterable<DeferredRef> refs) {
        for (DeferredRef ref : refs) {
            var targetTidOpt = idMaps.resolve(ref.sourceRefClass(), ref.targetRefClass(),
            var targetObjOpt = targetRepo.findTarget(ref.targetObjectClass(), ref.target(

            if (targetObjOpt.isEmpty()) {
                report.error("Target object for deferred ref not found: " + ref.targetObj
                continue;
            }

            TargetRecord targetObj = targetObjOpt.get();

            if (targetTidOpt.isPresent()) {
                targetObj.write(ref.targetRefPath(), targetTidOpt.get());
```

```
                report.info("Resolved ref " + ref.targetObjectClass() + "." + ref.targetF
            } else {
                String msg = "Missing reference mapping: " + ref.sourceRefClass() + ":" +
                        + " -> " + ref.targetRefClass();
                if (policy == MissingReferencePolicy.ERROR) {
                    report.error(msg);
                } else {
                    report.warn(msg);
                }
            }
        }
    }

    public enum MissingReferencePolicy { WARN, ERROR }
}
```

Repository für Records (MVP: In-Memory)

Java

```
package ch.example.ilimap.runtime;

import java.util.*;
import java.util.stream.Collectors;

public final class RecordRepository {
    private final Map<String, List<SourceRecord>> sourcesByClass = new HashMap<>();
    private final Map<String, Map<String, TargetRecord>> targetsByClassTid = new HashMap<

    public void addSource(SourceRecord source) {
        sourcesByClass.computeIfAbsent(source.classQName(), k -> new ArrayList<>()).add(s
    }
```

```
    public List<SourceRecord> sourcesOfClass(String classQName) {
        return sourcesByClass.getOrDefault(classQName, List.of());
    }

    public void saveTarget(TargetRecord target) {
        targetsByClassTid
                .computeIfAbsent(target.classQName(), k -> new HashMap<>())
                .put(target.tid(), target);
    }

    public Optional<TargetRecord> findTarget(String classQName, String tid) {
        return Optional.ofNullable(targetsByClassTid.getOrDefault(classQName, Map.of()).g
    }

    public List<TargetRecord> targetsOfClass(String classQName) {
        return new ArrayList<>(targetsByClassTid.getOrDefault(classQName, Map.of()).value
    }

    public Map<String, Long> targetCounts() {
        return targetsByClassTid.entrySet().stream()
                .collect(Collectors.toMap(Map.Entry::getKey, e -> (long) e.getValue().siz
    }
}
```

Report / Issues

‹/› Java

```
package ch.example.ilimap.runtime;

import java.util.ArrayList;
```

```
import java.util.List;

public final class MigrationReport {
    private final List<Issue> issues = new ArrayList<>();
    private final List<String> infos = new ArrayList<>();

    public void error(String message) { issues.add(new Issue(Severity.ERROR, message)); }
    public void warn(String message)  { issues.add(new Issue(Severity.WARN, message)); }
    public void info(String message)  { infos.add(message); }

    public List<Issue> issues() { return List.copyOf(issues); }
    public List<String> infos() { return List.copyOf(infos); }

    public long errorCount() { return issues.stream().filter(i -> i.severity() == Severit
    public long warnCount()  { return issues.stream().filter(i -> i.severity() == Severit

    public record Issue(Severity severity, String message) {}
    public enum Severity { ERROR, WARN }
}
```

Helper-Funktionen (Java-Registry)

Java

```
package ch.example.ilimap.runtime;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

public final class HelperFunctionRegistry {
```

```
    @FunctionalInterface
    public interface MappingFunction {
        Object apply(List<Object> args, EvalContext ctx);
    }

    private final Map<String, MappingFunction> functions = new HashMap<>();

    public HelperFunctionRegistry register(String name, MappingFunction fn) {
        functions.put(name, fn);
        return this;
    }

    public Object call(String name, List<Object> args, EvalContext ctx) {
        MappingFunction fn = functions.get(name);
        if (fn == null) {
            throw new IllegalArgumentException("Unknown helper function: " + name);
        }
        return fn.apply(args, ctx);
    }
}
```

Java

```
package ch.example.ilimap.runtime;

import java.util.HashMap;
import java.util.Map;

public final class EvalContext {
    private final MigrationContext migrationContext;
    private final SourceRecord currentSource;
    private final TargetRecord currentTarget;
    private final Map<String, Object> vars = new HashMap<>();

    public EvalContext(MigrationContext migrationContext, SourceRecord currentSource, Tar
```

```
        this.migrationContext = migrationContext;
        this.currentSource = currentSource;
        this.currentTarget = currentTarget;
    }

    public MigrationContext migration() { return migrationContext; }
    public SourceRecord source() { return currentSource; }
    public TargetRecord target() { return currentTarget; }
    public Map<String, Object> vars() { return vars; }
}
```

‹/› Java

```
package ch.example.ilimap.runtime;

import ch.example.ilimap.model.SchemaRegistry;

public final class MigrationContext {
    private final SchemaRegistry schemaA;
    private final SchemaRegistry schemaB;
    private final RecordRepository records;
    private final IdMapRegistry idMaps;
    private final DeferredRefStore deferredRefs;
    private final HelperFunctionRegistry helpers;
    private final MigrationReport report;

    public MigrationContext(SchemaRegistry schemaA,
                            SchemaRegistry schemaB,
                            RecordRepository records,
                            IdMapRegistry idMaps,
                            DeferredRefStore deferredRefs,
                            HelperFunctionRegistry helpers,
                            MigrationReport report) {
        this.schemaA = schemaA;
        this.schemaB = schemaB;
```

```
        this.records = records;
        this.idMaps = idMaps;
        this.deferredRefs = deferredRefs;
        this.helpers = helpers;
        this.report = report;
    }

    public SchemaRegistry schemaA() { return schemaA; }
    public SchemaRegistry schemaB() { return schemaB; }
    public RecordRepository records() { return records; }
    public IdMapRegistry idMaps() { return idMaps; }
    public DeferredRefStore deferredRefs() { return deferredRefs; }
    public HelperFunctionRegistry helpers() { return helpers; }
    public MigrationReport report() { return report; }
}
```

---

5) Parser-Modul: AST (Plain Java, kein Truffle)

Hier liegt die DSL als **neutraler AST**. Das ist wichtig: Parser/Grammatik und Truffle-Engine bleiben getrennt.

AST-Records (Auszug)

Java

```
package ch.example.ilimap.parser.ast;

import java.util.List;

public record MigrationAst(
        String sourceModel,
        String targetModel,
```

```
        List<ClassRuleAst> classRules
) {}
```

Java

```java
package ch.example.ilimap.parser.ast;

import java.util.List;

public record ClassRuleAst(
        String sourceClass,
        String targetClass,
        KeyMappingAst keyMapping,
        ExprAst whenExpr,                // null erlaubt
        List<StmtAst> statements
) {}
```

Java

```java
package ch.example.ilimap.parser.ast;

public record KeyMappingAst(String sourcePath, String targetPath) {}
```

Java

```java
package ch.example.ilimap.parser.ast;

public sealed interface StmtAst permits SetStmtAst, LetStmtAst, AssertStmtAst, RefStmtAst
```

Java

```
package ch.example.ilimap.parser.ast;

public record SetStmtAst(String targetPath, ExprAst expr) implements StmtAst {}
```

‹/› Java

```
package ch.example.ilimap.parser.ast;

public record LetStmtAst(String varName, ExprAst expr) implements StmtAst {}
```

‹/› Java

```
package ch.example.ilimap.parser.ast;

public record AssertStmtAst(ExprAst condition, SeverityAst severity, String message) impl
    public enum SeverityAst { WARN, ERROR }
}
```

‹/› Java

```
package ch.example.ilimap.parser.ast;

public record RefStmtAst(
        String targetRefPath,
        String sourceRefClass,   // A.Eigentuemer
        String sourceRefExprPath // source.eigentuemer_ref
) implements StmtAst {}
```

‹/› Java

```
package ch.example.ilimap.parser.ast;
```

```
public sealed interface ExprAst permits
        LiteralAst, SourcePathAst, TargetPathAst, VarAst, BinaryExprAst, FunctionCallAst,
```

(weitere Expr-ASTs analog)

Der Parser (`MappingParserFacade`) baut nur diesen AST. **Noch keine Ausführung**.

---

6) Semantikchecker (vor Truffle)

Das ist der beste Ort für **harte Fehler vor Laufzeit**.

`SemanticChecker` (Skelett)

Java

```java
package ch.example.ilimap.semantics;

import ch.example.ilimap.model.SchemaRegistry;
import ch.example.ilimap.parser.ast.*;

import java.util.ArrayList;
import java.util.List;

public final class SemanticChecker {
    private final SchemaRegistry schemaA;
    private final SchemaRegistry schemaB;

    public SemanticChecker(SchemaRegistry schemaA, SchemaRegistry schemaB) {
        this.schemaA = schemaA;
        this.schemaB = schemaB;
    }
```

```
    public SemanticCheckResult check(MigrationAst ast) {
        List<String> errors = new ArrayList<>();
        List<String> warnings = new ArrayList<>();

        for (ClassRuleAst rule : ast.classRules()) {
            if (schemaA.findClass(rule.sourceClass()).isEmpty()) {
                errors.add("Unknown source class: " + rule.sourceClass());
            }
            if (schemaB.findClass(rule.targetClass()).isEmpty()) {
                errors.add("Unknown target class: " + rule.targetClass());
            }

            // TODO:
            // - check source/target paths in statements
            // - enum value validity in MAP_ENUM
            // - reference target compatibility
            // - required target attributes coverage (best effort)
        }

        return new SemanticCheckResult(errors, warnings);
    }

    public record SemanticCheckResult(List<String> errors, List<String> warnings) {
        public boolean isOk() { return errors.isEmpty(); }
    }
}
```

---

7) Truffle-Modul: Sprache + RootNode + Nodes

`IliMapLanguage`

Java

```java
package ch.example.ilimap.truffle;

import ch.example.ilimap.parser.MappingParserFacade;
import ch.example.ilimap.parser.ast.MigrationAst;
import ch.example.ilimap.runtime.MigrationContext;
import ch.example.ilimap.semantics.SemanticChecker;
import com.oracle.truffle.api.CallTarget;
import com.oracle.truffle.api.TruffleLanguage;
import com.oracle.truffle.api.source.Source;

@TruffleLanguage.Registration(
        id = IliMapLanguage.ID,
        name = "INTERLIS Mapping DSL",
        version = "0.1.0",
        characterMimeTypes = IliMapLanguage.MIME_TYPE
)
public final class IliMapLanguage extends TruffleLanguage<MigrationContext> {
    public static final String ID = "ilimap";
    public static final String MIME_TYPE = "application/x-ilimap";

    public IliMapLanguage() {
    }

    @Override
    protected MigrationContext createContext(Env env) {
        // Im echten Betrieb Kontext über Launcher/Polyglot bindings injizieren.
        // Für MVP: ContextHolder / Factory verwenden.
        return MigrationContextHolder.require();
    }

    @Override
    protected CallTarget parse(ParsingRequest request) throws Exception {
        Source source = request.getSource();
```

```
        String code = source.getCharacters().toString();

        MigrationAst ast = MappingParserFacade.parse(code);

        MigrationContext ctx = MigrationContextHolder.peek();
        if (ctx != null) {
            var result = new SemanticChecker(ctx.schemaA(), ctx.schemaB()).check(ast);
            if (!result.isOk()) {
                throw new IllegalArgumentException("Semantic errors: " + result.errors())
            }
            result.warnings().forEach(ctx.report()::warn);
        }

        MigrationRootNode root = MigrationNodeFactory.build(this, ast, source);
        return root.getCallTarget();
    }
}
```

`@TruffleLanguage.Registration` ist der Standardweg, damit die Sprache über die Polyglot-API discoverbar ist. graalvm.org +1

MigrationRootNode

Java

```java
package ch.example.ilimap.truffle;

import ch.example.ilimap.runtime.MigrationContext;
import com.oracle.truffle.api.frame.VirtualFrame;
import com.oracle.truffle.api.nodes.RootNode;

public final class MigrationRootNode extends RootNode {
```

```
    @Child private ProgramNode programNode;

    public MigrationRootNode(IliMapLanguage language, ProgramNode programNode) {
        super(language);
        this.programNode = programNode;
    }

    @Override
    public Object execute(VirtualFrame frame) {
        MigrationContext ctx = MigrationContextHolder.require();
        return programNode.execute(frame, ctx);
    }
}
```

Basis-Nodes

Java

```
package ch.example.ilimap.truffle.nodes;

import ch.example.ilimap.runtime.EvalContext;
import com.oracle.truffle.api.frame.VirtualFrame;
import com.oracle.truffle.api.nodes.Node;

public abstract class ExprNode extends Node {
    public abstract Object executeGeneric(VirtualFrame frame, EvalContext ctx);

    public boolean executeBoolean(VirtualFrame frame, EvalContext ctx) {
        Object v = executeGeneric(frame, ctx);
        if (v instanceof Boolean b) return b;
        throw new IllegalStateException("Expected boolean, got: " + v);
    }
```

```
    public String executeString(VirtualFrame frame, EvalContext ctx) {
        Object v = executeGeneric(frame, ctx);
        return v == null ? null : String.valueOf(v);
    }
}
```

Java

```
package ch.example.ilimap.truffle.nodes;

import ch.example.ilimap.runtime.EvalContext;
import com.oracle.truffle.api.frame.VirtualFrame;
import com.oracle.truffle.api.nodes.Node;

public abstract class StmtNode extends Node {
    public abstract void executeVoid(VirtualFrame frame, EvalContext ctx);
}
```

`ProgramNode` (Pass 1 + Pass 2)

Java

```
package ch.example.ilimap.truffle;

import ch.example.ilimap.runtime.*;
import com.oracle.truffle.api.frame.VirtualFrame;
import com.oracle.truffle.api.nodes.Node;

import java.util.List;
```

```java
public final class ProgramNode extends Node {
    @Children private final ClassRuleNode[] classRules;

    public ProgramNode(List<ClassRuleNode> classRules) {
        this.classRules = classRules.toArray(ClassRuleNode[]::new);
    }

    public MigrationReport execute(VirtualFrame frame, MigrationContext ctx) {
        // Pass 1: Klassennormen anwenden, Targets erzeugen, ID-Maps + DeferredRefs samme
        for (ClassRuleNode rule : classRules) {
            rule.executePass1(frame, ctx);
        }

        // Pass 2: Deferred refs auflösen
        DeferredRefResolver resolver = new DeferredRefResolver(
                ctx.idMaps(),
                ctx.records(),
                ctx.report(),
                DeferredRefResolver.MissingReferencePolicy.WARN
        );
        resolver.resolveAll(ctx.deferredRefs().all());

        return ctx.report();
    }
}
```

`ClassRuleNode` (wichtigste Ausführungsmethode)

Java

```java
package ch.example.ilimap.truffle;
```

```java
import ch.example.ilimap.runtime.*;
import ch.example.ilimap.truffle.nodes.ExprNode;
import ch.example.ilimap.truffle.nodes.StmtNode;
import com.oracle.truffle.api.frame.VirtualFrame;
import com.oracle.truffle.api.nodes.Node;

public final class ClassRuleNode extends Node {
    private final String sourceClass;
    private final String targetClass;
    private final Path keySourcePath;
    private final Path keyTargetPath;

    @Child private ExprNode whenExpr; // nullable
    @Children private final StmtNode[] statements;

    public ClassRuleNode(String sourceClass,
                         String targetClass,
                         Path keySourcePath,
                         Path keyTargetPath,
                         ExprNode whenExpr,
                         StmtNode[] statements) {
        this.sourceClass = sourceClass;
        this.targetClass = targetClass;
        this.keySourcePath = keySourcePath;
        this.keyTargetPath = keyTargetPath;
        this.whenExpr = whenExpr;
        this.statements = statements;
    }

    public void executePass1(VirtualFrame frame, MigrationContext mctx) {
        for (SourceRecord src : mctx.records().sourcesOfClass(sourceClass)) {
            // temporäres Zielobjekt erzeugen
            TargetRecord target = new MapRecord(targetClass, "TEMP-" + src.tid());
            EvalContext ectx = new EvalContext(mctx, src, target);
```

```
            if (whenExpr != null && !whenExpr.executeBoolean(frame, ectx)) {
                mctx.report().info("Filtered out " + sourceClass + ":" + src.tid());
                continue;
            }

            // KEY anwenden (hier simpel: source.tid -> target.tid oder source.<pfad>)
            Object keyValue = "tid".equals(keySourcePath.toString()) ? src.tid() : src.re
            if (keyValue == null) {
                mctx.report().error("Missing key value for " + sourceClass + ":" + src.ti
                continue;
            }

            String targetTid = String.valueOf(keyValue);
            target.setTid(targetTid);

            // Target-Keystrecke optional auch schreiben (z.B. target.tid)
            if (!"tid".equals(keyTargetPath.toString())) {
                target.write(keyTargetPath, targetTid);
            }

            // ID-Map registrieren
            mctx.idMaps().put(sourceClass, targetClass, src.tid(), targetTid);

            // Statements ausführen
            for (StmtNode stmt : statements) {
                stmt.executeVoid(frame, ectx);
            }

            // persistieren
            mctx.records().saveTarget(target);
            mctx.report().info("Migrated " + sourceClass + ":" + src.tid() + " -> " + tar
        }
    }
}
```

SetStmtNode, LetStmtNode, AssertStmtNode, RefStmtNode

Java

```java
package ch.example.ilimap.truffle.nodes.stmt;

import ch.example.ilimap.runtime.EvalContext;
import ch.example.ilimap.runtime.Path;
import ch.example.ilimap.truffle.nodes.ExprNode;
import ch.example.ilimap.truffle.nodes.StmtNode;
import com.oracle.truffle.api.frame.VirtualFrame;

public final class SetStmtNode extends StmtNode {
    private final Path targetPath;
    @Child private ExprNode valueExpr;

    public SetStmtNode(Path targetPath, ExprNode valueExpr) {
        this.targetPath = targetPath;
        this.valueExpr = valueExpr;
    }

    @Override
    public void executeVoid(VirtualFrame frame, EvalContext ctx) {
        Object value = valueExpr.executeGeneric(frame, ctx);
        ctx.target().write(targetPath, value);
    }
}
```

Java

```java
package ch.example.ilimap.truffle.nodes.stmt;
```

```
import ch.example.ilimap.runtime.EvalContext;
import ch.example.ilimap.truffle.nodes.ExprNode;
import ch.example.ilimap.truffle.nodes.StmtNode;
import com.oracle.truffle.api.frame.VirtualFrame;

public final class LetStmtNode extends StmtNode {
    private final String varName;
    @Child private ExprNode expr;

    public LetStmtNode(String varName, ExprNode expr) {
        this.varName = varName;
        this.expr = expr;
    }

    @Override
    public void executeVoid(VirtualFrame frame, EvalContext ctx) {
        ctx.vars().put(varName, expr.executeGeneric(frame, ctx));
    }
}
```

Java

```
package ch.example.ilimap.truffle.nodes.stmt;

import ch.example.ilimap.runtime.EvalContext;
import ch.example.ilimap.runtime.MigrationReport;
import ch.example.ilimap.truffle.nodes.ExprNode;
import ch.example.ilimap.truffle.nodes.StmtNode;
import com.oracle.truffle.api.frame.VirtualFrame;

public final class AssertStmtNode extends StmtNode {
    public enum Severity { WARN, ERROR }

    @Child private ExprNode condition;
    private final Severity severity;
```

```
    private final String message;

    public AssertStmtNode(ExprNode condition, Severity severity, String message) {
        this.condition = condition;
        this.severity = severity;
        this.message = message;
    }

    @Override
    public void executeVoid(VirtualFrame frame, EvalContext ctx) {
        boolean ok = condition.executeBoolean(frame, ctx);
        if (ok) return;

        String msg = message + " @ " + ctx.source().classQName() + ":" + ctx.source().tic
        if (severity == Severity.ERROR) ctx.migration().report().error(msg);
        else ctx.migration().report().warn(msg);
    }
}
```

Java

```
package ch.example.ilimap.truffle.nodes.stmt;

import ch.example.ilimap.runtime.*;
import ch.example.ilimap.truffle.nodes.ExprNode;
import ch.example.ilimap.truffle.nodes.StmtNode;
import com.oracle.truffle.api.frame.VirtualFrame;

public final class RefStmtNode extends StmtNode {
    private final Path targetRefPath;
    private final String sourceRefClass; // z.B. A.Eigentuemer
    private final String targetRefClass; // z.B. B.Owner
    @Child private ExprNode sourceRefTidExpr;

    public RefStmtNode(Path targetRefPath, String sourceRefClass, String targetRefClass,
```

```
        this.targetRefPath = targetRefPath;
        this.sourceRefClass = sourceRefClass;
        this.targetRefClass = targetRefClass;
        this.sourceRefTidExpr = sourceRefTidExpr;
    }

    @Override
    public void executeVoid(VirtualFrame frame, EvalContext ctx) {
        Object refTidObj = sourceRefTidExpr.executeGeneric(frame, ctx);
        if (refTidObj == null) return; // optional ref
        String sourceRefTid = String.valueOf(refTidObj);

        ctx.migration().deferredRefs().add(new DeferredRef(
                ctx.target().classQName(),
                ctx.target().tid(),
                targetRefPath,
                sourceRefClass,
                targetRefClass,
                sourceRefTid
        ));
    }
}
```

Expressions (Auszug)

`SourcePathReadNode`, `VarReadNode`

Java

```
package ch.example.ilimap.truffle.nodes.expr;

import ch.example.ilimap.runtime.EvalContext;
```

```java
import ch.example.ilimap.runtime.Path;
import ch.example.ilimap.truffle.nodes.ExprNode;
import com.oracle.truffle.api.frame.VirtualFrame;

public final class SourcePathReadNode extends ExprNode {
    private final Path path;

    public SourcePathReadNode(Path path) {
        this.path = path;
    }

    @Override
    public Object executeGeneric(VirtualFrame frame, EvalContext ctx) {
        if ("tid".equals(path.toString())) return ctx.source().tid();
        return ctx.source().read(path);
    }
}
```

Java

```java
package ch.example.ilimap.truffle.nodes.expr;

import ch.example.ilimap.runtime.EvalContext;
import ch.example.ilimap.truffle.nodes.ExprNode;
import com.oracle.truffle.api.frame.VirtualFrame;

public final class VarReadNode extends ExprNode {
    private final String name;

    public VarReadNode(String name) {
        this.name = name;
    }

    @Override
    public Object executeGeneric(VirtualFrame frame, EvalContext ctx) {
```

```
        return ctx.vars().get(name);
    }
}
```

FunctionCallNode

Java

```java
package ch.example.ilimap.truffle.nodes.expr;

import ch.example.ilimap.runtime.EvalContext;
import ch.example.ilimap.truffle.nodes.ExprNode;
import com.oracle.truffle.api.frame.VirtualFrame;

import java.util.ArrayList;
import java.util.List;

public final class FunctionCallNode extends ExprNode {
    private final String functionName;
    @Children private final ExprNode[] argNodes;

    public FunctionCallNode(String functionName, ExprNode[] argNodes) {
        this.functionName = functionName;
        this.argNodes = argNodes;
    }

    @Override
    public Object executeGeneric(VirtualFrame frame, EvalContext ctx) {
        List<Object> args = new ArrayList<>(argNodes.length);
        for (ExprNode arg : argNodes) {
            args.add(arg.executeGeneric(frame, ctx));
        }
        return ctx.migration().helpers().call(functionName, args, ctx);
```

```
    }
}
```

`MapEnumExprNode` (zentral für Migrationen)

Java

```java
package ch.example.ilimap.truffle.nodes.expr;

import ch.example.ilimap.runtime.EvalContext;
import ch.example.ilimap.truffle.nodes.ExprNode;
import com.oracle.truffle.api.frame.VirtualFrame;

import java.util.Map;

public final class MapEnumExprNode extends ExprNode {
    @Child private ExprNode sourceExpr;
    private final Map<String, String> mappings;
    private final String defaultValue;      // nullable
    private final String defaultWarnMessage; // nullable

    public MapEnumExprNode(ExprNode sourceExpr,
                           Map<String, String> mappings,
                           String defaultValue,
                           String defaultWarnMessage) {
        this.sourceExpr = sourceExpr;
        this.mappings = Map.copyOf(mappings);
        this.defaultValue = defaultValue;
        this.defaultWarnMessage = defaultWarnMessage;
    }

    @Override
    public Object executeGeneric(VirtualFrame frame, EvalContext ctx) {
        Object raw = sourceExpr.executeGeneric(frame, ctx);
```

```
        if (raw == null) return null;

        String key = String.valueOf(raw);
        String mapped = mappings.get(key);
        if (mapped != null) return mapped;

        if (defaultValue != null) {
            if (defaultWarnMessage != null && !defaultWarnMessage.isBlank()) {
                ctx.migration().report().warn(defaultWarnMessage + " (value=" + key + ")'
            }
            return defaultValue;
        }

        ctx.migration().report().error("Unmapped enum value without DEFAULT: " + key);
        return null;
    }
}
```

---

8) `MigrationNodeFactory` (AST -> Truffle Nodes)

Das ist die Übersetzungsschicht zwischen Parser-AST und Laufzeit-AST.

Java

```
package ch.example.ilimap.truffle;

import ch.example.ilimap.parser.ast.*;
import ch.example.ilimap.runtime.Path;
import ch.example.ilimap.truffle.nodes.ExprNode;
import ch.example.ilimap.truffle.nodes.StmtNode;
import ch.example.ilimap.truffle.nodes.expr.*;
import ch.example.ilimap.truffle.nodes.stmt.*;
```

```
import com.oracle.truffle.api.source.Source;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public final class MigrationNodeFactory {

    private MigrationNodeFactory() {}

    public static MigrationRootNode build(IliMapLanguage language, MigrationAst ast, Sour
        List<ClassRuleNode> classRules = new ArrayList<>();
        for (ClassRuleAst cr : ast.classRules()) {
            ExprNode when = cr.whenExpr() == null ? null : buildExpr(cr.whenExpr());
            StmtNode[] stmts = cr.statements().stream().map(MigrationNodeFactory::buildSt

            classRules.add(new ClassRuleNode(
                    cr.sourceClass(),
                    cr.targetClass(),
                    Path.of(cr.keyMapping().sourcePath()),
                    Path.of(cr.keyMapping().targetPath()),
                    when,
                    stmts
            ));
        }
        return new MigrationRootNode(language, new ProgramNode(classRules));
    }

    private static StmtNode buildStmt(StmtAst stmt) {
        return switch (stmt) {
            case SetStmtAst s -> new SetStmtNode(Path.of(s.targetPath()), buildExpr(s.exp
            case LetStmtAst s -> new LetStmtNode(s.varName(), buildExpr(s.expr()));
            case AssertStmtAst s -> new AssertStmtNode(
                    buildExpr(s.condition()),
```

```
                s.severity() == AssertStmtAst.SeverityAst.ERROR ? AssertStmtNode.Seve
                s.message()
        );
        case RefStmtAst s -> new RefStmtNode(
                Path.of(s.targetRefPath()),
                s.sourceRefClass(),
                inferTargetRefClass(s), // TODO: aus Semantikcheck/Schema ableiten
                new SourcePathReadNode(Path.of(s.sourceRefExprPath().replaceFirst("^s
        );
    };
}

private static String inferTargetRefClass(RefStmtAst s) {
    // MVP-Placeholder. Besser: vom Semantikcheck vorab annotieren lassen.
    return switch (s.sourceRefClass()) {
        case "A.Eigentuemer" -> "B.Owner";
        default -> throw new IllegalArgumentException("No target ref class mapping kr
    };
}

private static ExprNode buildExpr(ExprAst expr) {
    return switch (expr) {
        case LiteralAst l      -> new LiteralNode(l.value());
        case SourcePathAst p   -> new SourcePathReadNode(Path.of(p.path()));
        case TargetPathAst p   -> new TargetPathReadNode(Path.of(p.path()));
        case VarAst v          -> new VarReadNode(v.name());
        case FunctionCallAst f -> new FunctionCallNode(
                f.name(),
                f.args().stream().map(MigrationNodeFactory::buildExpr).toArray(ExprNo
        );
        case MapEnumExprAst m  -> {
            Map<String, String> mm = new HashMap<>();
            for (MapEnumExprAst.Entry e : m.entries()) mm.put(e.sourceValue(), e.targ
            yield new MapEnumExprNode(buildExpr(m.sourceExpr()), mm, m.defaultValue()
        }
```

```
            case BinaryExprAst b   -> new BinaryOpNode(buildExpr(b.left()), b.op(), build
        };
    }
}
```

---

9) Truffle Instrument (MVP-schlank)

Du brauchst nicht sofort Vollausbau — aber ein kleiner Instrument-Hook für Trace/Stats lohnt sich früh.

‹/› Java

```java
package ch.example.ilimap.truffle.instrument;

import com.oracle.truffle.api.instrumentation.TruffleInstrument;

@TruffleInstrument.Registration(
        id = "ilimap-trace",
        name = "ILIMap Trace",
        version = "0.1.0"
)
public final class MigrationInstrument extends TruffleInstrument {

    @Override
    protected void onCreate(Env env) {
        // MVP: hier später Instrumenter verwenden (SourceSectionFilter + ExecutionEventN
        env.registerService(this);
    }

    @Override
    protected void onDispose(Env env) {
    }
}
```

Instrumente werden wie Sprachen per `@Registration` registriert. graalvm.org

---

10) CLI-Skelett (Dry Run / Run)

Java

```java
package ch.example.ilimap.cli;

import ch.example.ilimap.runtime.*;
import ch.example.ilimap.truffle.IliMapLanguage;
import ch.example.ilimap.truffle.MigrationContextHolder;
import org.graalvm.polyglot.Context;
import org.graalvm.polyglot.Source;

import java.nio.file.Files;
import java.nio.file.Path as NioPath;

public final class Main {
    public static void main(String[] args) throws Exception {
        if (args.length < 1) {
            System.err.println("Usage: ilimap <mapping-file>");
            System.exit(2);
        }

        String mappingCode = Files.readString(NioPath.of(args[0]));

        // TODO: echte INTERLIS-Adapter (ili2c/IMD/XTF/DB)
        MigrationContext mctx = DemoContextFactory.createDemoContext();
        MigrationContextHolder.set(mctx);

        try (Context polyglot = Context.newBuilder()
                .allowAllAccess(true)
```

```
                .build()) {

            var src = Source.newBuilder(IliMapLanguage.ID, mappingCode, "migration.ilimap
                    .mimeType(IliMapLanguage.MIME_TYPE)
                    .build();

            polyglot.eval(src);

            System.out.println("Migration done.");
            System.out.println("Errors: " + mctx.report().errorCount());
            System.out.println("Warnings: " + mctx.report().warnCount());
            System.out.println("Target counts: " + mctx.records().targetCounts());
        }
    }
}
```

Für den MVP nutze ich hier `MigrationContextHolder` als einfache Brücke. Später sauberer über Polyglot bindings / Launcher-Kontext.

---

11) JUnit-Tests (wichtig, damit das Projekt stabil bleibt)

Test 1: `MapEnumExprNode` mit Default + Warnung

Java

```java
package ch.example.ilimap.truffle.nodes.expr;

import ch.example.ilimap.model.SchemaRegistry;
import ch.example.ilimap.runtime.*;
import ch.example.ilimap.truffle.nodes.ExprNode;
import com.oracle.truffle.api.frame.VirtualFrame;
```

```
import org.junit.jupiter.api.Test;

import java.util.Map;

import static org.junit.jupiter.api.Assertions.*;

class MapEnumExprNodeTest {

    @Test
    void usesDefaultAndWarnsForUnknownEnum() {
        SourceRecord src = new MapRecord("A.Gebaeude", "G1").put("nutzung", "Spezial");
        TargetRecord tgt = new MapRecord("B.Building", "TEMP");
        MigrationReport report = new MigrationReport();

        MigrationContext mctx = new MigrationContext(
                new SchemaRegistry(),
                new SchemaRegistry(),
                new RecordRepository(),
                new IdMapRegistry(),
                new DeferredRefStore(),
                new HelperFunctionRegistry(),
                report
        );
        EvalContext ectx = new EvalContext(mctx, src, tgt);

        ExprNode sourceExpr = new SourcePathReadNode(ch.example.ilimap.runtime.Path.of("r
        MapEnumExprNode node = new MapEnumExprNode(
                sourceExpr,
                Map.of("Wohnen", "Residential"),
                "Unknown",
                "Unbekannte Nutzung"
        );

        Object result = node.executeGeneric((VirtualFrame) null, ectx);
```

```
        assertEquals("Unknown", result);
        assertEquals(1, report.warnCount());
        assertEquals(0, report.errorCount());
    }
}
```

Test 2: DeferredRefResolver löst Referenz korrekt auf

Java

```java
package ch.example.ilimap.runtime;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class DeferredRefResolverTest {

    @Test
    void resolvesDeferredReferenceViaIdMap() {
        IdMapRegistry idMaps = new IdMapRegistry();
        idMaps.put("A.Eigentuemer", "B.Owner", "E1", "E1");

        RecordRepository repo = new RecordRepository();
        MapRecord building = new MapRecord("B.Building", "G1");
        repo.saveTarget(building);

        MigrationReport report = new MigrationReport();
        DeferredRefResolver resolver = new DeferredRefResolver(
                idMaps, repo, report, DeferredRefResolver.MissingReferencePolicy.WARN
        );
```

```
        DeferredRef ref = new DeferredRef(
                "B.Building",
                "G1",
                Path.of("owner_ref"),
                "A.Eigentuemer",
                "B.Owner",
                "E1"
        );

        resolver.resolveAll(java.util.List.of(ref));

        assertEquals("E1", building.read(Path.of("owner_ref")));
        assertEquals(0, report.errorCount());
    }
}
```

Test 3: `ClassRuleNode` erzeugt Target + ID-Map

Java

```
package ch.example.ilimap.truffle;

import ch.example.ilimap.model.SchemaRegistry;
import ch.example.ilimap.runtime.*;
import ch.example.ilimap.truffle.nodes.StmtNode;
import ch.example.ilimap.truffle.nodes.expr.LiteralNode;
import ch.example.ilimap.truffle.nodes.stmt.SetStmtNode;
import com.oracle.truffle.api.frame.VirtualFrame;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;
```

```
class ClassRuleNodeTest {

    @Test
    void createsTargetAndRegistersIdMap() {
        RecordRepository repo = new RecordRepository();
        repo.addSource(new MapRecord("A.Gebaeude", "G1").put("bezeichnung", "Haus Nord"))

        MigrationContext mctx = new MigrationContext(
                new SchemaRegistry(),
                new SchemaRegistry(),
                repo,
                new IdMapRegistry(),
                new DeferredRefStore(),
                new HelperFunctionRegistry(),
                new MigrationReport()
        );

        StmtNode[] stmts = new StmtNode[] {
                new SetStmtNode(Path.of("name"), new LiteralNode("Haus Nord"))
        };

        ClassRuleNode rule = new ClassRuleNode(
                "A.Gebaeude",
                "B.Building",
                Path.of("tid"),
                Path.of("tid"),
                null,
                stmts
        );

        rule.executePass1((VirtualFrame) null, mctx);

        var targetOpt = repo.findTarget("B.Building", "G1");
        assertTrue(targetOpt.isPresent());
        assertEquals("Haus Nord", targetOpt.get().read(Path.of("name")));
```

```
        assertEquals("G1", mctx.idMaps().resolve("A.Gebaeude", "B.Building", "G1").orElse
    }
}
```

---

12) Was dir für „kompilierbar“ noch fehlt (aber gezielt klein halten)

Damit das Projekt wirklich läuft, brauchst du als nächstes noch diese Teile:

1. **Parser-Fassade**
   - `MappingParserFacade.parse(String)` (ANTLR -> AST)
2. **fehlende Expr-Nodes**
   - `LiteralNode`
   - `BinaryOpNode`
   - `TargetPathReadNode`
3. **Context-Brücke für Truffle**
   - `MigrationContextHolder` (ThreadLocal für MVP)
4. **DemoContextFactory**
   - `HelperFunctionRegistry` befüllen (`normalize_text` , `split_name` , `to_m2` , ...)

Beispiel: `MigrationContextHolder` (MVP-only, pragmatisch)

Java

```java
package ch.example.ilimap.truffle;

import ch.example.ilimap.runtime.MigrationContext;
```

```java
public final class MigrationContextHolder {
    private static final ThreadLocal<MigrationContext> TL = new ThreadLocal<>();

    private MigrationContextHolder() {}

    public static void set(MigrationContext ctx) { TL.set(ctx); }
    public static MigrationContext peek() { return TL.get(); }
    public static MigrationContext require() {
        MigrationContext ctx = TL.get();
        if (ctx == null) throw new IllegalStateException("MigrationContext not set");
        return ctx;
    }
    public static void clear() { TL.remove(); }
}
```

---

13) Priorisierte To-do-Liste für deinen ersten lauffähigen End-to-End MVP

Wenn du das wirklich bauen willst, würde ich **genau in dieser Reihenfolge** arbeiten:

Sprint 1

- `ilimap-runtime` fertig (Records, IDMap, DeferredRefResolver, Report)
- Tests für Runtime grün
- `HelperFunctionRegistry` + 2-3 Funktionen

Sprint 2

- `ilimap-parser` (nur Minimalsyntax)
- AST + Parser-Tests

Sprint 3

- `ilimap-truffle`:
  - `SET`, `LET`, `ASSERT`, `MAP_ENUM`
  - 1:1 Klassenmigration
- End-to-End Test ohne Referenzen

Sprint 4

- `REF ... RESOLVE ...` + Pass 2
- End-to-End Test mit Owner/Building Beispiel

Sprint 5

- `ilimap-semantics` (harte Checks)
- CLI + `dry-run`

---

14) Praktischer Tipp für INTERLIS-Anbindung

Für den Anfang nicht direkt XTF/DB schreiben. Nimm zuerst:

- **Schema A/B** als manuell erzeugtes `SchemaRegistry` (oder JSON)
- **Source-Daten** als `MapRecord`-Fixtures

Dann kannst du die DSL/Engine stabilisieren.

Erst danach:

- `ilimap-adapter-interlis`:
  - ili2c/IMD-Import für Schema
  - XTF/DB-Adapter für Records

Das spart dir extrem viel Frust am Anfang.

Wenn du möchtest, mache ich dir als Nächstes die **fehlenden 3 Kernteile komplett ausprogrammiert**:

- `LiteralNode`, `BinaryOpNode`, `TargetPathReadNode`
- `MappingParserFacade` (mit minimaler ANTLR-Visitor-Implementierung)
- einen **echten End-to-End JUnit-Test** mit deinem Beispiel `A.Gebaeude -> B.Building` inklusive `MAP_ENUM + REF` (Pass 1/Pass 2).

Ja, gerne.

Thought for 1m 40s

Ja — super. Ich mache dir jetzt die **3 fehlenden Kernteile** so, dass du damit wirklich weiterbauen kannst:

1. `LiteralNode`, `BinaryOpNode`, `TargetPathReadNode`
2. `MappingParserFacade` **+ minimaler ANTLR-Visitor**
3. **ein echter End-to-End JUnit-Test** mit `MAP_ENUM + REF` (Pass 1 / Pass 2)

Ich halte es **MVP-kompatibel** und bewusst pragmatisch.

---

0) Zwei kleine Korrekturen vorweg

A) Java-Import-Alias

Im letzten CLI-Beispiel war das hier **nicht gültiges Java**:

```java
import java.nio.file.Path as NioPath;
```

Java kennt keine Import-Aliase. Stattdessen einfach vollqualifiziert verwenden:

```java
String mappingCode = Files.readString(java.nio.file.Path.of(args[0]));
```

B) Parser/AST-Schicht bleibt getrennt

Der Parser erzeugt **Plain Java AST** (noch kein Truffle). Erst danach baut `MigrationNodeFactory` die Truffle-Nodes.

Das ist wichtig — und genau richtig so.

---

1) Fehlende Truffle-Expr-Nodes

`LiteralNode`

```java
package ch.example.ilimap.truffle.nodes.expr;

import ch.example.ilimap.runtime.EvalContext;
```

```
import ch.example.ilimap.truffle.nodes.ExprNode;
import com.oracle.truffle.api.frame.VirtualFrame;

public final class LiteralNode extends ExprNode {
    private final Object value;

    public LiteralNode(Object value) {
        this.value = value;
    }

    @Override
    public Object executeGeneric(VirtualFrame frame, EvalContext ctx) {
        return value;
    }
}
```

TargetPathReadNode

Wird z. B. für `ASSERT target.area_m2 >= 0` gebraucht.

Java

```
package ch.example.ilimap.truffle.nodes.expr;

import ch.example.ilimap.runtime.EvalContext;
import ch.example.ilimap.runtime.Path;
import ch.example.ilimap.truffle.nodes.ExprNode;
import com.oracle.truffle.api.frame.VirtualFrame;

public final class TargetPathReadNode extends ExprNode {
    private final Path path;

    public TargetPathReadNode(Path path) {
```

```
        this.path = path;
    }

    @Override
    public Object executeGeneric(VirtualFrame frame, EvalContext ctx) {
        if ("tid".equals(path.toString())) return ctx.target().tid();
        return ctx.target().read(path);
    }
}
```

`BinaryOpNode`

Unterstützt fürs MVP:

- `==`, `!=`, `<`, `>`, `<=`, `>=`
- `AND`, `OR`
- `+`, `-`, `*`, `/`

Mit robuster Typbehandlung (String/Number/Boolean).

Java

```
package ch.example.ilimap.truffle.nodes.expr;

import ch.example.ilimap.runtime.EvalContext;
import ch.example.ilimap.truffle.nodes.ExprNode;
import com.oracle.truffle.api.frame.VirtualFrame;

import java.math.BigDecimal;
import java.math.MathContext;
import java.util.Objects;
```

```
public final class BinaryOpNode extends ExprNode {
    @Child private ExprNode left;
    @Child private ExprNode right;
    private final String op;

    public BinaryOpNode(ExprNode left, String op, ExprNode right) {
        this.left = left;
        this.op = op;
        this.right = right;
    }

    @Override
    public Object executeGeneric(VirtualFrame frame, EvalContext ctx) {
        return switch (op) {
            case "AND" -> toBoolean(left.executeGeneric(frame, ctx)) && toBoolean(right.e
            case "OR"  -> toBoolean(left.executeGeneric(frame, ctx)) || toBoolean(right.e

            case "==" -> Objects.equals(left.executeGeneric(frame, ctx), right.executeGer
            case "!=" -> !Objects.equals(left.executeGeneric(frame, ctx), right.executeGe

            case "<", ">", "<=", ">=" -> compare(
                    left.executeGeneric(frame, ctx),
                    right.executeGeneric(frame, ctx),
                    op
            );

            case "+" -> plus(left.executeGeneric(frame, ctx), right.executeGeneric(frame,
            case "-" -> toBigDecimal(left.executeGeneric(frame, ctx)).subtract(toBigDecim
            case "*" -> toBigDecimal(left.executeGeneric(frame, ctx)).multiply(toBigDecim
            case "/" -> toBigDecimal(left.executeGeneric(frame, ctx)).divide(toBigDecimal

            default -> throw new IllegalStateException("Unsupported binary op: " + op);
        };
    }
```

```
    private static Object plus(Object l, Object r) {
        if (l == null && r == null) return null;
        if (l instanceof String || r instanceof String) {
            return String.valueOf(l) + String.valueOf(r);
        }
        return toBigDecimal(l).add(toBigDecimal(r));
    }

    private static boolean compare(Object l, Object r, String op) {
        if (l == null || r == null) {
            throw new IllegalStateException("Cannot compare null values with operator " +
        }

        // numerisch
        if (isNumericLike(l) && isNumericLike(r)) {
            int c = toBigDecimal(l).compareTo(toBigDecimal(r));
            return switch (op) {
                case "<"  -> c < 0;
                case ">"  -> c > 0;
                case "<=" -> c <= 0;
                case ">=" -> c >= 0;
                default   -> throw new IllegalStateException("Unexpected comparator " + c
            };
        }

        // String-Vergleich (lexikographisch)
        if (l instanceof String || r instanceof String) {
            int c = String.valueOf(l).compareTo(String.valueOf(r));
            return switch (op) {
                case "<"  -> c < 0;
                case ">"  -> c > 0;
                case "<=" -> c <= 0;
                case ">=" -> c >= 0;
                default   -> throw new IllegalStateException("Unexpected comparator " + c
            };
```

```
        }

        throw new IllegalStateException("Unsupported comparison types: " + l.getClass() +
    }

    private static boolean isNumericLike(Object o) {
        return o instanceof Number || (o instanceof String s && s.matches("-?\\d+(\\.\\d+
    }

    private static BigDecimal toBigDecimal(Object o) {
        if (o == null) throw new IllegalStateException("Expected number, got null");
        if (o instanceof BigDecimal bd) return bd;
        if (o instanceof Integer i) return BigDecimal.valueOf(i);
        if (o instanceof Long l) return BigDecimal.valueOf(l);
        if (o instanceof Double d) return BigDecimal.valueOf(d);
        if (o instanceof Float f) return BigDecimal.valueOf(f.doubleValue());
        if (o instanceof Number n) return new BigDecimal(n.toString());
        if (o instanceof String s) return new BigDecimal(s);
        throw new IllegalStateException("Expected number, got: " + o + " (" + o.getClass(
    }

    private static boolean toBoolean(Object o) {
        if (o instanceof Boolean b) return b;
        if (o instanceof String s) return Boolean.parseBoolean(s);
        throw new IllegalStateException("Expected boolean, got: " + o);
    }
}
```

---

2) Fehlende Parser-AST-Typen (Expressions)

Damit `MigrationNodeFactory.buildExpr(...)` kompilieren kann.

Expr-ASTs (ergänzend zu deinen vorhandenen AST-Records)

Java

```java
package ch.example.ilimap.parser.ast;

public record LiteralAst(Object value) implements ExprAst {}
```

Java

```java
package ch.example.ilimap.parser.ast;

public record SourcePathAst(String path) implements ExprAst {}
```

Java

```java
package ch.example.ilimap.parser.ast;

public record TargetPathAst(String path) implements ExprAst {}
```

Java

```java
package ch.example.ilimap.parser.ast;

public record VarAst(String name) implements ExprAst {}
```

Java

```java
package ch.example.ilimap.parser.ast;

public record BinaryExprAst(ExprAst left, String op, ExprAst right) implements ExprAst {}
```

Java

```java
package ch.example.ilimap.parser.ast;

import java.util.List;

public record FunctionCallAst(String name, List<ExprAst> args) implements ExprAst {}
```

Java

```java
package ch.example.ilimap.parser.ast;

import java.util.List;

public record MapEnumExprAst(
        ExprAst sourceExpr,
        List<Entry> entries,
        String defaultValue,
        String defaultWarnMessage
) implements ExprAst {
    public record Entry(String sourceValue, String targetValue) {}
}
```

---

3) `MappingParserFacade` + minimaler ANTLR-Visitor

Das ist eine **minimale, aber brauchbare** Implementierung für den MVP und den End-to-End-Test.

> Annahme: Die ANTLR-generierten Klassen liegen im Package `ch.example.ilimap.parser.antlr` (ggf. anpassen).

MappingParserFacade

Java

```java
package ch.example.ilimap.parser;

import ch.example.ilimap.parser.ast.MigrationAst;
import ch.example.ilimap.parser.antlr.InterlisMapLexer;
import ch.example.ilimap.parser.antlr.InterlisMapParser;
import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.misc.ParseCancellationException;

public final class MappingParserFacade {
    private MappingParserFacade() {}

    public static MigrationAst parse(String code) {
        CharStream input = CharStreams.fromString(code);
        InterlisMapLexer lexer = new InterlisMapLexer(input);

        CommonTokenStream tokens = new CommonTokenStream(lexer);
        InterlisMapParser parser = new InterlisMapParser(tokens);

        lexer.removeErrorListeners();
        parser.removeErrorListeners();

        BaseErrorListener err = new BaseErrorListener() {
            @Override
            public void syntaxError(Recognizer<?, ?> recognizer,
                                    Object offendingSymbol,
                                    int line,
                                    int charPositionInLine,
                                    String msg,
                                    RecognitionException e) {
                throw new ParseCancellationException("Syntax error at " + line + ":" + ch
            }
```

```
        };
        lexer.addErrorListener(err);
        parser.addErrorListener(err);

        parser.setErrorHandler(new BailErrorStrategy());

        var tree = parser.migration();
        return new AstBuilderVisitor().visitMigration(tree);
    }
}
```

`AstBuilderVisitor` (minimaler Visitor für deinen MVP)

Java

```java
package ch.example.ilimap.parser;

import ch.example.ilimap.parser.ast.*;
import ch.example.ilimap.parser.antlr.InterlisMapBaseVisitor;
import ch.example.ilimap.parser.antlr.InterlisMapParser;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public final class AstBuilderVisitor extends InterlisMapBaseVisitor<Object> {

    @Override
    public MigrationAst visitMigration(InterlisMapParser.MigrationContext ctx) {
        String sourceModel = unquote(ctx.STRING(0).getText());
        String targetModel = unquote(ctx.STRING(1).getText());
```

```
        List<ClassRuleAst> classRules = new ArrayList<>();
        for (var body : ctx.migrationBody()) {
            if (body.classRule() != null) {
                classRules.add((ClassRuleAst) visit(body.classRule()));
            }
            // optionsBlock wird im MVP ignoriert
        }

        return new MigrationAst(sourceModel, targetModel, classRules);
    }

    @Override
    public ClassRuleAst visitClassRule(InterlisMapParser.ClassRuleContext ctx) {
        String sourceClass = ctx.qname(0).getText();
        String targetClass = ctx.qname(1).getText();

        KeyMappingAst key = (KeyMappingAst) visit(ctx.keyClause());
        ExprAst whenExpr = ctx.whenClause() != null ? (ExprAst) visit(ctx.whenClause()) :

        List<StmtAst> statements = new ArrayList<>();
        for (var s : ctx.classStmt()) {
            // classStmt hat genau 1 konkrete Alternative
            StmtAst stmt = null;
            if (s.setStmt() != null) stmt = (StmtAst) visit(s.setStmt());
            else if (s.letStmt() != null) stmt = (StmtAst) visit(s.letStmt());
            else if (s.assertStmt() != null) stmt = (StmtAst) visit(s.assertStmt());
            else if (s.refStmt() != null) stmt = (StmtAst) visit(s.refStmt());

            if (stmt != null) statements.add(stmt);
        }

        return new ClassRuleAst(sourceClass, targetClass, key, whenExpr, statements);
    }

    @Override
```

```java
    public KeyMappingAst visitKeyClause(InterlisMapParser.KeyClauseContext ctx) {
        // Grammatik: KEY source.<IDENT> AS target.<path>
        String sourcePath = ctx.IDENT().getText();
        String targetPath = ctx.path().getText();
        return new KeyMappingAst(sourcePath, targetPath);
    }

    @Override
    public ExprAst visitWhenClause(InterlisMapParser.WhenClauseContext ctx) {
        return (ExprAst) visit(ctx.expr());
    }

    @Override
    public StmtAst visitSetStmt(InterlisMapParser.SetStmtContext ctx) {
        String targetPath = ctx.path().getText();
        ExprAst expr = (ExprAst) visit(ctx.expr());
        return new SetStmtAst(targetPath, expr);
    }

    @Override
    public StmtAst visitLetStmt(InterlisMapParser.LetStmtContext ctx) {
        String varName = ctx.IDENT().getText();
        ExprAst expr = (ExprAst) visit(ctx.expr());
        return new LetStmtAst(varName, expr);
    }

    @Override
    public StmtAst visitAssertStmt(InterlisMapParser.AssertStmtContext ctx) {
        ExprAst cond = (ExprAst) visit(ctx.expr());
        var sev = "ERROR".equals(ctx.severity().getText())
                ? AssertStmtAst.SeverityAst.ERROR
                : AssertStmtAst.SeverityAst.WARN;
        String message = unquote(ctx.STRING().getText());
        return new AssertStmtAst(cond, sev, message);
    }
```

```
@Override
public StmtAst visitRefStmt(InterlisMapParser.RefStmtContext ctx) {
    String targetRefPath = ctx.path().getText(); // target.<path>
    String sourceRefClass = ctx.qname().getText();
    String sourceRefExprPath = "source." + ctx.sourceRef().path().getText();
    return new RefStmtAst(targetRefPath, sourceRefClass, sourceRefExprPath);
}

// --------------------
// Exprs
// --------------------

@Override
public ExprAst visitSourcePathExpr(InterlisMapParser.SourcePathExprContext ctx) {
    return new SourcePathAst(ctx.path().getText());
}

@Override
public ExprAst visitTargetPathExpr(InterlisMapParser.TargetPathExprContext ctx) {
    return new TargetPathAst(ctx.path().getText());
}

@Override
public ExprAst visitVarExpr(InterlisMapParser.VarExprContext ctx) {
    return new VarAst(ctx.IDENT().getText());
}

@Override
public ExprAst visitStringExpr(InterlisMapParser.StringExprContext ctx) {
    return new LiteralAst(unquote(ctx.STRING().getText()));
}

@Override
public ExprAst visitNumberExpr(InterlisMapParser.NumberExprContext ctx) {
```

```
        String n = ctx.NUMBER().getText();
        // Für den MVP als BigDecimal lesen wäre auch möglich; hier String->Double/Long s
        if (n.contains(".")) return new LiteralAst(Double.parseDouble(n));
        return new LiteralAst(Long.parseLong(n));
    }

    @Override
    public ExprAst visitTrueExpr(InterlisMapParser.TrueExprContext ctx) {
        return new LiteralAst(Boolean.TRUE);
    }

    @Override
    public ExprAst visitFalseExpr(InterlisMapParser.FalseExprContext ctx) {
        return new LiteralAst(Boolean.FALSE);
    }

    @Override
    public ExprAst visitParenExpr(InterlisMapParser.ParenExprContext ctx) {
        return (ExprAst) visit(ctx.expr());
    }

    @Override
    public ExprAst visitNotExpr(InterlisMapParser.NotExprContext ctx) {
        // Vereinfachung: NOT x => BinaryExprAst(x == false) vermeiden wir.
        // Sauberer wäre ein UnaryNotAst + UnaryNotNode. Für MVP:
        ExprAst inner = (ExprAst) visit(ctx.expr());
        return new FunctionCallAst("__not", List.of(inner));
    }

    @Override
    public ExprAst visitMulDivExpr(InterlisMapParser.MulDivExprContext ctx) {
        return new BinaryExprAst(
                (ExprAst) visit(ctx.expr(0)),
                ctx.getChild(1).getText(),
                (ExprAst) visit(ctx.expr(1))
```

```
        );
    }

    @Override
    public ExprAst visitAddSubExpr(InterlisMapParser.AddSubExprContext ctx) {
        return new BinaryExprAst(
                (ExprAst) visit(ctx.expr(0)),
                ctx.getChild(1).getText(),
                (ExprAst) visit(ctx.expr(1))
        );
    }

    @Override
    public ExprAst visitCmpExpr(InterlisMapParser.CmpExprContext ctx) {
        return new BinaryExprAst(
                (ExprAst) visit(ctx.expr(0)),
                ctx.getChild(1).getText(),
                (ExprAst) visit(ctx.expr(1))
        );
    }

    @Override
    public ExprAst visitBoolExpr(InterlisMapParser.BoolExprContext ctx) {
        return new BinaryExprAst(
                (ExprAst) visit(ctx.expr(0)),
                ctx.getChild(1).getText(), // AND/OR
                (ExprAst) visit(ctx.expr(1))
        );
    }

    @Override
    public ExprAst visitFuncCallExpr(InterlisMapParser.FuncCallExprContext ctx) {
        return (ExprAst) visit(ctx.funcCall());
    }
```

```java
    @Override
    public ExprAst visitFuncCall(InterlisMapParser.FuncCallContext ctx) {
        String name = ctx.IDENT().getText();
        List<ExprAst> args = new ArrayList<>();
        for (var e : ctx.expr()) {
            args.add((ExprAst) visit(e));
        }
        return new FunctionCallAst(name, args);
    }

    @Override
    public ExprAst visitMapEnumExpr(InterlisMapParser.MapEnumExprContext ctx) {
        ExprAst sourceExpr = (ExprAst) visit(ctx.expr());

        List<MapEnumExprAst.Entry> entries = new ArrayList<>();
        for (var e : ctx.enumMapEntry()) {
            String src = unquote(e.STRING(0).getText());
            String tgt = unquote(e.STRING(1).getText());
            entries.add(new MapEnumExprAst.Entry(src, tgt));
        }

        String defaultValue = null;
        String defaultWarnMessage = null;
        if (ctx.enumDefault() != null) {
            defaultValue = unquote(ctx.enumDefault().STRING(0).getText());
            if (ctx.enumDefault().STRING().size() > 1) {
                defaultWarnMessage = unquote(ctx.enumDefault().STRING(1).getText());
            }
        }

        return new MapEnumExprAst(sourceExpr, entries, defaultValue, defaultWarnMessage);
    }

    private static String unquote(String s) {
        Objects.requireNonNull(s);
```

```
        if (s.length() >= 2 && s.startsWith("\"") && s.endsWith("\"")) {
            String body = s.substring(1, s.length() - 1);
            // minimale Escape-Behandlung
            return body
                    .replace("\\\"", "\"")
                    .replace("\\\\", "\\")
                    .replace("\\n", "\n")
                    .replace("\\t", "\t")
                    .replace("\\r", "\r");
        }
        return s;
    }
}
```

---

4) Kleines Update für `MigrationNodeFactory` wegen `NOT`

Wenn du im Parser `NOT` als `FunctionCallAst("__not", ...)` modellierst, dann braucht die Factory nur einen kleinen Spezialfall.

Patch in `buildExpr(...)`

Java

```
case FunctionCallAst f -> {
    if ("__not".equals(f.name())) {
        if (f.args().size() != 1) {
            throw new IllegalArgumentException("__not expects exactly 1 arg");
        }
        ExprNode inner = buildExpr(f.args().get(0));
        yield new FunctionCallNode("__not", new ExprNode[]{ inner });
    }
    yield new FunctionCallNode(
```

```
                f.name(),
                f.args().stream().map(MigrationNodeFactory::buildExpr).toArray(ExprNode[]::ne
        );
    }
```

Und dann im `HelperFunctionRegistry` einfach `__not` registrieren (siehe Test unten).

> Alternativ schöner: `UnaryNotAst` + `UnaryNotNode`. Für MVP ist der Helper-Trick okay.

---

5) End-to-End JUnit-Test (DSL → Parser → Truffle → Pass2 Referenzen)

Das ist ein **echter Integrations-Test** für deinen Kern:

- Parsing
- NodeFactory
- Pass 1 (`SET` , `MAP_ENUM` , `LET` , `ASSERT` , `REF`)
- Pass 2 (DeferredRefResolver)

Ich mache ihn bewusst **ohne Polyglot-** `Context` , damit du keine Service-Registration-Probleme im Test hast.
Trotzdem ist es End-to-End über deine Engine.

`MigrationEndToEndTest`

Java

```java
package ch.example.ilimap.truffle;

import ch.example.ilimap.model.*;
```

```
import ch.example.ilimap.parser.MappingParserFacade;
import ch.example.ilimap.runtime.*;
import com.oracle.truffle.api.source.Source;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;

class MigrationEndToEndTest {

    @AfterEach
    void cleanup() {
        MigrationContextHolder.clear();
    }

    @Test
    void migratesBuildingOwner_withEnumMappingAndDeferredRefResolution() throws Exceptior
        // ---------- Arrange ----------
        MigrationContext mctx = createDemoContext();
        MigrationContextHolder.set(mctx);

        String dsl = """
            MIGRATION "DM_A_1_0" -> "DM_B_2_0" {

              CLASS A.Gebaeude -> B.Building
              KEY source.tid AS target.tid
              WHEN source.status != "GELOESCHT" {

                SET target.name = normalize_text(source.bezeichnung);
```

```
      SET target.address.street = parse_street(source.adresse_text);
      SET target.address.number = parse_house_no(source.adresse_text);

      SET target.usage = MAP_ENUM(source.nutzung) {
        "Wohnen"  -> "Residential";
        "Gewerbe" -> "Commercial";
        "Misch"   -> "Mixed";
        DEFAULT   -> "Unknown" WARN "Unbekannte Nutzung";
      };

      SET target.area_m2 = to_m2(source.flaeche, source.flaeche_einheit);

      ASSERT target.area_m2 >= 0 ERROR "Fläche darf nicht negativ sein";

      REF target.owner_ref = RESOLVE A.Eigentuemer BY source.eigentuemer_ref;
    }

    CLASS A.Eigentuemer -> B.Owner
    KEY source.tid AS target.tid {

      LET nameParts = split_name(source.vollname);

      SET target.first_name = first(nameParts);
      SET target.last_name  = last(nameParts);

      SET target.kind = MAP_ENUM(source.typ) {
        "Privatperson" -> "Person";
        "Firma"        -> "Organisation";
        DEFAULT        -> "Unknown";
      };
    }
  }
  """;

// Parser -> AST
```

```
var ast = MappingParserFacade.parse(dsl);

// AST -> Truffle Nodes
var truffleSource = Source.newBuilder(IliMapLanguage.ID, dsl, "test.ilimap")
        .mimeType(IliMapLanguage.MIME_TYPE)
        .build();

var root = MigrationNodeFactory.build(new IliMapLanguage(), ast, truffleSource);

// ---------- Act ----------
Object result = root.getCallTarget().call();

// ---------- Assert ----------
assertTrue(result instanceof MigrationReport);
MigrationReport report = (MigrationReport) result;

assertEquals(0, report.errorCount(), () -> "Errors: " + report.issues());
// Warnungen sind möglich, hier sollten keine auftreten
assertEquals(0, report.warnCount(), () -> "Warnings: " + report.issues());

var ownerOpt = mctx.records().findTarget("B.Owner", "E1");
assertTrue(ownerOpt.isPresent(), "Owner E1 should exist");
TargetRecord owner = ownerOpt.get();

assertEquals("Max", owner.read(Path.of("first_name")));
assertEquals("Muster", owner.read(Path.of("last_name")));
assertEquals("Person", owner.read(Path.of("kind")));

var buildingOpt = mctx.records().findTarget("B.Building", "G1");
assertTrue(buildingOpt.isPresent(), "Building G1 should exist");
TargetRecord building = buildingOpt.get();

assertEquals("Haus Nord", building.read(Path.of("name")));
assertEquals("Bahnhofstrasse", building.read(Path.of("address.street")));
assertEquals("12", building.read(Path.of("address.number")));
```

```
        assertEquals("Mixed", building.read(Path.of("usage")));

        Object area = building.read(Path.of("area_m2"));
        assertNotNull(area);
        assertEquals(0, new BigDecimal(area.toString()).compareTo(new BigDecimal("1500"))

        // Pass 2 Referenzauflösung
        assertEquals("E1", building.read(Path.of("owner_ref")));

        // ID-Maps wurden aufgebaut
        assertEquals("G1", mctx.idMaps().resolve("A.Gebaeude", "B.Building", "G1").orElse
        assertEquals("E1", mctx.idMaps().resolve("A.Eigentuemer", "B.Owner", "E1").orElse
    }

    private static MigrationContext createDemoContext() {
        // --- Schemas (minimal; Semantikcheck prüft in MVP primär Klassenexistenz) ---
        SchemaRegistry schemaA = new SchemaRegistry();
        schemaA.register(new ClassSchema(
                "A.Gebaeude",
                Map.of(
                        "bezeichnung", new AttributeSchema("bezeichnung", ValueType.STRIN
                        "adresse_text", new AttributeSchema("adresse_text", ValueType.STR
                        "nutzung", new AttributeSchema("nutzung", ValueType.ENUM, Set.of(
                        "flaeche", new AttributeSchema("flaeche", ValueType.NUMBER, Set.o
                        "flaeche_einheit", new AttributeSchema("flaeche_einheit", ValueTy
                        "eigentuemer_ref", new AttributeSchema("eigentuemer_ref", ValueTy
                        "status", new AttributeSchema("status", ValueType.STRING, Set.of(
                ),
                Set.of()
        ));
        schemaA.register(new ClassSchema(
                "A.Eigentuemer",
                Map.of(
                        "vollname", new AttributeSchema("vollname", ValueType.STRING, Set
                        "typ", new AttributeSchema("typ", ValueType.ENUM, Set.of("Privatp
```

```
        ),
        Set.of()
));

SchemaRegistry schemaB = new SchemaRegistry();
schemaB.register(new ClassSchema("B.Building", Map.of(), Set.of()));
schemaB.register(new ClassSchema("B.Owner", Map.of(), Set.of()));

// --- Source-Daten ---
RecordRepository repo = new RecordRepository();
repo.addSource(new MapRecord("A.Eigentuemer", "E1")
        .put("vollname", "Max Muster")
        .put("typ", "Privatperson"));

repo.addSource(new MapRecord("A.Gebaeude", "G1")
        .put("bezeichnung", " Haus Nord ")
        .put("adresse_text", "Bahnhofstrasse 12")
        .put("nutzung", "Misch")
        .put("flaeche", new BigDecimal("0.15"))
        .put("flaeche_einheit", "ha")
        .put("eigentuemer_ref", "E1")
        .put("status", "AKTIV"));

// --- Helper-Funktionen ---
HelperFunctionRegistry helpers = new HelperFunctionRegistry()
        .register("__not", (args, ctx) -> {
            Object v = args.get(0);
            if (!(v instanceof Boolean b)) throw new IllegalArgumentException("__
            return !b;
        })
        .register("normalize_text", (args, ctx) -> {
            Object v = args.get(0);
            return v == null ? null : String.valueOf(v).trim().replaceAll("\\s+",
        })
        .register("parse_street", (args, ctx) -> {
```

```
            String s = String.valueOf(args.get(0));
            int idx = s.lastIndexOf(' ');
            return idx < 0 ? s : s.substring(0, idx);
        })
        .register("parse_house_no", (args, ctx) -> {
            String s = String.valueOf(args.get(0));
            int idx = s.lastIndexOf(' ');
            return idx < 0 ? "" : s.substring(idx + 1);
        })
        .register("to_m2", (args, ctx) -> {
            BigDecimal value = new BigDecimal(String.valueOf(args.get(0)));
            String unit = String.valueOf(args.get(1));
            return switch (unit) {
                case "m2" -> value;
                case "ha" -> value.multiply(new BigDecimal("10000"));
                default -> {
                    ctx.migration().report().warn("Unbekannte Einheit: " + unit);
                    yield null;
                }
            };
        })
        .register("split_name", (args, ctx) -> {
            String full = String.valueOf(args.get(0)).trim();
            if (full.isBlank()) return List.of("", "");
            String[] parts = full.split("\\s+");
            if (parts.length == 1) return List.of(parts[0], "");
            return List.of(parts[0], parts[parts.length - 1]);
        })
        .register("first", (args, ctx) -> {
            @SuppressWarnings("unchecked")
            List<Object> parts = (List<Object>) args.get(0);
            return parts.isEmpty() ? "" : parts.get(0);
        })
        .register("last", (args, ctx) -> {
            @SuppressWarnings("unchecked")
```

```
                    List<Object> parts = (List<Object>) args.get(0);
                    return parts.size() < 2 ? "" : parts.get(1);
                });

        return new MigrationContext(
                schemaA,
                schemaB,
                repo,
                new IdMapRegistry(),
                new DeferredRefStore(),
                helpers,
                new MigrationReport()
        );
    }
}
```

---

6) Kleiner, wichtiger Patch in `MigrationNodeFactory` für `REF` - SourcePath

In deinem bisherigen Code war das grundsätzlich okay, aber ich würde es robuster machen:

`RefStmtAst` → `RefStmtNode` Erzeugung (Patch)

Java

```
case RefStmtAst s -> {
    String sourceExprPath = s.sourceRefExprPath();
    if (!sourceExprPath.startsWith("source.")) {
        throw new IllegalArgumentException("Ref source path must start with source.: " +
    }
    String sourcePathOnly = sourceExprPath.substring("source.".length());
```

```
    yield new RefStmtNode(
            Path.of(s.targetRefPath()),
            s.sourceRefClass(),
            inferTargetRefClass(s), // später vom Semantikchecker annotieren lassen
            new SourcePathReadNode(Path.of(sourcePathOnly))
    );
}
```

---

7) Was du als Nächstes sofort machen solltest (sehr konkret)

Wenn du mit dem obigen Stand weitermachst, kommen als nächste "echte Produktivitäts-Booster" genau diese 4 Punkte:

1) `inferTargetRefClass(...)` raus aus der Factory

Statt Heuristik/Hardcode:

- im **Semantikcheck** die Referenz-Zielklasse bestimmen
- `RefStmtAst` mit Zielklasse annotieren (oder separates „typed AST")

2) `SourceSection` auf Nodes setzen

Dann kannst du im Report sagen:

- Fehler in `migration.ilimap`, Zeile 23, Spalte 17

3) Typcheck für `MAP_ENUM`

Prüfe statisch:

- Quellwerte existieren in A-Domain?
- Zielwerte existieren in B-Domain?

- `DEFAULT` vorhanden / erforderlich?

4) `RecordRepository` Adapter-Layer

Zuerst JSON/In-Memory, dann:

- XTF lesen
- XTF schreiben
- DB Adapter

---

8) Noch ein realistischer Hinweis zur Kompilierbarkeit

Die obigen Snippets sind absichtlich **nah an kompilierbar**, aber es gibt in deinem Gesamtstand noch ein paar offene Stellen aus früheren Teilen (z. B. vollständige Parser-AST-Klassen, `TargetPathAst`, `LiteralAst`, evtl. Package-Namen der ANTLR-Generates).

Das ist normal. Mit dem hier hast du aber jetzt den **Kernpfad vollständig**:

DSL-String → ANTLR → AST → Truffle-Nodes → Pass1 → DeferredRefs → Pass2 → Ergebnis

Und genau das ist der schwierigste Teil.

Wenn du willst, gehe ich als Nächstes einen Schritt weiter und mache dir ein **„typed AST"-Design** (nach dem Semantikcheck), damit `MigrationNodeFactory` keine Heuristiken wie `inferTargetRefClass(...)` mehr braucht und Referenzen/Enums sauber typisiert sind. Das wäre der nächste große Qualitätssprung.

Ja, gerne.

Thought for 12s ›

Ja — das ist **genau der richtige nächste Schritt**.

Ein **typed AST** macht aus deinem MVP einen ernsthaften Compiler/Runner, weil du damit:

- Heuristiken entfernst (`inferTargetRefClass(...)`)
- Fehler **vor** der Ausführung findest
- Node-Bau massiv vereinfachst
- Traces/Reports besser machst (Typ- und Pfadinfos sind überall verfügbar)

Ziel des typed AST

Du hast aktuell:

- **Parser-AST** = „was im DSL steht“
- **Truffle-Nodes** = „wie es ausgeführt wird“

Dazwischen fehlt die Schicht:

- **Typed AST / Bound AST** = „was es *bedeutet* im Kontext von Schema A/B“

Das typed AST ist also dein **semantisch aufgelöstes** AST.

Pipeline mit typed AST

Vorher

```
DSL → Parser-AST → Truffle-Nodes
```

Nachher

```
DSL → Parser-AST → Semantic Binding/Typing → Typed AST → Truffle-Nodes
```

Das ist der große Sprung.

Was im typed AST zusätzlich drinsteckt

Im Parser-AST steht z. B.:

- `source.eigentuemer_ref`
- `RESOLVE A.Eigentuemer`

Im typed AST steht dann zusätzlich:

- Quellpfad ist gültig ✅
- Typ des Ausdrucks = `REFERENCE(A.Eigentuemer)` ✅
- Zielpfad `target.owner_ref` ist `REFERENCE(B.Owner)` ✅
- Mapping von `A.Eigentuemer -> B.Owner` existiert ✅
- `targetRefClass = B.Owner` (direkt gebunden) ✅

Damit braucht `MigrationNodeFactory` keine Rate-Logik mehr.

Vorschlag für die Architektur

Ich würde das in **3 AST-Ebenen** trennen:

1) Parser-AST (`parser.ast`)

Nahe an der Syntax, ungetypt.

2) Typed AST (`semantics.tast` oder `typedast`)

Semantisch geprüft + gebunden + typisiert.

3) Runtime/Truffle AST (`truffle.nodes`)

Ausführungsknoten.

Kernideen im typed AST

A) Jeder Ausdruck hat einen statischen Typ

Zum Beispiel:

- `source.bezeichnung` → `STRING`
- `target.area_m2 >= 0` → `BOOLEAN`
- `to_m2(...)` → `NUMBER`
- `MAP_ENUM(...)` → `ENUM(B.Building.usage)` oder zumindest `STRING/ENUM`

B) Jeder Pfad ist gebunden (nicht mehr nur String)

Statt `"address.street"` speicherst du ein Objekt wie:

- Zielklasse: `B.Building`
- Pfad: `address.street`
- Typ: `STRING`
- nullable: `false/true`

C) Referenz-Statements sind vollständig aufgelöst

```
REF target.owner_ref = RESOLVE A.Eigentuemer BY source.eigentuemer_ref
```

wird zu etwas wie:

- target path schema = `B.Building.owner_ref : REF B.Owner`
- source ref expr type = `REF A.Eigentuemer`
- source class mapping rule = `A.Eigentuemer -> B.Owner`
- missing-ref policy = `WARN`

Konkretes Typed-AST-Modell (Java-Skelett)

Gemeinsame Typen

Java

```java
package ch.example.ilimap.semantics.tast;

public sealed interface TType permits TPrimitiveType, TEnumType, TReferenceType, TObjectT
    boolean nullable();
}
```

Java

```java
package ch.example.ilimap.semantics.tast;

public record TPrimitiveType(Kind kind, boolean nullable) implements TType {
```

```
    public enum Kind { STRING, NUMBER, BOOLEAN }
}
```

‹/› Java

```java
package ch.example.ilimap.semantics.tast;

import java.util.Set;

public record TEnumType(
        String enumName,          // z.B. B.Building.usage (oder Domainname)
        Set<String> allowedValues,
        boolean nullable
) implements TType {}
```

‹/› Java

```java
package ch.example.ilimap.semantics.tast;

public record TReferenceType(
        String targetClassQName,  // z.B. B.Owner oder A.Eigentuemer
        boolean nullable
) implements TType {}
```

‹/› Java

```java
package ch.example.ilimap.semantics.tast;

public record TObjectType(String name, boolean nullable) implements TType {}
```

‹/› Java

```
package ch.example.ilimap.semantics.tast;

public record TListType(TType elementType, boolean nullable) implements TType {}
```

Gebundene Pfade (wichtig)

‹/› Java

```
package ch.example.ilimap.semantics.tast;

public record BoundPath(
        ScopeKind scope,              // SOURCE / TARGET / LOCAL
        String ownerClassQName,       // A.Gebaeude oder B.Building (bei LOCAL evtl. null
        String path,                  // "address.street"
        TType type
) {
    public enum ScopeKind { SOURCE, TARGET, LOCAL }
}
```

Damit ist `"source.adresse_text"` nicht mehr nur ein String, sondern ein gebundener, typisierter Zugriff.

Typed Migration / Typed Class Rule

‹/› Java

```
package ch.example.ilimap.semantics.tast;

import java.util.List;
```

```
public record TMigration(
        String sourceModel,
        String targetModel,
        List<TClassRule> classRules
) {}
```

‹/› Java

```
package ch.example.ilimap.semantics.tast;

import java.util.List;

public record TClassRule(
        String sourceClass,
        String targetClass,
        TKeyMapping keyMapping,
        TExpr whenExpr,                 // nullable
        List<TStmt> statements,
        RuleBinding binding             // Zusatzinfos (z.B. class mapping id)
) {}
```

‹/› Java

```
package ch.example.ilimap.semantics.tast;

public record TKeyMapping(
        BoundPath sourceKeyPath,        // SOURCE, typisiert
        BoundPath targetKeyPath         // TARGET, typisiert
) {}
```

‹/› Java

```java
package ch.example.ilimap.semantics.tast;

public record RuleBinding(
        String mappingId                // z.B. "A.Eigentuemer->B.Owner"
) {}
```

Typed Statements

Statement-Basis

‹/› Java

```java
package ch.example.ilimap.semantics.tast;

public sealed interface TStmt permits TSetStmt, TLetStmt, TAssertStmt, TRefStmt {}
```

SET

‹/› Java

```java
package ch.example.ilimap.semantics.tast;

public record TSetStmt(
        BoundPath targetPath,
        TExpr expr,
        Coercion coercion              // optional: implizite Konversionen dokumentieren
) implements TStmt {}
```

`Coercion` ist super nützlich (z. B. `NUMBER -> STRING`, `ENUM -> STRING`, etc.).

Java

```java
package ch.example.ilimap.semantics.tast;

public sealed interface Coercion permits NoCoercion, PrimitiveCoercion {}

public record NoCoercion() implements Coercion {}

public record PrimitiveCoercion(
        TType fromType,
        TType toType,
        String strategy              // z.B. "numberToString"
) implements Coercion {}
```

LET

Java

```java
package ch.example.ilimap.semantics.tast;

public record TLetStmt(
        String varName,
        TExpr expr,
        TType varType
) implements TStmt {}
```

ASSERT

Java

```java
package ch.example.ilimap.semantics.tast;

public record TAssertStmt(
        TExpr condition,              // muss BOOLEAN sein (bereits geprüft)
        Severity severity,
        String message
) implements TStmt {
    public enum Severity { WARN, ERROR }
}
```

REF (hier gewinnt typed AST massiv)

Java

```java
package ch.example.ilimap.semantics.tast;

public record TRefStmt(
        BoundPath targetRefPath,                  // TARGET, Typ = REF(B.Owner)
        TExpr sourceRefExpr,                      // Typ = REF(A.Eigentuemer) oder STRING
        RefResolutionBinding resolutionBinding
) implements TStmt {}
```

Java

```java
package ch.example.ilimap.semantics.tast;

public record RefResolutionBinding(
        String sourceRefClass,                    // A.Eigentuemer
        String targetRefClass,                    // B.Owner  <-- kein infer mehr nötig
```

```
        String classMappingId,                  // "A.Eigentuemer->B.Owner"
        MissingRefPolicy missingRefPolicy
) {
    public enum MissingRefPolicy { WARN, ERROR }
}
```

Wichtig: `targetRefClass` wird hier vom Semantikbinder gesetzt.

`MigrationNodeFactory` liest nur noch ab.

Typed Expressions

Basis

Java

```java
package ch.example.ilimap.semantics.tast;

public sealed interface TExpr permits
        TLiteralExpr, TPathExpr, TVarExpr, TBinaryExpr, TFunctionCallExpr, TMapEnumExpr {
    TType type();
}
```

Literale / Pfade / Variablen

Java

```java
package ch.example.ilimap.semantics.tast;
```

```
public record TLiteralExpr(Object value, TType type) implements TExpr {}
```

‹/› Java

```java
package ch.example.ilimap.semantics.tast;

public record TPathExpr(BoundPath path) implements TExpr {
    @Override public TType type() { return path.type(); }
}
```

‹/› Java

```java
package ch.example.ilimap.semantics.tast;

public record TVarExpr(String name, TType type) implements TExpr {}
```

Binär-Ausdrücke (mit Resultattyp)

‹/› Java

```java
package ch.example.ilimap.semantics.tast;

public record TBinaryExpr(
        TExpr left,
        String op,
        TExpr right,
        TType type
) implements TExpr {}
```

Beispiele:

- `NUMBER >= NUMBER → BOOLEAN`
- `STRING + STRING → STRING`
- `NUMBER + NUMBER → NUMBER`

Funktionsaufruf (signaturgeprüft)

Java

```java
package ch.example.ilimap.semantics.tast;

import java.util.List;

public record TFunctionCallExpr(
        String functionName,
        List<TExpr> args,
        FunctionSignature signature,
        TType type
) implements TExpr {}
```

Java

```java
package ch.example.ilimap.semantics.tast;

import java.util.List;

public record FunctionSignature(
        String name,
        List<TType> paramTypes,
        TType returnType
) {}
```

Das erlaubt dir später:

- statischen Check gegen Registry
- bessere Fehlertexte
- evtl. Spezialisierungen im Node-Bau

`MAP_ENUM` (semantisch reich)

Java

```java
package ch.example.ilimap.semantics.tast;

import java.util.Map;

public record TMapEnumExpr(
        TExpr sourceExpr,
        TEnumType sourceEnumType,                // Quell-Domain bekannt
        TType resultType,                        // oft TEnumType oder STRING
        Map<String, String> mappings,
        String defaultValue,
        String defaultWarnMessage,
        EnumCoverage coverage
) implements TExpr {
    @Override public TType type() { return resultType; }
}
```

Java

```java
package ch.example.ilimap.semantics.tast;

import java.util.Set;
```

```
public record EnumCoverage(
        Set<String> mappedSourceValues,
        Set<String> unmappedKnownSourceValues,   // statisch berechnet, falls Domain beka
        boolean hasDefault
) {}
```

Das ist Gold für `dry-run` und Reviews.

Der eigentliche Schritt: Semantic Binder (Parser-AST → Typed AST)

Das ist mehr als „Checker“: er **prüft + bindet + typisiert**.

Komponenten aufteilen

Ich würde statt einem Monster- `SemanticChecker` lieber 4 Klassen machen:

1) `SymbolTables`

- Klassenregeln (`A.X -> B.Y`)
- lokale Variablen (`LET`)
- Funktionssignaturen

2) `TypeResolver`

- Pfadtyp ermitteln (`source.foo` , `target.bar`)
- Literaltypen
- Operatorresultattypen
- Funktionssignaturen prüfen

3) `BindingResolver`

- Referenzauflösung (`A.Eigentuemer -> B.Owner`)
- `REF` -Binding
- `MAP_ENUM` Domains anreichern

4) `TypedAstBuilder`

- baut `TMigration` , `TClassRule` , `TStmt` , `TExpr`

Beispiel-Skelett: `SemanticBinder`

Java

```java
package ch.example.ilimap.semantics;

import ch.example.ilimap.model.SchemaRegistry;
import ch.example.ilimap.parser.ast.MigrationAst;
import ch.example.ilimap.semantics.tast.TMigration;

public final class SemanticBinder {
    private final SchemaRegistry schemaA;
    private final SchemaRegistry schemaB;
    private final FunctionCatalog functions;

    public SemanticBinder(SchemaRegistry schemaA, SchemaRegistry schemaB, FunctionCatalog
        this.schemaA = schemaA;
        this.schemaB = schemaB;
        this.functions = functions;
    }

    public BindResult bind(MigrationAst ast) {
        DiagnosticBag diags = new DiagnosticBag();
```

```
        // 1) class mapping registry aufbauen
        ClassMappingRegistry classMappings = ClassMappingRegistry.from(ast, diags);

        // 2) typed AST bauen
        TypedAstBuilder builder = new TypedAstBuilder(schemaA, schemaB, functions, classM
        TMigration migration = builder.build(ast);

        return new BindResult(migration, diags);
    }

    public record BindResult(TMigration migration, DiagnosticBag diagnostics) {
        public boolean isOk() { return diagnostics.errors().isEmpty(); }
    }
}
```

Diagnostik-Modell (sehr wichtig)

Wenn du typed AST einführst, bau gleich **richtige Diagnostik** mit Source-Lokation.

Java

```
package ch.example.ilimap.semantics;

import java.util.ArrayList;
import java.util.List;

public final class DiagnosticBag {
    private final List<Diagnostic> errors = new ArrayList<>();
    private final List<Diagnostic> warnings = new ArrayList<>();

    public void error(String code, String message, SourceSpan span) {
        errors.add(new Diagnostic(Diagnostic.Severity.ERROR, code, message, span));
```

```
    }

    public void warn(String code, String message, SourceSpan span) {
        warnings.add(new Diagnostic(Diagnostic.Severity.WARN, code, message, span));
    }

    public List<Diagnostic> errors() { return List.copyOf(errors); }
    public List<Diagnostic> warnings() { return List.copyOf(warnings); }
}
```

‹/› Java

```
package ch.example.ilimap.semantics;

public record Diagnostic(
        Severity severity,
        String code,        // z.B. ILIMAP-TYPE-003
        String message,
        SourceSpan span
) {
    public enum Severity { ERROR, WARN }
}
```

‹/› Java

```
package ch.example.ilimap.semantics;

public record SourceSpan(String file, int line, int column) {}
```

Wichtig: Dafür sollte dein Parser-AST optional `SourceSpan` tragen (mind. auf Statements/Exprs).

Wie `REF` mit typed AST sauber wird (dein Hauptproblem)

Nehmen wir:

Plain text

```
REF target.owner_ref = RESOLVE A.Eigentuemer BY source.eigentuemer_ref;
```

Der Binder prüft:

1. `target.owner_ref` existiert in Zielklasse `B.Building` ?
2. Typ davon ist `REFERENCE(B.Owner)` ?
3. `source.eigentuemer_ref` existiert in Quellklasse `A.Gebaeude` ?
4. Typ davon ist `REFERENCE(A.Eigentuemer)` oder TID-String?
5. Gibt es eine Klassenregel `A.Eigentuemer -> B.Owner` ?

Dann erzeugt er:

Java

```
new TRefStmt(
  targetRefPath = BoundPath(TARGET, "B.Building", "owner_ref", new TReferenceType("B.Owne
  sourceRefExpr = ...,
  resolutionBinding = new RefResolutionBinding(
      "A.Eigentuemer",
      "B.Owner",
      "A.Eigentuemer->B.Owner",
      WARN
  )
)
```

Und **genau damit** ist `inferTargetRefClass(...)` erledigt.

`MigrationNodeFactory` nach typed AST wird viel einfacher

Statt Parser-AST zu interpretieren:

Vorher

- Strings parsen
- Heuristiken
- Sonderfälle

Nachher

- nur noch „mechanisches Mapping“ typed AST → Nodes

Beispiel `TRefStmt`

Java

```java
private static StmtNode buildStmt(TStmt stmt) {
    return switch (stmt) {
        case TRefStmt s -> new RefStmtNode(
                ch.example.ilimap.runtime.Path.of(s.targetRefPath().path()),
                s.resolutionBinding().sourceRefClass(),
                s.resolutionBinding().targetRefClass(),
                buildExpr(s.sourceRefExpr())
        );
        // ...
    };
}
```

Kein Infer. Kein Switch-Hardcode. Kein Raten.

Zusätzlicher Gewinn: implizite Konversionen explizit machen

Ein typed AST erlaubt dir, **Conversions** nicht „heimlich“ im Runtime-Node zu machen, sondern im Binder festzulegen.

Beispiel:

```
Plain text
SET target.code = source.id;   // NUMBER -> STRING
```

Binder entscheidet:

- erlaubt ✅
- `PrimitiveCoercion(NUMBER -> STRING, "numberToString")`

Dann kann `SetStmtNode` gezielt coercen (oder ein `CoercingExprNode` wird eingefügt).

Das ist deutlich stabiler.

Empfehlung: Typed AST als „annotated immutable IR“

Mach das typed AST möglichst:

- **immutable** (Records sind super)
- **explizit** (Bindings/Typen direkt drin)
- **langweilig** (kein Verhalten, nur Daten)

Dann kannst du es:

- testen
- serialisieren (später)
- debuggen (`--dump-typed-ast`)
- diffen zwischen Versionen

Praktisches Vorgehen für den Umbau (ohne Big Bang)

Du musst nicht alles auf einmal umstellen.

Schritt 1

Nur `REF` über typed AST lösen:

- `TRefStmt`
- `RefResolutionBinding`
- `inferTargetRefClass(...)` entfernen

Schritt 2

Pfad-Ausdrücke typisieren:

- `TPathExpr` , `BoundPath`

Schritt 3

`MAP_ENUM` anreichern:

- Domains + Coverage

Schritt 4

Funktionssignaturen typisieren:

- `TFunctionCallExpr`

So bekommst du schnell echten Mehrwert.

Konkrete Module im Repo ergänzen

Ich würde ein neues Modul einführen:

Plain text

```
ilimap-semantics/
  ├─ ... checker ...
  ├─ ... binder ...
  ├─ tast/
  │  ├─ TMigration.java
  │  ├─ TClassRule.java
  │  ├─ TStmt...
  │  ├─ TExpr...
  │  ├─ TType...
  │  └─ BoundPath.java
  └─ diagnostics/
```

Und dann:

- `MigrationNodeFactory.build(...)` nimmt künftig `TMigration` statt `MigrationAst`.

Ein sehr nützlicher Bonus: `--explain-rule` / `--dump-typed-ast`

Mit typed AST kannst du später super Features bauen:

- `--dump-typed-ast` (JSON/text)
- `--check-only`
- `--explain-ref-resolve`
- `--enum-coverage-report`

Das macht dein Tool für Fachstellen richtig stark.

Nächster sinnvoller Schritt

Wenn du magst, kann ich dir als Nächstes **konkret den Binder für** `REF` **+** `MAP_ENUM` ausprogrammieren (Skelette + zentrale Methoden), also:

- `ClassMappingRegistry`
- `FunctionCatalog`
- `TypedAstBuilder.bindRefStmt(...)`
- `TypedAstBuilder.bindMapEnumExpr(...)`
- Beispieltests dafür

Das wäre die direkteste Brücke von deinem aktuellen Stand zum typed AST in der Praxis.

Ja, mache das.

Thought for 51s ›

Perfekt — wir gehen jetzt **konkret auf den Binder**.

Unten bekommst du ein **praktisch verwendbares Skelett** für den typed-AST-Binder mit Fokus auf:

- `ClassMappingRegistry`
- `FunctionCatalog`
- `TypedAstBuilder.bindRefStmt(...)`
- `TypedAstBuilder.bindMapEnumExpr(...)`
- Beispieltests (JUnit)

Ich halte es absichtlich **MVP-nah**: robust genug zum Einbauen, aber ohne unnötigen Overhead.

Annahmen

Ich beziehe mich auf die Strukturen aus deinen bisherigen Schritten:

- `SchemaRegistry`, `ClassSchema`, `AttributeSchema`, `ValueType` (aus `ilimap-model`)
- Parser-AST (`MigrationAst`, `ClassRuleAst`, `StmtAst`, `ExprAst`, `RefStmtAst`, `MapEnumExprAst`, …)
- Typed-AST (`TMigration`, `TClassRule`, `TRefStmt`, `TMapEnumExpr`, `BoundPath`, `TType`, …)

Wenn bei dir Paketnamen leicht anders sind: einfach anpassen.

---

1) `ClassMappingRegistry`

Diese Klasse macht aus den `CLASS A.X -> B.Y` Regeln ein sauberes Registry, damit `REF`-Binding **ohne Heuristik** funktioniert.

Java

```java
package ch.example.ilimap.semantics;

import ch.example.ilimap.parser.ast.ClassRuleAst;
import ch.example.ilimap.parser.ast.MigrationAst;

import java.util.*;

public final class ClassMappingRegistry {

    public record ClassMapping(String sourceClass, String targetClass, String mappingId)

    private final Map<String, ClassMapping> bySourceClass = new HashMap<>();
    private final Map<String, ClassMapping> byMappingId = new HashMap<>();

    private ClassMappingRegistry() {}

    public static ClassMappingRegistry from(MigrationAst ast, DiagnosticBag diags) {
        ClassMappingRegistry reg = new ClassMappingRegistry();

        for (ClassRuleAst r : ast.classRules()) {
            String source = r.sourceClass();
            String target = r.targetClass();
            String mappingId = source + "->" + target;

            ClassMapping existing = reg.bySourceClass.get(source);
            if (existing != null && !existing.targetClass().equals(target)) {
                diags.error(
                        "ILIMAP-BIND-CLASSMAP-001",
                        "Mehrdeutiges Klassenmapping für " + source + ": "
                                + existing.targetClass() + " und " + target,
                        null
                );
                continue;
```

```
        }

        ClassMapping cm = new ClassMapping(source, target, mappingId);
        reg.bySourceClass.put(source, cm);
        reg.byMappingId.put(mappingId, cm);
    }

    return reg;
}

public Optional<ClassMapping> findBySourceClass(String sourceClass) {
    return Optional.ofNullable(bySourceClass.get(sourceClass));
}

public ClassMapping requireBySourceClass(String sourceClass, DiagnosticBag diags, Sou
    return findBySourceClass(sourceClass).orElseGet(() -> {
        diags.error(
                "ILIMAP-BIND-CLASSMAP-404",
                "Kein Klassenmapping für Quellklasse gefunden: " + sourceClass,
                span
        );
        return null;
    });
}

public Optional<String> findTargetClassForSource(String sourceClass) {
    return findBySourceClass(sourceClass).map(ClassMapping::targetClass);
}

public boolean hasMapping(String sourceClass, String targetClass) {
    return byMappingId.containsKey(sourceClass + "->" + targetClass);
}

public Collection<ClassMapping> all() {
    return List.copyOf(byMappingId.values());
```

```
    }
}
```

---

2) `FunctionCatalog`

Der Binder braucht eine **statische Sicht** auf Funktionen (Signaturen / Rückgabetypen).

Für den MVP reicht ein einfaches Katalogmodell.

‹/› Java

```java
package ch.example.ilimap.semantics;

import ch.example.ilimap.semantics.tast.*;

import java.util.*;

public final class FunctionCatalog {

    public record FunctionSpec(
            String name,
            int minArgs,
            int maxArgs,
            List<TType> paramTypes,   // optional / best effort
            TType returnType
    ) {
        public boolean acceptsArity(int arity) {
            return arity >= minArgs && arity <= maxArgs;
        }
    }

    private final Map<String, FunctionSpec> specs = new HashMap<>();
```

```
    public FunctionCatalog register(FunctionSpec spec) {
        specs.put(spec.name(), spec);
        return this;
    }

    public Optional<FunctionSpec> find(String name) {
        return Optional.ofNullable(specs.get(name));
    }

    public FunctionSpec require(String name, DiagnosticBag diags, SourceSpan span) {
        return find(name).orElseGet(() -> {
            diags.error(
                    "ILIMAP-BIND-FUNC-404",
                    "Unbekannte Funktion: " + name,
                    span
            );
            return null;
        });
    }

    public static FunctionCatalog defaultCatalog() {
        TPrimitiveType str = new TPrimitiveType(TPrimitiveType.Kind.STRING, true);
        TPrimitiveType num = new TPrimitiveType(TPrimitiveType.Kind.NUMBER, true);
        TPrimitiveType bool = new TPrimitiveType(TPrimitiveType.Kind.BOOLEAN, false);

        return new FunctionCatalog()
                .register(new FunctionSpec("normalize_text", 1, 1, List.of(str), str))
                .register(new FunctionSpec("parse_street", 1, 1, List.of(str), str))
                .register(new FunctionSpec("parse_house_no", 1, 1, List.of(str), str))
                .register(new FunctionSpec("to_m2", 2, 2, List.of(num, str), num))
                .register(new FunctionSpec("split_name", 1, 1, List.of(str), new TListTyp
                .register(new FunctionSpec("first", 1, 1, List.of(new TListType(str, fals
                .register(new FunctionSpec("last", 1, 1, List.of(new TListType(str, false
                .register(new FunctionSpec("__not", 1, 1, List.of(bool), bool));
```

```
    }
}
```

---

3) `TypedAstBuilder` (Fokus: `REF` + `MAP_ENUM`)

Das hier ist das Herzstück. Ich zeige dir:

- Struktur des Builders
- `bindRefStmt(...)`
- `bindMapEnumExpr(...)`
- wichtige Hilfsmethoden (`bindPath` , `valueTypeToTType` , etc.)

Ich nehme `SourceSpan` hier optional mit `null` , weil dein Parser-AST im aktuellen Stand noch keine Spans trägt.

Java

```java
package ch.example.ilimap.semantics;

import ch.example.ilimap.model.*;
import ch.example.ilimap.parser.ast.*;
import ch.example.ilimap.semantics.tast.*;

import java.util.*;
import java.util.stream.Collectors;

public final class TypedAstBuilder {

    private final SchemaRegistry schemaA;
    private final SchemaRegistry schemaB;
    private final FunctionCatalog functions;
```

```
    private final ClassMappingRegistry classMappings;
    private final DiagnosticBag diags;

    public TypedAstBuilder(SchemaRegistry schemaA,
                           SchemaRegistry schemaB,
                           FunctionCatalog functions,
                           ClassMappingRegistry classMappings,
                           DiagnosticBag diags) {
        this.schemaA = schemaA;
        this.schemaB = schemaB;
        this.functions = functions;
        this.classMappings = classMappings;
        this.diags = diags;
    }

    public TMigration build(MigrationAst ast) {
        List<TClassRule> rules = new ArrayList<>();
        for (ClassRuleAst cr : ast.classRules()) {
            TClassRule tr = bindClassRule(cr);
            if (tr != null) rules.add(tr);
        }
        return new TMigration(ast.sourceModel(), ast.targetModel(), rules);
    }

    private TClassRule bindClassRule(ClassRuleAst cr) {
        ClassSchema srcClass = schemaA.findClass(cr.sourceClass()).orElse(null);
        ClassSchema tgtClass = schemaB.findClass(cr.targetClass()).orElse(null);

        if (srcClass == null) {
            diags.error("ILIMAP-BIND-CLASS-404", "Unbekannte Quellklasse: " + cr.sourceCl
            return null;
        }
        if (tgtClass == null) {
            diags.error("ILIMAP-BIND-CLASS-404", "Unbekannte Zielklasse: " + cr.targetCla
            return null;
```

```
        }

        RuleBindContext ctx = new RuleBindContext(srcClass, tgtClass);

        TKeyMapping key = bindKeyMapping(cr.keyMapping(), ctx);

        TExpr when = null;
        if (cr.whenExpr() != null) {
            when = bindExpr(cr.whenExpr(), ctx, expectedBoolean());
            if (when != null && !(when.type() instanceof TPrimitiveType p && p.kind() ==
                diags.error("ILIMAP-BIND-TYPE-BOOL", "WHEN-Ausdruck muss BOOLEAN sein", r
            }
        }

        List<TStmt> stmts = new ArrayList<>();
        for (StmtAst s : cr.statements()) {
            TStmt ts = bindStmt(s, ctx);
            if (ts != null) stmts.add(ts);
        }

        return new TClassRule(
                cr.sourceClass(),
                cr.targetClass(),
                key,
                when,
                stmts,
                new RuleBinding(cr.sourceClass() + "->" + cr.targetClass())
        );
    }

    private TKeyMapping bindKeyMapping(KeyMappingAst km, RuleBindContext ctx) {
        BoundPath src = bindSourcePath(km.sourcePath(), ctx);
        BoundPath tgt = bindTargetPath(km.targetPath(), ctx);

        // Best effort Type-Check
```

```
        if (src != null && tgt != null) {
            if (!isAssignable(tgt.type(), src.type())) {
                diags.warn(
                        "ILIMAP-BIND-KEY-TYPE",
                        "KEY-Mapping hat möglicherweise inkompatible Typen: source="
                                + renderType(src.type()) + ", target=" + renderType(tgt.t
                        null
                );
            }
        }

        return new TKeyMapping(src, tgt);
    }

    private TStmt bindStmt(StmtAst s, RuleBindContext ctx) {
        return switch (s) {
            case SetStmtAst x -> bindSetStmt(x, ctx);
            case LetStmtAst x -> bindLetStmt(x, ctx);
            case AssertStmtAst x -> bindAssertStmt(x, ctx);
            case RefStmtAst x -> bindRefStmt(x, ctx);
        };
    }

    private TStmt bindSetStmt(SetStmtAst s, RuleBindContext ctx) {
        BoundPath targetPath = bindTargetPath(s.targetPath(), ctx);
        if (targetPath == null) return null;

        TExpr expr = bindExpr(s.expr(), ctx, targetPath.type());
        if (expr == null) return null;

        Coercion coercion = deriveCoercion(expr.type(), targetPath.type());
        if (coercion == null) {
            diags.error(
                    "ILIMAP-BIND-SET-TYPE",
                    "Typinkompatible Zuweisung auf target." + targetPath.path()
```

```
                        + ": " + renderType(expr.type()) + " -> " + renderType(target
                null
        );
        return null;
    }

    return new TSetStmt(targetPath, expr, coercion);
}

private TStmt bindLetStmt(LetStmtAst s, RuleBindContext ctx) {
    TExpr expr = bindExpr(s.expr(), ctx, null);
    if (expr == null) return null;
    ctx.locals.put(s.varName(), expr.type());
    return new TLetStmt(s.varName(), expr, expr.type());
}

private TStmt bindAssertStmt(AssertStmtAst s, RuleBindContext ctx) {
    TExpr cond = bindExpr(s.condition(), ctx, expectedBoolean());
    if (cond == null) return null;

    if (!(cond.type() instanceof TPrimitiveType p && p.kind() == TPrimitiveType.Kind.
        diags.error("ILIMAP-BIND-ASSERT-TYPE", "ASSERT-Bedingung muss BOOLEAN sein",
        return null;
    }

    return new TAssertStmt(
            cond,
            s.severity() == AssertStmtAst.SeverityAst.ERROR ? TAssertStmt.Severity.ER
            s.message()
    );
}

// ============================================================
// REF binding (wichtig)
// ============================================================
```

```
private TStmt bindRefStmt(RefStmtAst s, RuleBindContext ctx) {
    // target.owner_ref
    BoundPath targetRefPath = bindTargetPath(s.targetRefPath(), ctx);
    if (targetRefPath == null) return null;

    // Zielpfad muss Referenz sein
    if (!(targetRefPath.type() instanceof TReferenceType targetRefType)) {
        diags.error(
                "ILIMAP-BIND-REF-TARGET-NOTREF",
                "REF-Zielpfad ist keine Referenz: target." + s.targetRefPath(),
                null
        );
        return null;
    }

    // source.eigentuemer_ref (oder TID-String)
    String sourceExprPath = s.sourceRefExprPath();
    if (!sourceExprPath.startsWith("source.")) {
        diags.error(
                "ILIMAP-BIND-REF-SOURCE-SCOPE",
                "REF BY erwartet source.<pfad>, gefunden: " + sourceExprPath,
                null
        );
        return null;
    }

    String sourcePathOnly = sourceExprPath.substring("source.".length());
    TExpr sourceRefExpr = bindExpr(new SourcePathAst(sourcePathOnly), ctx, null);
    if (sourceRefExpr == null) return null;

    // explizite Quellreferenzklasse aus DSL: RESOLVE A.Eigentuemer
    String sourceRefClass = s.sourceRefClass();

    // Klassenmapping für Quellreferenzklasse finden: A.Eigentuemer -> B.Owner
    ClassMappingRegistry.ClassMapping refClassMapping =
```

```
        classMappings.requireBySourceClass(sourceRefClass, diags, null);
if (refClassMapping == null) return null;

String resolvedTargetRefClass = refClassMapping.targetClass();

// Konsistenzcheck: target.owner_ref zeigt auf B.Owner? (aus Zielmodell)
if (!Objects.equals(targetRefType.targetClassQName(), resolvedTargetRefClass)) {
    diags.error(
            "ILIMAP-BIND-REF-TARGET-MISMATCH",
            "REF-Zielpfad erwartet " + targetRefType.targetClassQName()
                    + ", aber Klassenmapping für " + sourceRefClass
                    + " liefert " + resolvedTargetRefClass,
            null
    );
    return null;
}

// Konsistenzcheck Quelle: wenn sourceRefExpr ein REF-Typ ist, muss er auf source
if (sourceRefExpr.type() instanceof TReferenceType sourceRefType) {
    if (!Objects.equals(sourceRefType.targetClassQName(), sourceRefClass)) {
        diags.error(
                "ILIMAP-BIND-REF-SOURCE-MISMATCH",
                "REF BY source-Pfad referenziert " + sourceRefType.targetClassQNa
                        + ", aber RESOLVE gibt " + sourceRefClass + " an",
                null
        );
        return null;
    }
} else if (!(sourceRefExpr.type() instanceof TPrimitiveType p
        && p.kind() == TPrimitiveType.Kind.STRING)) {
    diags.warn(
            "ILIMAP-BIND-REF-SOURCE-NONSTRING",
            "REF BY source-Ausdruck ist weder REF noch STRING-TID: " + renderType
            null
    );
```

```
        }

        // Policy später aus OPTIONS lesen; MVP = WARN
        RefResolutionBinding binding = new RefResolutionBinding(
                sourceRefClass,
                resolvedTargetRefClass,
                refClassMapping.mappingId(),
                RefResolutionBinding.MissingRefPolicy.WARN
        );

        return new TRefStmt(targetRefPath, sourceRefExpr, binding);
    }

    // ============================================================
    // MAP_ENUM binding (wichtig)
    // ============================================================
    private TExpr bindMapEnumExpr(MapEnumExprAst m, RuleBindContext ctx, TType expectedTy
        TExpr sourceExpr = bindExpr(m.sourceExpr(), ctx, null);
        if (sourceExpr == null) return null;

        // Quelltyp sollte ENUM sein (idealerweise)
        TEnumType sourceEnumType = null;
        if (sourceExpr.type() instanceof TEnumType et) {
            sourceEnumType = et;
        } else if (sourceExpr.type() instanceof TPrimitiveType p && p.kind() == TPrimitiv
            diags.warn(
                    "ILIMAP-BIND-MAPENUM-SOURCE-STRING",
                    "MAP_ENUM-Quelle ist STRING statt ENUM; statische Abdeckungsprüfung e
                    null
            );
        } else {
            diags.error(
                    "ILIMAP-BIND-MAPENUM-SOURCE-TYPE",
                    "MAP_ENUM erwartet ENUM/STRING als Quelle, gefunden: " + renderType(s
                    null
```

```
        );
        return null;
    }

    // Zieltyp (Resultattyp) aus expectedType ableiten, sonst STRING
    TType resultType = expectedType != null
            ? expectedType
            : new TPrimitiveType(TPrimitiveType.Kind.STRING, true);

    // Mappings sammeln + Duplikate prüfen
    Map<String, String> mappings = new LinkedHashMap<>();
    for (MapEnumExprAst.Entry e : m.entries()) {
        String prev = mappings.put(e.sourceValue(), e.targetValue());
        if (prev != null) {
            diags.error(
                    "ILIMAP-BIND-MAPENUM-DUPSRC",
                    "MAP_ENUM enthält doppelten Quellwert: " + e.sourceValue(),
                    null
            );
        }
    }

    // Quellwerte gegen bekannte Enum-Domain prüfen
    Set<String> mappedSourceValues = new LinkedHashSet<>(mappings.keySet());
    Set<String> unmappedKnownSourceValues = new LinkedHashSet<>();

    if (sourceEnumType != null) {
        Set<String> allowed = sourceEnumType.allowedValues();

        for (String srcVal : mappedSourceValues) {
            if (!allowed.contains(srcVal)) {
                diags.warn(
                        "ILIMAP-BIND-MAPENUM-SRC-UNKNOWN",
                        "MAP_ENUM mappt Quellwert '" + srcVal + "', der nicht in der
                                + sourceEnumType.enumName() + "' vorkommt",
```

```
                        null
                );
            }
        }

        unmappedKnownSourceValues.addAll(allowed);
        unmappedKnownSourceValues.removeAll(mappedSourceValues);

        if (!unmappedKnownSourceValues.isEmpty() && m.defaultValue() == null) {
            diags.warn(
                    "ILIMAP-BIND-MAPENUM-INCOMPLETE",
                    "MAP_ENUM deckt nicht alle bekannten Quellwerte ab und hat kein D
                            + unmappedKnownSourceValues,
                    null
            );
        }
    }

    // Zielwerte gegen erwarteten Enum-Typ prüfen (falls expectedType Enum ist)
    if (resultType instanceof TEnumType targetEnumType) {
        Set<String> allowedTarget = targetEnumType.allowedValues();

        for (String targetVal : mappings.values()) {
            if (!allowedTarget.contains(targetVal)) {
                diags.error(
                        "ILIMAP-BIND-MAPENUM-TGT-INVALID",
                        "MAP_ENUM Zielwert '" + targetVal + "' ist nicht erlaubt in Z
                                + targetEnumType.enumName() + "'",
                        null
                );
            }
        }

        if (m.defaultValue() != null && !allowedTarget.contains(m.defaultValue())) {
            diags.error(
```

```
                        "ILIMAP-BIND-MAPENUM-DEFAULT-INVALID",
                        "MAP_ENUM DEFAULT-Zielwert '" + m.defaultValue()
                                + "' ist nicht erlaubt in Ziel-Domain '" + targetEnumType
                        null
                );
            }
        }

        EnumCoverage coverage = new EnumCoverage(
                mappedSourceValues,
                unmappedKnownSourceValues,
                m.defaultValue() != null
        );

        return new TMapEnumExpr(
                sourceExpr,
                sourceEnumType, // kann null sein, wenn Quelle nur STRING
                resultType,
                Map.copyOf(mappings),
                m.defaultValue(),
                m.defaultWarnMessage(),
                coverage
        );
    }

    // ============================================================
    // Expr binding (gekürzt, aber funktional)
    // ============================================================
    private TExpr bindExpr(ExprAst expr, RuleBindContext ctx, TType expectedType) {
        return switch (expr) {
            case LiteralAst l -> bindLiteral(l);
            case SourcePathAst p -> {
                BoundPath bp = bindSourcePath(p.path(), ctx);
                yield bp == null ? null : new TPathExpr(bp);
            }
```

```
            case TargetPathAst p -> {
                BoundPath bp = bindTargetPath(p.path(), ctx);
                yield bp == null ? null : new TPathExpr(bp);
            }
            case VarAst v -> bindVar(v, ctx);
            case BinaryExprAst b -> bindBinaryExpr(b, ctx);
            case FunctionCallAst f -> bindFunctionCallExpr(f, ctx);
            case MapEnumExprAst m -> bindMapEnumExpr(m, ctx, expectedType);
        };
    }

    private TExpr bindLiteral(LiteralAst l) {
        Object v = l.value();
        if (v == null) return new TLiteralExpr(null, new TPrimitiveType(TPrimitiveType.Ki
        if (v instanceof Boolean) return new TLiteralExpr(v, expectedBoolean());
        if (v instanceof Number) return new TLiteralExpr(v, new TPrimitiveType(TPrimitive
        return new TLiteralExpr(v, new TPrimitiveType(TPrimitiveType.Kind.STRING, false))
    }

    private TExpr bindVar(VarAst v, RuleBindContext ctx) {
        TType t = ctx.locals.get(v.name());
        if (t == null) {
            diags.error("ILIMAP-BIND-VAR-404", "Unbekannte LET-Variable: " + v.name(), nu
            return null;
        }
        return new TVarExpr(v.name(), t);
    }

    private TExpr bindFunctionCallExpr(FunctionCallAst f, RuleBindContext ctx) {
        FunctionCatalog.FunctionSpec spec = functions.require(f.name(), diags, null);
        if (spec == null) return null;

        if (!spec.acceptsArity(f.args().size())) {
            diags.error(
                    "ILIMAP-BIND-FUNC-ARITY",
```

```
                "Funktion " + f.name() + " erwartet " + spec.minArgs() + ".." + spec.
                        + " Argumente, erhalten: " + f.args().size(),
                null
            );
            return null;
        }

        List<TExpr> args = new ArrayList<>();
        for (ExprAst a : f.args()) {
            TExpr ta = bindExpr(a, ctx, null);
            if (ta == null) return null;
            args.add(ta);
        }

        FunctionSignature sig = new FunctionSignature(
                spec.name(),
                spec.paramTypes(),
                spec.returnType()
        );

        return new TFunctionCallExpr(f.name(), List.copyOf(args), sig, spec.returnType()
    }

    private TExpr bindBinaryExpr(BinaryExprAst b, RuleBindContext ctx) {
        TExpr left = bindExpr(b.left(), ctx, null);
        TExpr right = bindExpr(b.right(), ctx, null);
        if (left == null || right == null) return null;

        String op = b.op();

        // Vergleich / bool
        if (Set.of("==", "!=", "<", ">", "<=", ">=").contains(op)) {
            return new TBinaryExpr(left, op, right, expectedBoolean());
        }
        if (Set.of("AND", "OR").contains(op)) {
```

```
            // Grober Check
            if (!isBoolean(left.type()) || !isBoolean(right.type())) {
                diags.error("ILIMAP-BIND-BIN-BOOL", "AND/OR erwartet BOOLEAN Operanden",
                return null;
            }
            return new TBinaryExpr(left, op, right, expectedBoolean());
        }

        // Arithmetik / String-Konkatenation
        if ("+".equals(op)) {
            if (isString(left.type()) || isString(right.type())) {
                return new TBinaryExpr(left, op, right, new TPrimitiveType(TPrimitiveType
            }
            return new TBinaryExpr(left, op, right, new TPrimitiveType(TPrimitiveType.Kir
        }
        if (Set.of("-", "*", "/").contains(op)) {
            return new TBinaryExpr(left, op, right, new TPrimitiveType(TPrimitiveType.Kir
        }

        diags.error("ILIMAP-BIND-BIN-OP", "Unbekannter Operator: " + op, null);
        return null;
    }

    // ============================================================
    // Path binding
    // ============================================================
    private BoundPath bindSourcePath(String path, RuleBindContext ctx) {
        return bindPath(BoundPath.ScopeKind.SOURCE, ctx.sourceClass, path);
    }

    private BoundPath bindTargetPath(String path, RuleBindContext ctx) {
        return bindPath(BoundPath.ScopeKind.TARGET, ctx.targetClass, path);
    }

    private BoundPath bindPath(BoundPath.ScopeKind scope, ClassSchema ownerClass, String
```

```
        // Sonderfall tid (nicht zwingend als Attribut im Schema modelliert)
        if ("tid".equals(path)) {
            return new BoundPath(
                    scope,
                    ownerClass.qName(),
                    path,
                    new TPrimitiveType(TPrimitiveType.Kind.STRING, false)
            );
        }

        AttributeSchema attr = ownerClass.findAttribute(path).orElse(null);
        if (attr == null) {
            diags.error(
                    "ILIMAP-BIND-PATH-404",
                    "Unbekannter Pfad " + scope.name().toLowerCase() + "." + path
                            + " in Klasse " + ownerClass.qName(),
                    null
            );
            return null;
        }

        return new BoundPath(
                scope,
                ownerClass.qName(),
                path,
                valueTypeToTType(attr)
        );
    }

    private TType valueTypeToTType(AttributeSchema a) {
        if (a.reference()) {
            return new TReferenceType(a.referenceTargetClass(), true);
        }

        return switch (a.type()) {
```

```
            case STRING -> new TPrimitiveType(TPrimitiveType.Kind.STRING, false);
            case NULLABLE_STRING -> new TPrimitiveType(TPrimitiveType.Kind.STRING, true);
            case NUMBER -> new TPrimitiveType(TPrimitiveType.Kind.NUMBER, false);
            case NULLABLE_NUMBER -> new TPrimitiveType(TPrimitiveType.Kind.NUMBER, true);
            case BOOLEAN -> new TPrimitiveType(TPrimitiveType.Kind.BOOLEAN, false);
            case ENUM -> new TEnumType(
                    a.path(), // später schöner: Domain-Name
                    a.enumValues() == null ? Set.of() : a.enumValues(),
                    true
            );
            case REFERENCE -> new TReferenceType(a.referenceTargetClass(), true);
            case OBJECT -> new TObjectType(a.path(), true);
        };
    }

    // ============================================================
    // Typ-Kompatibilität / Coercions (MVP)
    // ============================================================
    private Coercion deriveCoercion(TType from, TType to) {
        if (from == null || to == null) return null;
        if (sameType(from, to)) return new NoCoercion();

        // ENUM -> STRING
        if (from instanceof TEnumType && to instanceof TPrimitiveType p && p.kind() == TP
            return new PrimitiveCoercion(from, to, "enumToString");
        }

        // STRING -> ENUM nicht implizit erlauben (MVP)
        if (from instanceof TPrimitiveType p && p.kind() == TPrimitiveType.Kind.STRING &&
            return null;
        }

        // NUMBER -> STRING (optional erlauben)
        if (from instanceof TPrimitiveType pf && pf.kind() == TPrimitiveType.Kind.NUMBER
                && to instanceof TPrimitiveType pt && pt.kind() == TPrimitiveType.Kind.ST
```

```
            return new PrimitiveCoercion(from, to, "numberToString");
        }

        // Referenzen als TID-String erlaubt?
        if (from instanceof TReferenceType && to instanceof TPrimitiveType pt2 && pt2.kir
            return new PrimitiveCoercion(from, to, "refTidToString");
        }

        return null;
    }

    private boolean isAssignable(TType to, TType from) {
        return deriveCoercion(from, to) != null;
    }

    private boolean sameType(TType a, TType b) {
        return Objects.equals(a, b);
    }

    private static TPrimitiveType expectedBoolean() {
        return new TPrimitiveType(TPrimitiveType.Kind.BOOLEAN, false);
    }

    private boolean isBoolean(TType t) {
        return t instanceof TPrimitiveType p && p.kind() == TPrimitiveType.Kind.BOOLEAN;
    }

    private boolean isString(TType t) {
        return t instanceof TPrimitiveType p && p.kind() == TPrimitiveType.Kind.STRING;
    }

    private String renderType(TType t) {
        if (t == null) return "<null>";
        return t.toString();
    }
```

```
    // ============================================================
    // Pro-Klassenregel Kontext
    // ============================================================
    private static final class RuleBindContext {
        final ClassSchema sourceClass;
        final ClassSchema targetClass;
        final Map<String, TType> locals = new HashMap<>();

        RuleBindContext(ClassSchema sourceClass, ClassSchema targetClass) {
            this.sourceClass = sourceClass;
            this.targetClass = targetClass;
        }
    }
}
```

---

4) `SemanticBinder` (so verwendest du den Builder)

Falls du die Klasse noch nicht finalisiert hast:

Java

```
package ch.example.ilimap.semantics;

import ch.example.ilimap.model.SchemaRegistry;
import ch.example.ilimap.parser.ast.MigrationAst;
import ch.example.ilimap.semantics.tast.TMigration;

public final class SemanticBinder {
    private final SchemaRegistry schemaA;
    private final SchemaRegistry schemaB;
    private final FunctionCatalog functions;
```

```
    public SemanticBinder(SchemaRegistry schemaA, SchemaRegistry schemaB, FunctionCatalog
        this.schemaA = schemaA;
        this.schemaB = schemaB;
        this.functions = functions;
    }

    public BindResult bind(MigrationAst ast) {
        DiagnosticBag diags = new DiagnosticBag();
        ClassMappingRegistry mappings = ClassMappingRegistry.from(ast, diags);

        TypedAstBuilder builder = new TypedAstBuilder(schemaA, schemaB, functions, mappin
        TMigration migration = builder.build(ast);

        return new BindResult(migration, diags);
    }

    public record BindResult(TMigration migration, DiagnosticBag diagnostics) {
        public boolean isOk() {
            return diagnostics.errors().isEmpty();
        }
    }
}
```

---

5) JUnit-Tests für `REF` und `MAP_ENUM`

Damit du schnell Vertrauen bekommst, dass der Binder wirklich macht, was er soll.

Test 1: `REF` wird sauber gebunden (kein Infer mehr nötig)

Java

```java
package ch.example.ilimap.semantics;

import ch.example.ilimap.model.*;
import ch.example.ilimap.parser.ast.*;
import ch.example.ilimap.semantics.tast.*;
import org.junit.jupiter.api.Test;

import java.util.List;
import java.util.Map;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;

class SemanticBinderRefTest {

    @Test
    void bindsRefStmtWithResolvedTargetRefClassFromClassMapping() {
        // Schema A
        SchemaRegistry schemaA = new SchemaRegistry();
        schemaA.register(new ClassSchema(
                "A.Gebaeude",
                Map.of(
                        "eigentuemer_ref", new AttributeSchema(
                                "eigentuemer_ref",
                                ValueType.REFERENCE,
                                Set.of(),
                                true,
                                "A.Eigentuemer"
                        )
                ),
                Set.of()
        ));
        schemaA.register(new ClassSchema("A.Eigentuemer", Map.of(), Set.of()));

        // Schema B
```

```
SchemaRegistry schemaB = new SchemaRegistry();
schemaB.register(new ClassSchema(
        "B.Building",
        Map.of(
                "owner_ref", new AttributeSchema(
                        "owner_ref",
                        ValueType.REFERENCE,
                        Set.of(),
                        true,
                        "B.Owner"
                )
        ),
        Set.of()
));
schemaB.register(new ClassSchema("B.Owner", Map.of(), Set.of()));

// AST: zwei Klassenregeln (inkl. A.Eigentuemer -> B.Owner)
MigrationAst ast = new MigrationAst(
        "DM_A",
        "DM_B",
        List.of(
                new ClassRuleAst(
                        "A.Gebaeude",
                        "B.Building",
                        new KeyMappingAst("tid", "tid"),
                        null,
                        List.of(
                                new RefStmtAst(
                                        "owner_ref",
                                        "A.Eigentuemer",
                                        "source.eigentuemer_ref"
                                )
                        )
                ),
                new ClassRuleAst(
```

```
                                "A.Eigentuemer",
                                "B.Owner",
                                new KeyMappingAst("tid", "tid"),
                                null,
                                List.of()
                        )
                )
        );

        SemanticBinder binder = new SemanticBinder(schemaA, schemaB, FunctionCatalog.defa
        var result = binder.bind(ast);

        assertTrue(result.isOk(), () -> "Errors: " + result.diagnostics().errors());

        TMigration tm = result.migration();
        TClassRule buildingRule = tm.classRules().stream()
                .filter(r -> r.sourceClass().equals("A.Gebaeude"))
                .findFirst().orElseThrow();

        assertEquals(1, buildingRule.statements().size());
        assertTrue(buildingRule.statements().get(0) instanceof TRefStmt);

        TRefStmt ref = (TRefStmt) buildingRule.statements().get(0);

        assertEquals("owner_ref", ref.targetRefPath().path());
        assertTrue(ref.targetRefPath().type() instanceof TReferenceType tr);
        TReferenceType targetRefType = (TReferenceType) ref.targetRefPath().type();
        assertEquals("B.Owner", targetRefType.targetClassQName());

        assertEquals("A.Eigentuemer", ref.resolutionBinding().sourceRefClass());
        assertEquals("B.Owner", ref.resolutionBinding().targetRefClass());
        assertEquals("A.Eigentuemer->B.Owner", ref.resolutionBinding().classMappingId());
    }
}
```

Test 2: `MAP_ENUM` bekommt Coverage + Zielenum-Prüfung

Java

```java
package ch.example.ilimap.semantics;

import ch.example.ilimap.model.*;
import ch.example.ilimap.parser.ast.*;
import ch.example.ilimap.semantics.tast.*;
import org.junit.jupiter.api.Test;

import java.util.List;
import java.util.Map;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;

class SemanticBinderMapEnumTest {

    @Test
    void bindsMapEnumAndComputesCoverageAgainstSourceAndTargetDomains() {
        // Schema A
        SchemaRegistry schemaA = new SchemaRegistry();
        schemaA.register(new ClassSchema(
                "A.Gebaeude",
                Map.of(
                        "nutzung", new AttributeSchema(
                                "nutzung",
                                ValueType.ENUM,
                                Set.of("Wohnen", "Gewerbe", "Misch"),
                                false,
                                null
                        )
```

```
                ),
                Set.of()
        ));

        // Schema B
        SchemaRegistry schemaB = new SchemaRegistry();
        schemaB.register(new ClassSchema(
                "B.Building",
                Map.of(
                        "usage", new AttributeSchema(
                                "usage",
                                ValueType.ENUM,
                                Set.of("Residential", "Commercial", "Mixed", "Unknown"),
                                false,
                                null
                        )
                ),
                Set.of()
        ));

        // AST: absichtlich "Gewerbe" fehlt -> Coverage soll das zeigen; DEFAULT vorhande
        MigrationAst ast = new MigrationAst(
                "DM_A",
                "DM_B",
                List.of(
                        new ClassRuleAst(
                                "A.Gebaeude",
                                "B.Building",
                                new KeyMappingAst("tid", "tid"),
                                null,
                                List.of(
                                        new SetStmtAst(
                                                "usage",
                                                new MapEnumExprAst(
                                                        new SourcePathAst("nutzung"),
```

```
                                                List.of(
                                                        new MapEnumExprAst.Entry(
                                                        new MapEnumExprAst.Entry(
                                                ),
                                                "Unknown",
                                                "Unbekannte Nutzung"
                                        )
                                )
                        )
                )
        )
);

SemanticBinder binder = new SemanticBinder(schemaA, schemaB, FunctionCatalog.defa
var result = binder.bind(ast);

assertTrue(result.isOk(), () -> "Errors: " + result.diagnostics().errors());

TClassRule rule = result.migration().classRules().get(0);
assertEquals(1, rule.statements().size());
assertTrue(rule.statements().get(0) instanceof TSetStmt);

TSetStmt set = (TSetStmt) rule.statements().get(0);
assertTrue(set.expr() instanceof TMapEnumExpr);

TMapEnumExpr mapEnum = (TMapEnumExpr) set.expr();

assertEquals("Unknown", mapEnum.defaultValue());
assertEquals("Unbekannte Nutzung", mapEnum.defaultWarnMessage());

// Coverage prüfen
assertEquals(Set.of("Wohnen", "Misch"), mapEnum.coverage().mappedSourceValues());
assertEquals(Set.of("Gewerbe"), mapEnum.coverage().unmappedKnownSourceValues());
assertTrue(mapEnum.coverage().hasDefault());
```

```
        // Resultattyp sollte Zieltyp des SETs sein (ENUM von B.Building.usage)
        assertTrue(mapEnum.type() instanceof TEnumType);
        TEnumType resultEnumType = (TEnumType) mapEnum.type();
        assertEquals(Set.of("Residential", "Commercial", "Mixed", "Unknown"), resultEnumT
    }

    @Test
    void reportsErrorForInvalidTargetEnumValueInMapEnum() {
        SchemaRegistry schemaA = new SchemaRegistry();
        schemaA.register(new ClassSchema(
                "A.Gebaeude",
                Map.of(
                        "nutzung", new AttributeSchema("nutzung", ValueType.ENUM,
                                Set.of("Wohnen", "Gewerbe", "Misch"), false, null)
                ),
                Set.of()
        ));

        SchemaRegistry schemaB = new SchemaRegistry();
        schemaB.register(new ClassSchema(
                "B.Building",
                Map.of(
                        "usage", new AttributeSchema("usage", ValueType.ENUM,
                                Set.of("Residential", "Commercial", "Mixed", "Unknown"),
                ),
                Set.of()
        ));

        MigrationAst ast = new MigrationAst(
                "DM_A",
                "DM_B",
                List.of(
                        new ClassRuleAst(
                                "A.Gebaeude",
                                "B.Building",
```

```
                                new KeyMappingAst("tid", "tid"),
                                null,
                                List.of(
                                        new SetStmtAst(
                                                "usage",
                                                new MapEnumExprAst(
                                                        new SourcePathAst("nutzung"),
                                                        List.of(
                                                                new MapEnumExprAst.Entry(
                                                        ),
                                                        "Unknown",
                                                        null
                                                )
                                        )
                                )
                        )
                )
        );

        SemanticBinder binder = new SemanticBinder(schemaA, schemaB, FunctionCatalog.defa
        var result = binder.bind(ast);

        assertFalse(result.isOk());
        assertTrue(
                result.diagnostics().errors().stream()
                        .anyMatch(d -> "ILIMAP-BIND-MAPENUM-TGT-INVALID".equals(d.code()))
                () -> "Expected invalid target enum diagnostic, got: " + result.diagnosti
        );
    }
}
```

---

6) `MigrationNodeFactory` umstellen auf `TMigration`

Sobald du typed AST hast, sollte die Factory **nicht mehr** `MigrationAst` nehmen.

Vorher

```java
MigrationRootNode build(IliMapLanguage lang, MigrationAst ast, Source source)
```

Nachher

```java
MigrationRootNode build(IliMapLanguage lang, TMigration ast, Source source)
```

Und dann bei `TRefStmt`:

```java
case TRefStmt s -> new RefStmtNode(
        ch.example.ilimap.runtime.Path.of(s.targetRefPath().path()),
        s.resolutionBinding().sourceRefClass(),
        s.resolutionBinding().targetRefClass(),   // <-- direkt aus Binding
        buildExpr(s.sourceRefExpr())
);
```

Das ist der eigentliche Gewinn: **keine** `inferTargetRefClass(...)` **-Heuristik mehr**.

---

7) Zwei kleine Erweiterungen, die ich dir als Nächstes empfehlen würde

A) Parser-AST mit `SourceSpan`

Wenn du auf `StmtAst` / `ExprAst` `SourceSpan` mitträgst, werden Diagnosen extrem wertvoll.

Dann kannst du in `TypedAstBuilder` überall statt `null` echte Spans verwenden.

B) `TypedAstBuilder` als 2-stufiger Binder

Der nächste Qualitätssprung wäre:

1. **Name/Path Binding**
2. **Type Checking + Coercion Insertion**

Dann kannst du z. B. implizite Konversionen gezielt als `TCoerceExpr` im typed AST modellieren.

---

8) Was du jetzt praktisch tun kannst

Wenn du diesen Stand einbaust, hast du bereits:

- sauberes **Klassenmapping-Registry**
- `REF` -Binding ohne Heuristik
- `MAP_ENUM` mit **Domain-Checks + Coverage**
- typed AST als saubere Zwischenrepräsentation

Damit bist du an dem Punkt, wo sich das Projekt von „Interpreter-Prototyp“ zu „Compiler/Runner mit echter Semantik“ entwickelt.

Wenn du willst, mache ich als nächsten Schritt den **Umbau der** `MigrationNodeFactory` **auf** `TMigration` **komplett** (inkl. `buildExpr(TExpr)` für `TMapEnumExpr` , `TFunctionCallExpr` , `TBinaryExpr` , `TPathExpr`) — dann ist der typed-AST-Pfad bis Truffle-Ende einmal sauber durchgezogen.

Gerne.

Thought for 33s ›

Perfekt — jetzt ziehen wir den typed-AST-Pfad **bis in die Truffle-Nodes** sauber durch.
Das ist der Punkt, an dem dein System wirklich „Compiler-ähnlich“ wird.

Ich gebe dir:

1. **Umbau** `MigrationNodeFactory` **auf** `TMigration`
2. `buildStmt(TStmt)` **/** `buildExpr(TExpr)` **komplett**
3. `CoerceExprNode` **für typed Coercions**
4. **Patch für** `IliMapLanguage.parse(...)` **(Parser-AST → Binder → Typed AST → Nodes)**
5. **ein kleiner Test-Ansatz für den neuen Pfad**

---

1) Neue Zielrichtung der Factory

Vorher

- `MigrationNodeFactory.build(..., MigrationAst ast, ...)`

Nachher

- `MigrationNodeFactory.build(..., TMigration tast, ...)`

Die Factory soll jetzt **nur noch mechanisch** typed AST in Nodes übersetzen.

---

2) `MigrationNodeFactory` auf `TMigration` umstellen

Annahme: Deine typed-AST-Klassen liegen in `ch.example.ilimap.semantics.tast.*`

Java

```java
package ch.example.ilimap.truffle;

import ch.example.ilimap.runtime.Path;
import ch.example.ilimap.semantics.tast.*;
import ch.example.ilimap.truffle.nodes.ExprNode;
import ch.example.ilimap.truffle.nodes.StmtNode;
import ch.example.ilimap.truffle.nodes.expr.*;
import ch.example.ilimap.truffle.nodes.stmt.*;
import com.oracle.truffle.api.source.Source;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public final class MigrationNodeFactory {

    private MigrationNodeFactory() {}

    public static MigrationRootNode build(IliMapLanguage language, TMigration migration,
        List<ClassRuleNode> classRules = new ArrayList<>();

        for (TClassRule cr : migration.classRules()) {
            classRules.add(buildClassRule(cr));
```

```
        }

        return new MigrationRootNode(language, new ProgramNode(classRules));
    }

    private static ClassRuleNode buildClassRule(TClassRule cr) {
        ExprNode when = cr.whenExpr() == null ? null : buildExpr(cr.whenExpr());

        StmtNode[] stmts = cr.statements().stream()
                .map(MigrationNodeFactory::buildStmt)
                .toArray(StmtNode[]::new);

        return new ClassRuleNode(
                cr.sourceClass(),
                cr.targetClass(),
                Path.of(cr.keyMapping().sourceKeyPath().path()),
                Path.of(cr.keyMapping().targetKeyPath().path()),
                when,
                stmts
        );
    }

    private static StmtNode buildStmt(TStmt stmt) {
        return switch (stmt) {
            case TSetStmt s -> {
                ExprNode expr = buildExpr(s.expr());
                ExprNode coerced = applyCoercion(expr, s.coercion());
                yield new SetStmtNode(Path.of(s.targetPath().path()), coerced);
            }

            case TLetStmt s -> new LetStmtNode(s.varName(), buildExpr(s.expr()));

            case TAssertStmt s -> new AssertStmtNode(
                    buildExpr(s.condition()),
                    s.severity() == TAssertStmt.Severity.ERROR
```

```
                        ? AssertStmtNode.Severity.ERROR
                        : AssertStmtNode.Severity.WARN,
                s.message()
        );

        case TRefStmt s -> new RefStmtNode(
                Path.of(s.targetRefPath().path()),
                s.resolutionBinding().sourceRefClass(),
                s.resolutionBinding().targetRefClass(),
                buildExpr(s.sourceRefExpr())
        );
    };
}

private static ExprNode buildExpr(TExpr expr) {
    return switch (expr) {
        case TLiteralExpr l -> new LiteralNode(l.value());

        case TPathExpr p -> buildPathExpr(p.path());

        case TVarExpr v -> new VarReadNode(v.name());

        case TBinaryExpr b -> new BinaryOpNode(
                buildExpr(b.left()),
                b.op(),
                buildExpr(b.right())
        );

        case TFunctionCallExpr f -> new FunctionCallNode(
                f.functionName(),
                f.args().stream().map(MigrationNodeFactory::buildExpr).toArray(ExprNo
        );

        case TMapEnumExpr m -> {
            Map<String, String> mm = new HashMap<>(m.mappings());
```

```
                yield new MapEnumExprNode(
                        buildExpr(m.sourceExpr()),
                        mm,
                        m.defaultValue(),
                        m.defaultWarnMessage()
                );
            }
        };
    }

    private static ExprNode buildPathExpr(BoundPath path) {
        return switch (path.scope()) {
            case SOURCE -> new SourcePathReadNode(Path.of(path.path()));
            case TARGET -> new TargetPathReadNode(Path.of(path.path()));
            case LOCAL  -> new VarReadNode(path.path()); // Zukunftssicherheit; aktuell 1
        };
    }

    private static ExprNode applyCoercion(ExprNode expr, Coercion coercion) {
        if (coercion == null || coercion instanceof NoCoercion) {
            return expr;
        }
        if (coercion instanceof PrimitiveCoercion pc) {
            return new CoerceExprNode(expr, pc.strategy());
        }
        throw new IllegalStateException("Unsupported coercion: " + coercion);
    }
}
```

---

3) Neuer `CoerceExprNode` für typed Coercions

Das typed AST trägt `Coercion` bereits explizit.

Jetzt braucht die Runtime ein Node, das diese Konversion auch ausführt.

`CoerceExprNode`

Java

```java
package ch.example.ilimap.truffle.nodes.expr;

import ch.example.ilimap.runtime.EvalContext;
import ch.example.ilimap.truffle.nodes.ExprNode;
import com.oracle.truffle.api.frame.VirtualFrame;

public final class CoerceExprNode extends ExprNode {
    @Child private ExprNode inner;
    private final String strategy;

    public CoerceExprNode(ExprNode inner, String strategy) {
        this.inner = inner;
        this.strategy = strategy;
    }

    @Override
    public Object executeGeneric(VirtualFrame frame, EvalContext ctx) {
        Object v = inner.executeGeneric(frame, ctx);
        if (v == null) return null;

        return switch (strategy) {
            case "numberToString" -> String.valueOf(v);
            case "enumToString" -> String.valueOf(v);
            case "refTidToString" -> String.valueOf(v);
            default -> throw new IllegalStateException("Unknown coercion strategy: " + st
        };
```

```
    }
}
```

Warum eigener Node statt „heimlich im SetStmtNode“?

Weil es exakt zur typed-AST-Idee passt:

- Binder entscheidet **was** konvertiert wird
- Runtime führt nur aus

---

4) Patch für `IliMapLanguage.parse(...)`

Jetzt soll `parse(...)` nicht mehr direkt Parser-AST in Nodes bauen, sondern:

```
DSL -> Parser-AST -> SemanticBinder -> TMigration -> Nodes
```

Aktualisierte `IliMapLanguage`

Java

```java
package ch.example.ilimap.truffle;

import ch.example.ilimap.parser.MappingParserFacade;
import ch.example.ilimap.parser.ast.MigrationAst;
import ch.example.ilimap.runtime.MigrationContext;
import ch.example.ilimap.semantics.FunctionCatalog;
import ch.example.ilimap.semantics.SemanticBinder;
import ch.example.ilimap.semantics.tast.TMigration;
import com.oracle.truffle.api.CallTarget;
import com.oracle.truffle.api.TruffleLanguage;
import com.oracle.truffle.api.source.Source;
```

```
@TruffleLanguage.Registration(
        id = IliMapLanguage.ID,
        name = "INTERLIS Mapping DSL",
        version = "0.2.0",
        characterMimeTypes = IliMapLanguage.MIME_TYPE
)
public final class IliMapLanguage extends TruffleLanguage<MigrationContext> {
    public static final String ID = "ilimap";
    public static final String MIME_TYPE = "application/x-ilimap";

    @Override
    protected MigrationContext createContext(Env env) {
        return MigrationContextHolder.require();
    }

    @Override
    protected CallTarget parse(ParsingRequest request) throws Exception {
        Source source = request.getSource();
        String code = source.getCharacters().toString();

        // 1) Parser-AST
        MigrationAst parserAst = MappingParserFacade.parse(code);

        // 2) Context (für Schema A/B) holen
        MigrationContext ctx = MigrationContextHolder.peek();
        if (ctx == null) {
            throw new IllegalStateException("MigrationContext not set");
        }

        // 3) Typed Binding
        SemanticBinder binder = new SemanticBinder(
                ctx.schemaA(),
                ctx.schemaB(),
                FunctionCatalog.defaultCatalog()
        );
```

```
        SemanticBinder.BindResult bind = binder.bind(parserAst);

        bind.diagnostics().warnings().forEach(w -> ctx.report().warn("[BIND] " + w.code()

        if (!bind.isOk()) {
            String msg = bind.diagnostics().errors().stream()
                    .map(d -> d.code() + ": " + d.message())
                    .reduce((a, b) -> a + "; " + b)
                    .orElse("Unknown semantic binding error");
            throw new IllegalArgumentException("Semantic binding failed: " + msg);
        }

        TMigration typedAst = bind.migration();

        // 4) Typed AST -> Truffle Nodes
        MigrationRootNode root = MigrationNodeFactory.build(this, typedAst, source);
        return root.getCallTarget();
    }
}
```

---

5) `MigrationNodeFactory` -Pfad testen (typed AST -> Nodes)

Du hast schon Binder-Tests. Jetzt lohnt sich ein kleiner Test für die Factory mit typed AST, damit der Umbau stabil ist.

Beispieltest: `TRefStmt` verwendet gebundenes `targetRefClass`

‹/› Java

```
package ch.example.ilimap.truffle;
```

```
import ch.example.ilimap.semantics.tast.*;
import org.junit.jupiter.api.Test;

import java.util.List;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;

class MigrationNodeFactoryTypedAstTest {

    @Test
    void buildsNodesFromTypedAstWithoutRefInference() {
        TType str = new TPrimitiveType(TPrimitiveType.Kind.STRING, false);

        TClassRule ownerRule = new TClassRule(
                "A.Eigentuemer",
                "B.Owner",
                new TKeyMapping(
                        new BoundPath(BoundPath.ScopeKind.SOURCE, "A.Eigentuemer", "tid",
                        new BoundPath(BoundPath.ScopeKind.TARGET, "B.Owner", "tid", str)
                ),
                null,
                List.of(),
                new RuleBinding("A.Eigentuemer->B.Owner")
        );

        TClassRule buildingRule = new TClassRule(
                "A.Gebaeude",
                "B.Building",
                new TKeyMapping(
                        new BoundPath(BoundPath.ScopeKind.SOURCE, "A.Gebaeude", "tid", st
                        new BoundPath(BoundPath.ScopeKind.TARGET, "B.Building", "tid", st
                ),
                null,
                List.of(
```

```
                        new TRefStmt(
                                new BoundPath(
                                        BoundPath.ScopeKind.TARGET,
                                        "B.Building",
                                        "owner_ref",
                                        new TReferenceType("B.Owner", true)
                                ),
                                new TPathExpr(new BoundPath(
                                        BoundPath.ScopeKind.SOURCE,
                                        "A.Gebaeude",
                                        "eigentuemer_ref",
                                        new TReferenceType("A.Eigentuemer", true)
                                )),
                                new RefResolutionBinding(
                                        "A.Eigentuemer",
                                        "B.Owner",
                                        "A.Eigentuemer->B.Owner",
                                        RefResolutionBinding.MissingRefPolicy.WARN
                                )
                        )
                ),
                new RuleBinding("A.Gebaeude->B.Building")
        );

        TMigration tm = new TMigration("DM_A", "DM_B", List.of(buildingRule, ownerRule));

        // Build darf ohne inferTargetRefClass-Heuristik funktionieren
        assertDoesNotThrow(() -> MigrationNodeFactory.build(new IliMapLanguage(), tm, nul
    }
}
```

Hinweis: Falls dein `MigrationRootNode` /Factory `Source` strikt braucht, gib im Test ein minimales `Source` -Objekt mit.

6) Kleine, wichtige Folgepatches (damit alles sauber zusammenpasst)

A) `SemanticBinder` im CLI/Tests verwenden

Wenn du irgendwo noch direkt `MigrationNodeFactory.build(..., MigrationAst, ...)` aufrufst: ersetzen durch Binder-Schritt.

Vorher

```java
var ast = MappingParserFacade.parse(dsl);
var root = MigrationNodeFactory.build(new IliMapLanguage(), ast, truffleSource);
```

Nachher

```java
var parserAst = MappingParserFacade.parse(dsl);

var binder = new SemanticBinder(schemaA, schemaB, FunctionCatalog.defaultCatalog());
var bind = binder.bind(parserAst);
if (!bind.isOk()) {
    throw new IllegalStateException("Bind errors: " + bind.diagnostics().errors());
}

var root = MigrationNodeFactory.build(new IliMapLanguage(), bind.migration(), truffleSour
```

B) `SetStmtNode` bleibt unverändert

Weil die Coercion jetzt als `CoerceExprNode` im Expr-Baum sitzt, musst du `SetStmtNode` nicht aufblasen. Das ist gut.

C) `RefStmtNode` profitiert sofort

`RefStmtNode` bekommt bereits `targetRefClass` aus `TRefStmt.resolutionBinding()`.
Die frühere Heuristik ist damit vollständig obsolet.

---

7) Was du damit jetzt gewonnen hast

Mit diesem Umbau hast du den kompletten Pfad:

- **Parser-AST** (Syntax)
- **Typed AST** (Semantik + Typen + Bindings)
- **Truffle Nodes** (Ausführung)

und damit:

- kein `inferTargetRefClass(...)`
- `MAP_ENUM` schon vor Runtime validiert
- Coercions explizit
- Truffle-Factory deutlich simpler und stabiler

Das ist ein sehr sauberer Architekturpunkt.

---

8) Nächster sinnvoller Schritt (wirklich lohnend)

Der nächste große Qualitätsschritt wäre jetzt:

`SourceSpan` durchziehen

- Parser-AST trägt `SourceSpan`
- Binder kopiert Spans in typed AST
- `MigrationNodeFactory` setzt `SourceSection` auf Nodes
- Report/Diagnostics zeigen Zeile/Spalte der DSL-Regel

Dann bekommst du:

- verständliche Fehlermeldungen
- Debugging/Tracing mit DSL-Lokation
- später Truffle-Instrumentierung mit echter Quellzuordnung

Wenn du willst, mache ich dir im nächsten Schritt genau diesen **SourceSpan/SourceSection-Durchstich** (Parser → typed AST → Truffle-Nodes → Report).

JA. gerne.

Thought for 1m 24s

Ja — **sehr guter Schritt**. Das ist genau der Punkt, an dem dein Tool von „funktioniert irgendwie“ zu **wirklich gut debuggbar** wird.

Ich zeige dir den **SourceSpan/SourceSection-Durchstich** sauber entlang der ganzen Pipeline:

- **Parser (ANTLR) → Parser-AST mit `SourceSpan`**
- **Semantic Binder → Typed AST mit `SourceSpan`**
- **Truffle Node Factory → `SourceSection` auf Nodes**
- **Runtime/Report → Fehler/Warnungen mit DSL-Zeile/Spalte**

Zielbild

Am Ende willst du sowas sehen:

- **Bind-Fehler (statisch):**
  - `ILIMAP-BIND-REF-TARGET-MISMATCH at migration.ilimap:23:17`
- **Runtime-Fehler/Warnung (z. B. ASSERT / MAP_ENUM DEFAULT):**
  - `Fläche darf nicht negativ sein @ A.Gebaeude:G1 [migration.ilimap:18:9]`

Das ist machbar mit einer durchgezogenen Source-Metadaten-Schicht.

---

1) Parser-AST mit `SourceSpan`

1.1 `SourceSpan` einführen (parser-seitig)

Ich würde **char offsets + line/column** speichern. Dann bist du flexibel.

```java
package ch.example.ilimap.parser.ast;

public record SourceSpan(
        String fileName,
        int startLine,     // 1-based
```

```
        int startColumn,   // 1-based
        int endLine,       // 1-based
        int endColumn,     // 1-based
        int charStart,     // 0-based inclusive
        int charEnd        // 0-based exclusive
) {
    public int length() {
        return Math.max(0, charEnd - charStart);
    }

    @Override
    public String toString() {
        return fileName + ":" + startLine + ":" + startColumn;
    }
}
```

1.2 Parser-AST-Interfaces auf „spanned“ umstellen

Am besten bekommen **Statements und Expressions** immer `span()`.

‹/› Java

```
package ch.example.ilimap.parser.ast;

public interface PNode {
    SourceSpan span();
}
```

Statements

Java

```java
package ch.example.ilimap.parser.ast;

public sealed interface StmtAst extends PNode permits SetStmtAst, LetStmtAst, AssertStmtA
```

Java

```java
package ch.example.ilimap.parser.ast;

public record SetStmtAst(String targetPath, ExprAst expr, SourceSpan span) implements Str
```

Java

```java
package ch.example.ilimap.parser.ast;

public record LetStmtAst(String varName, ExprAst expr, SourceSpan span) implements StmtAs
```

Java

```java
package ch.example.ilimap.parser.ast;

public record AssertStmtAst(ExprAst condition, SeverityAst severity, String message, Sour
    public enum SeverityAst { WARN, ERROR }
}
```

Java

```java
package ch.example.ilimap.parser.ast;

public record RefStmtAst(
```

```
        String targetRefPath,
        String sourceRefClass,
        String sourceRefExprPath,
        SourceSpan span
) implements StmtAst {}
```

Expressions

Java

```
package ch.example.ilimap.parser.ast;

public sealed interface ExprAst extends PNode permits
        LiteralAst, SourcePathAst, TargetPathAst, VarAst, BinaryExprAst, FunctionCallAst,
```

Java

```
package ch.example.ilimap.parser.ast;

public record LiteralAst(Object value, SourceSpan span) implements ExprAst {}
```

Java

```
package ch.example.ilimap.parser.ast;

public record SourcePathAst(String path, SourceSpan span) implements ExprAst {}
```

Java

```
package ch.example.ilimap.parser.ast;
```

```
public record TargetPathAst(String path, SourceSpan span) implements ExprAst {}
```

Java

```
package ch.example.ilimap.parser.ast;

public record VarAst(String name, SourceSpan span) implements ExprAst {}
```

Java

```
package ch.example.ilimap.parser.ast;

public record BinaryExprAst(ExprAst left, String op, ExprAst right, SourceSpan span) impl
```

Java

```
package ch.example.ilimap.parser.ast;

import java.util.List;

public record FunctionCallAst(String name, List<ExprAst> args, SourceSpan span) implement
```

Java

```
package ch.example.ilimap.parser.ast;

import java.util.List;

public record MapEnumExprAst(
        ExprAst sourceExpr,
        List<Entry> entries,
        String defaultValue,
```

```
        String defaultWarnMessage,
        SourceSpan span
) implements ExprAst {
    public record Entry(String sourceValue, String targetValue) {}
}
```

Class/Migration-AST ebenfalls spannen (sehr hilfreich)

‹/› Java

```java
package ch.example.ilimap.parser.ast;

import java.util.List;

public record MigrationAst(
        String sourceModel,
        String targetModel,
        List<ClassRuleAst> classRules,
        SourceSpan span
) implements PNode {}
```

‹/› Java

```java
package ch.example.ilimap.parser.ast;

import java.util.List;

public record ClassRuleAst(
        String sourceClass,
        String targetClass,
        KeyMappingAst keyMapping,
        ExprAst whenExpr,
        List<StmtAst> statements,
```

```
        SourceSpan span
) implements PNode {}
```

Java

```
package ch.example.ilimap.parser.ast;

public record KeyMappingAst(String sourcePath, String targetPath, SourceSpan span) implem
```

---

2) ANTLR-Visitor: `SourceSpan` erzeugen

Das ist die wichtigste Stelle für den Durchstich.

2.1 Helper in `AstBuilderVisitor`

Java

```
package ch.example.ilimap.parser;

import ch.example.ilimap.parser.ast.SourceSpan;
import org.antlr.v4.runtime.ParserRuleContext;
import org.antlr.v4.runtime.Token;

final class SpanUtil {
    private SpanUtil() {}

    static SourceSpan span(ParserRuleContext ctx) {
        return span(ctx.getStart(), ctx.getStop());
    }

    static SourceSpan span(Token start, Token stop) {
```

```
        if (start == null) return null;

        String file = start.getTokenSource() != null
                && start.getTokenSource().getSourceName() != null
                ? start.getTokenSource().getSourceName()
                : "<input>";

        int startLine = start.getLine();
        int startCol = start.getCharPositionInLine() + 1;

        Token endTok = stop != null ? stop : start;
        String endText = endTok.getText() == null ? "" : endTok.getText();

        int endLine = endTok.getLine();
        int endCol = endTok.getCharPositionInLine() + Math.max(1, endText.length());

        int charStart = start.getStartIndex();
        int charEnd = endTok.getStopIndex() >= 0 ? endTok.getStopIndex() + 1 : charStart;

        return new SourceSpan(file, startLine, startCol, endLine, endCol, charStart, char
    }
}
```

`Token#getCharPositionInLine()` ist 0-basiert, darum `+1`.

2.2 Visitor patchen (Beispiele)

Migration

‹/› Java

```
@Override
public MigrationAst visitMigration(InterlisMapParser.MigrationContext ctx) {
    String sourceModel = unquote(ctx.STRING(0).getText());
    String targetModel = unquote(ctx.STRING(1).getText());

    List<ClassRuleAst> classRules = new ArrayList<>();
    for (var body : ctx.migrationBody()) {
        if (body.classRule() != null) {
            classRules.add((ClassRuleAst) visit(body.classRule()));
        }
    }

    return new MigrationAst(sourceModel, targetModel, classRules, SpanUtil.span(ctx));
}
```

Class rule

Java

```
@Override
public ClassRuleAst visitClassRule(InterlisMapParser.ClassRuleContext ctx) {
    String sourceClass = ctx.qname(0).getText();
    String targetClass = ctx.qname(1).getText();

    KeyMappingAst key = (KeyMappingAst) visit(ctx.keyClause());
    ExprAst whenExpr = ctx.whenClause() != null ? (ExprAst) visit(ctx.whenClause()) : nul

    List<StmtAst> statements = new ArrayList<>();
    for (var s : ctx.classStmt()) {
        if (s.setStmt() != null) statements.add((StmtAst) visit(s.setStmt()));
        else if (s.letStmt() != null) statements.add((StmtAst) visit(s.letStmt()));
        else if (s.assertStmt() != null) statements.add((StmtAst) visit(s.assertStmt()));
        else if (s.refStmt() != null) statements.add((StmtAst) visit(s.refStmt()));
    }
```

```
    return new ClassRuleAst(sourceClass, targetClass, key, whenExpr, statements, SpanUtil
}
```

Statements

Java

```java
@Override
public StmtAst visitSetStmt(InterlisMapParser.SetStmtContext ctx) {
    return new SetStmtAst(
            ctx.path().getText(),
            (ExprAst) visit(ctx.expr()),
            SpanUtil.span(ctx)
    );
}

@Override
public StmtAst visitRefStmt(InterlisMapParser.RefStmtContext ctx) {
    return new RefStmtAst(
            ctx.path().getText(),
            ctx.qname().getText(),
            "source." + ctx.sourceRef().path().getText(),
            SpanUtil.span(ctx)
    );
}
```

Expressions

Java

```java
@Override
public ExprAst visitSourcePathExpr(InterlisMapParser.SourcePathExprContext ctx) {
```

```
    return new SourcePathAst(ctx.path().getText(), SpanUtil.span(ctx));
}

@Override
public ExprAst visitNumberExpr(InterlisMapParser.NumberExprContext ctx) {
    String n = ctx.NUMBER().getText();
    Object val = n.contains(".") ? Double.parseDouble(n) : Long.parseLong(n);
    return new LiteralAst(val, SpanUtil.span(ctx));
}

@Override
public ExprAst visitCmpExpr(InterlisMapParser.CmpExprContext ctx) {
    return new BinaryExprAst(
            (ExprAst) visit(ctx.expr(0)),
            ctx.getChild(1).getText(),
            (ExprAst) visit(ctx.expr(1)),
            SpanUtil.span(ctx)
    );
}

@Override
public ExprAst visitFuncCall(InterlisMapParser.FuncCallContext ctx) {
    List<ExprAst> args = new ArrayList<>();
    for (var e : ctx.expr()) args.add((ExprAst) visit(e));
    return new FunctionCallAst(ctx.IDENT().getText(), args, SpanUtil.span(ctx));
}
```

---

3) Diagnostics auf `SourceSpan` umstellen

Du hast bereits `DiagnosticBag`. Jetzt nur noch `SourceSpan` aus Parser/Binder weiterreichen.

`Diagnostic` + `DiagnosticBag`

Java

```java
package ch.example.ilimap.semantics;

import ch.example.ilimap.parser.ast.SourceSpan;

public record Diagnostic(
        Severity severity,
        String code,
        String message,
        SourceSpan span
) {
    public enum Severity { ERROR, WARN }

    public String pretty() {
        if (span == null) return code + ": " + message;
        return code + " @ " + span + " - " + message;
    }
}
```

Java

```java
package ch.example.ilimap.semantics;

import ch.example.ilimap.parser.ast.SourceSpan;

import java.util.ArrayList;
import java.util.List;

public final class DiagnosticBag {
    private final List<Diagnostic> errors = new ArrayList<>();
    private final List<Diagnostic> warnings = new ArrayList<>();

    public void error(String code, String message, SourceSpan span) {
```

```
        errors.add(new Diagnostic(Diagnostic.Severity.ERROR, code, message, span));
    }

    public void warn(String code, String message, SourceSpan span) {
        warnings.add(new Diagnostic(Diagnostic.Severity.WARN, code, message, span));
    }

    public List<Diagnostic> errors() { return List.copyOf(errors); }
    public List<Diagnostic> warnings() { return List.copyOf(warnings); }
}
```

---

4) Typed AST ebenfalls mit `SourceSpan`

Das typed AST soll Parser-Spans **mittragen**, damit die Truffle-Factory später `SourceSection` setzen kann.

4.1 `TNode` Basis

Java

```
package ch.example.ilimap.semantics.tast;

import ch.example.ilimap.parser.ast.SourceSpan;

public interface TNode {
    SourceSpan span();
}
```

Typed Statements / Expressions

Java

```java
package ch.example.ilimap.semantics.tast;

public sealed interface TStmt extends TNode permits TSetStmt, TLetStmt, TAssertStmt, TRet
```

Java

```java
package ch.example.ilimap.semantics.tast;

public sealed interface TExpr extends TNode permits
        TLiteralExpr, TPathExpr, TVarExpr, TBinaryExpr, TFunctionCallExpr, TMapEnumExpr {
    TType type();
}
```

Beispiele mit `span`

Java

```java
package ch.example.ilimap.semantics.tast;

import ch.example.ilimap.parser.ast.SourceSpan;

public record TSetStmt(
        BoundPath targetPath,
        TExpr expr,
        Coercion coercion,
        SourceSpan span
) implements TStmt {}
```

Java

```
package ch.example.ilimap.semantics.tast;

import ch.example.ilimap.parser.ast.SourceSpan;

public record TRefStmt(
        BoundPath targetRefPath,
        TExpr sourceRefExpr,
        RefResolutionBinding resolutionBinding,
        SourceSpan span
) implements TStmt {}
```

‹/› Java

```
package ch.example.ilimap.semantics.tast;

import ch.example.ilimap.parser.ast.SourceSpan;

public record TMapEnumExpr(
        TExpr sourceExpr,
        TEnumType sourceEnumType,
        TType resultType,
        java.util.Map<String, String> mappings,
        String defaultValue,
        String defaultWarnMessage,
        EnumCoverage coverage,
        SourceSpan span
) implements TExpr {
    @Override public TType type() { return resultType; }
}
```

‹/› Java

```
package ch.example.ilimap.semantics.tast;
```

```
import ch.example.ilimap.parser.ast.SourceSpan;

public record TBinaryExpr(
        TExpr left,
        String op,
        TExpr right,
        TType type,
        SourceSpan span
) implements TExpr {}
```

(Analog für `TLiteralExpr`, `TPathExpr`, `TVarExpr`, `TFunctionCallExpr`, `TAssertStmt`, `TLetStmt`, `TClassRule`, `TMigration`, `TKeyMapping`)

---

5) `TypedAstBuilder` patchen: Spans durchreichen

Jetzt kommt der eigentliche „Durchstich“ im Binder.

5.1 Diagnostics mit Parser-Span auslösen

Beispiel `bindRefStmt(...)`

Wichtig: statt `null` immer `s.span()` (oder Child-Spans) benutzen.

Java

```
private TStmt bindRefStmt(RefStmtAst s, RuleBindContext ctx) {
    BoundPath targetRefPath = bindTargetPath(s.targetRefPath(), ctx, s.span());
    if (targetRefPath == null) return null;

    if (!(targetRefPath.type() instanceof TReferenceType targetRefType)) {
        diags.error(
                "ILIMAP-BIND-REF-TARGET-NOTREF",
```

```
                "REF-Zielpfad ist keine Referenz: target." + s.targetRefPath(),
                s.span()
        );
        return null;
    }

    String sourceExprPath = s.sourceRefExprPath();
    if (!sourceExprPath.startsWith("source.")) {
        diags.error(
                "ILIMAP-BIND-REF-SOURCE-SCOPE",
                "REF BY erwartet source.<pfad>, gefunden: " + sourceExprPath,
                s.span()
        );
        return null;
    }

    String sourcePathOnly = sourceExprPath.substring("source.".length());
    TExpr sourceRefExpr = bindExpr(new SourcePathAst(sourcePathOnly, s.span()), ctx, null
    if (sourceRefExpr == null) return null;

    String sourceRefClass = s.sourceRefClass();

    ClassMappingRegistry.ClassMapping refClassMapping =
            classMappings.requireBySourceClass(sourceRefClass, diags, s.span());
    if (refClassMapping == null) return null;

    String resolvedTargetRefClass = refClassMapping.targetClass();

    if (!java.util.Objects.equals(targetRefType.targetClassQName(), resolvedTargetRefClas
        diags.error(
                "ILIMAP-BIND-REF-TARGET-MISMATCH",
                "REF-Zielpfad erwartet " + targetRefType.targetClassQName()
                        + ", aber Klassenmapping für " + sourceRefClass
                        + " liefert " + resolvedTargetRefClass,
                s.span()
```

```
        );
        return null;
    }

    RefResolutionBinding binding = new RefResolutionBinding(
            sourceRefClass,
            resolvedTargetRefClass,
            refClassMapping.mappingId(),
            RefResolutionBinding.MissingRefPolicy.WARN
    );

    return new TRefStmt(targetRefPath, sourceRefExpr, binding, s.span());
}
```

5.2 `MAP_ENUM` Binding mit Span

Java

```
private TExpr bindMapEnumExpr(MapEnumExprAst m, RuleBindContext ctx, TType expectedType)
    TExpr sourceExpr = bindExpr(m.sourceExpr(), ctx, null);
    if (sourceExpr == null) return null;

    TEnumType sourceEnumType = null;
    if (sourceExpr.type() instanceof TEnumType et) {
        sourceEnumType = et;
    } else if (sourceExpr.type() instanceof TPrimitiveType p && p.kind() == TPrimitiveTyp
        diags.warn(
                "ILIMAP-BIND-MAPENUM-SOURCE-STRING",
                "MAP_ENUM-Quelle ist STRING statt ENUM; statische Abdeckungsprüfung einge
                m.span()
        );
    } else {
```

```
        diags.error(
                "ILIMAP-BIND-MAPENUM-SOURCE-TYPE",
                "MAP_ENUM erwartet ENUM/STRING als Quelle, gefunden: " + renderType(sourc
                m.span()
        );
        return null;
    }

    TType resultType = expectedType != null
            ? expectedType
            : new TPrimitiveType(TPrimitiveType.Kind.STRING, true);

    java.util.Map<String, String> mappings = new java.util.LinkedHashMap<>();
    for (MapEnumExprAst.Entry e : m.entries()) {
        String prev = mappings.put(e.sourceValue(), e.targetValue());
        if (prev != null) {
            diags.error(
                    "ILIMAP-BIND-MAPENUM-DUPSRC",
                    "MAP_ENUM enthält doppelten Quellwert: " + e.sourceValue(),
                    m.span()
            );
        }
    }

    java.util.Set<String> mappedSourceValues = new java.util.LinkedHashSet<>(mappings.key
    java.util.Set<String> unmappedKnownSourceValues = new java.util.LinkedHashSet<>();

    if (sourceEnumType != null) {
        java.util.Set<String> allowed = sourceEnumType.allowedValues();

        for (String srcVal : mappedSourceValues) {
            if (!allowed.contains(srcVal)) {
                diags.warn(
                        "ILIMAP-BIND-MAPENUM-SRC-UNKNOWN",
                        "MAP_ENUM mappt Quellwert '" + srcVal + "', der nicht in der Quel
```

```
                    m.span()
                );
            }
        }

        unmappedKnownSourceValues.addAll(allowed);
        unmappedKnownSourceValues.removeAll(mappedSourceValues);

        if (!unmappedKnownSourceValues.isEmpty() && m.defaultValue() == null) {
            diags.warn(
                    "ILIMAP-BIND-MAPENUM-INCOMPLETE",
                    "MAP_ENUM deckt nicht alle bekannten Quellwerte ab und hat kein DEFAU
                            + unmappedKnownSourceValues,
                    m.span()
            );
        }
    }

    if (resultType instanceof TEnumType targetEnumType) {
        for (String targetVal : mappings.values()) {
            if (!targetEnumType.allowedValues().contains(targetVal)) {
                diags.error(
                        "ILIMAP-BIND-MAPENUM-TGT-INVALID",
                        "MAP_ENUM Zielwert '" + targetVal + "' ist nicht erlaubt in Ziel-
                        m.span()
                );
            }
        }
    }

    EnumCoverage coverage = new EnumCoverage(mappedSourceValues, unmappedKnownSourceValue

    return new TMapEnumExpr(
            sourceExpr,
            sourceEnumType,
```

```
            resultType,
            java.util.Map.copyOf(mappings),
            m.defaultValue(),
            m.defaultWarnMessage(),
            coverage,
            m.span()
    );
}
```

5.3 Path-Binding mit Span für bessere Fehler

Java

```
private BoundPath bindTargetPath(String path, RuleBindContext ctx, SourceSpan span) {
    return bindPath(BoundPath.ScopeKind.TARGET, ctx.targetClass, path, span);
}

private BoundPath bindSourcePath(String path, RuleBindContext ctx, SourceSpan span) {
    return bindPath(BoundPath.ScopeKind.SOURCE, ctx.sourceClass, path, span);
}

private BoundPath bindPath(BoundPath.ScopeKind scope, ClassSchema ownerClass, String path
    if ("tid".equals(path)) {
        return new BoundPath(scope, ownerClass.qName(), path, new TPrimitiveType(TPrimiti
    }

    AttributeSchema attr = ownerClass.findAttribute(path).orElse(null);
    if (attr == null) {
        diags.error(
                "ILIMAP-BIND-PATH-404",
                "Unbekannter Pfad " + scope.name().toLowerCase() + "." + path + " in " +
                span
```

```
        );
        return null;
    }

    return new BoundPath(scope, ownerClass.qName(), path, valueTypeToTType(attr));
}
```

Dann in `bindSetStmt`, `bindRefStmt`, `bindExpr(SourcePathAst/TargetPathAst)` immer `expr.span()` bzw. `stmt.span()` weitergeben.

---

6) Truffle-Nodes mit `SourceSection`

Jetzt machen wir die Runtime-Nodes „source-aware“.

6.1 Gemeinsame Basisnode mit `SourceSection`

Java

```java
package ch.example.ilimap.truffle.nodes;

import com.oracle.truffle.api.nodes.Node;
import com.oracle.truffle.api.source.SourceSection;

public abstract class IliMapNode extends Node {
    private SourceSection sourceSection;

    public final void setSourceSection(SourceSection sourceSection) {
        this.sourceSection = sourceSection;
    }

    @Override
    public SourceSection getSourceSection() {
```

```
        return sourceSection;
    }
}
```

`ExprNode` / `StmtNode` darauf umstellen

Java

```java
package ch.example.ilimap.truffle.nodes;

import ch.example.ilimap.runtime.EvalContext;
import com.oracle.truffle.api.frame.VirtualFrame;

public abstract class ExprNode extends IliMapNode {
    public abstract Object executeGeneric(VirtualFrame frame, EvalContext ctx);

    public boolean executeBoolean(VirtualFrame frame, EvalContext ctx) {
        Object v = executeGeneric(frame, ctx);
        if (v instanceof Boolean b) return b;
        throw new IllegalStateException("Expected boolean, got: " + v);
    }
}
```

Java

```java
package ch.example.ilimap.truffle.nodes;

import ch.example.ilimap.runtime.EvalContext;
import com.oracle.truffle.api.frame.VirtualFrame;

public abstract class StmtNode extends IliMapNode {
    public abstract void executeVoid(VirtualFrame frame, EvalContext ctx);
}
```

`ClassRuleNode` und `ProgramNode` können ebenfalls `IliMapNode` erben (optional, aber sinnvoll für spätere Instrumentierung).

6.2 `MigrationNodeFactory: SourceSpan -> SourceSection`

Jetzt wird die typed-AST-Span in echte Truffle- `SourceSection` übersetzt.

Helper in der Factory

Java

```java
package ch.example.ilimap.truffle;

import ch.example.ilimap.parser.ast.SourceSpan;
import ch.example.ilimap.truffle.nodes.IliMapNode;
import com.oracle.truffle.api.source.Source;
import com.oracle.truffle.api.source.SourceSection;

final class NodeSourceSections {
    private NodeSourceSections() {}

    static <T extends IliMapNode> T attach(T node, Source source, SourceSpan span) {
        if (node == null || source == null || span == null) return node;

        SourceSection ss = toSection(source, span);
        if (ss != null) node.setSourceSection(ss);
        return node;
    }

    static SourceSection toSection(Source source, SourceSpan span) {
        try {
            int len = Math.max(1, span.length());
```

```
            // Diese Signatur ist in aktuellen Truffle-Versionen üblich.
            return source.createSection(span.startLine(), span.startColumn(), len);
        } catch (Throwable ignored) {
            // Fallback für API-Differenzen / inkonsistente Span-Werte
            try {
                int start = Math.max(0, span.charStart());
                int len = Math.max(1, span.length());
                return source.createSection(start, len);
            } catch (Throwable ignored2) {
                return null;
            }
        }
    }
}
```

6.3 `MigrationNodeFactory` patchen (typed AST + SourceSection)

Wichtig: `Source` an rekursive `build...` -Methoden durchreichen.

Java

```
public static MigrationRootNode build(IliMapLanguage language, TMigration migration, Sour
    List<ClassRuleNode> classRules = new ArrayList<>();

    for (TClassRule cr : migration.classRules()) {
        classRules.add(buildClassRule(cr, source));
    }

    MigrationRootNode root = new MigrationRootNode(language, new ProgramNode(classRules))
    // RootNode hat i.d.R. eigene SourceSection-Mechanik; optional:
    // (wenn MigrationRootNode von eigener Base ableitet, sonst weglassen)
    return root;
}
```

```
private static ClassRuleNode buildClassRule(TClassRule cr, Source source) {
    ExprNode when = cr.whenExpr() == null ? null : buildExpr(cr.whenExpr(), source);

    StmtNode[] stmts = cr.statements().stream()
            .map(s -> buildStmt(s, source))
            .toArray(StmtNode[]::new);

    ClassRuleNode node = new ClassRuleNode(
            cr.sourceClass(),
            cr.targetClass(),
            Path.of(cr.keyMapping().sourceKeyPath().path()),
            Path.of(cr.keyMapping().targetKeyPath().path()),
            when,
            stmts
    );
    // Falls ClassRuleNode von IliMapNode erbt:
    // NodeSourceSections.attach(node, source, cr.span());
    return node;
}
```

`buildStmt(...)` mit spans

Java

```
private static StmtNode buildStmt(TStmt stmt, Source source) {
    StmtNode node = switch (stmt) {
        case TSetStmt s -> {
            ExprNode expr = buildExpr(s.expr(), source);
            ExprNode coerced = applyCoercion(expr, s.coercion(), source, s.span());
            yield new SetStmtNode(Path.of(s.targetPath().path()), coerced);
        }

        case TLetStmt s -> new LetStmtNode(s.varName(), buildExpr(s.expr(), source));
```

```
        case TAssertStmt s -> new AssertStmtNode(
                buildExpr(s.condition(), source),
                s.severity() == TAssertStmt.Severity.ERROR
                        ? AssertStmtNode.Severity.ERROR
                        : AssertStmtNode.Severity.WARN,
                s.message()
        );

        case TRefStmt s -> new RefStmtNode(
                Path.of(s.targetRefPath().path()),
                s.resolutionBinding().sourceRefClass(),
                s.resolutionBinding().targetRefClass(),
                buildExpr(s.sourceRefExpr(), source)
        );
    };

    return NodeSourceSections.attach(node, source, stmt.span());
}
```

`buildExpr(...)` mit spans

Java

```
private static ExprNode buildExpr(TExpr expr, Source source) {
    ExprNode node = switch (expr) {
        case TLiteralExpr l -> new LiteralNode(l.value());

        case TPathExpr p -> buildPathExpr(p.path());

        case TVarExpr v -> new VarReadNode(v.name());

        case TBinaryExpr b -> new BinaryOpNode(
                buildExpr(b.left(), source),
```

```
                b.op(),
                buildExpr(b.right(), source)
        );

        case TFunctionCallExpr f -> new FunctionCallNode(
                f.functionName(),
                f.args().stream().map(a -> buildExpr(a, source)).toArray(ExprNode[]::new)
        );

        case TMapEnumExpr m -> new MapEnumExprNode(
                buildExpr(m.sourceExpr(), source),
                new java.util.HashMap<>(m.mappings()),
                m.defaultValue(),
                m.defaultWarnMessage()
        );
    };

    return NodeSourceSections.attach(node, source, expr.span());
}
```

`Coercion` -Node ebenfalls spanbar machen

‹/› Java

```
private static ExprNode applyCoercion(ExprNode expr, Coercion coercion, Source source, Sc
    if (coercion == null || coercion instanceof NoCoercion) return expr;
    if (coercion instanceof PrimitiveCoercion pc) {
        return NodeSourceSections.attach(new CoerceExprNode(expr, pc.strategy()), source,
    }
    throw new IllegalStateException("Unsupported coercion: " + coercion);
}
```

7) Runtime-Report mit DSL-Position

Jetzt machen wir die Meldungen im Report wirklich nützlich.

7.1 `MigrationReport` erweitern

Ich würde im Report **keine Truffle-** `SourceSection` **direkt** speichern (nicht serialisierbar / runtime-gebunden), sondern eine kleine DTO-Lokation.

‹/› Java

```java
package ch.example.ilimap.runtime;

public record DslLocation(
        String sourceName,
        int startLine,
        int startColumn,
        int endLine,
        int endColumn
) {
    @Override
    public String toString() {
        return sourceName + ":" + startLine + ":" + startColumn;
    }
}
```

‹/› Java

```java
package ch.example.ilimap.runtime;

import com.oracle.truffle.api.nodes.Node;
import com.oracle.truffle.api.source.SourceSection;

import java.util.ArrayList;
```

```
import java.util.List;

public final class MigrationReport {
    private final List<Issue> issues = new ArrayList<>();
    private final List<String> infos = new ArrayList<>();

    public void error(String message) { issues.add(new Issue(Severity.ERROR, message, nul
    public void warn(String message)  { issues.add(new Issue(Severity.WARN, message, null
    public void info(String message)  { infos.add(message); }

    public void error(String message, EvalContext ctx, Node origin) {
        issues.add(new Issue(
                Severity.ERROR, message,
                ctx.source().classQName(), ctx.source().tid(),
                ctx.target().classQName(), ctx.target().tid(),
                toDslLocation(origin)
        ));
    }

    public void warn(String message, EvalContext ctx, Node origin) {
        issues.add(new Issue(
                Severity.WARN, message,
                ctx.source().classQName(), ctx.source().tid(),
                ctx.target().classQName(), ctx.target().tid(),
                toDslLocation(origin)
        ));
    }

    public List<Issue> issues() { return List.copyOf(issues); }
    public List<String> infos() { return List.copyOf(infos); }

    public long errorCount() { return issues.stream().filter(i -> i.severity() == Severit
    public long warnCount()  { return issues.stream().filter(i -> i.severity() == Severit

    private static DslLocation toDslLocation(Node origin) {
```

```
        if (origin == null) return null;
        SourceSection ss = origin.getSourceSection();
        if (ss == null || !ss.isAvailable()) return null;

        String src = ss.getSource() != null ? ss.getSource().getName() : "<unknown>";
        return new DslLocation(src, ss.getStartLine(), ss.getStartColumn(), ss.getEndLine
    }

    public record Issue(
            Severity severity,
            String message,
            String sourceClass,
            String sourceTid,
            String targetClass,
            String targetTid,
            DslLocation dslLocation
    ) {
        public String pretty() {
            String loc = dslLocation != null ? " [" + dslLocation + "]" : "";
            String src = (sourceClass != null && sourceTid != null) ? " @ " + sourceClass
            return severity + ": " + message + src + loc;
        }
    }

    public enum Severity { ERROR, WARN }
}
```

7.2 Nodes melden jetzt mit `origin=this`

`AssertStmtNode`

Java

```java
@Override
public void executeVoid(VirtualFrame frame, EvalContext ctx) {
    boolean ok = condition.executeBoolean(frame, ctx);
    if (ok) return;

    String msg = message;
    if (severity == Severity.ERROR) ctx.migration().report().error(msg, ctx, this);
    else ctx.migration().report().warn(msg, ctx, this);
}
```

`MapEnumExprNode` (Default-Warnung / unmapped enum)

Java

```java
@Override
public Object executeGeneric(VirtualFrame frame, EvalContext ctx) {
    Object raw = sourceExpr.executeGeneric(frame, ctx);
    if (raw == null) return null;

    String key = String.valueOf(raw);
    String mapped = mappings.get(key);
    if (mapped != null) return mapped;

    if (defaultValue != null) {
        if (defaultWarnMessage != null && !defaultWarnMessage.isBlank()) {
            ctx.migration().report().warn(defaultWarnMessage + " (value=" + key + ")", ct
        }
        return defaultValue;
    }

    ctx.migration().report().error("Unmapped enum value without DEFAULT: " + key, ctx, th
```

```
    return null;
}
```

`DeferredRefResolver` (optional Truffle-node-origin)

Hier hast du aktuell nur `DeferredRef` ohne Node-Referenz. Wenn du DSL-Lokation im Pass 2 willst, speichere eine Location **beim Erzeugen**.

---

8) Pass-2 Referenzauflösung mit DSL-Lokation

Das ist ein sehr guter Bonus: missing refs entstehen oft erst in Pass 2.

8.1 `DeferredRef` um `DslLocation` erweitern

Statt Truffle- `Node` zu speichern (unsauber), speichere direkt `DslLocation` .

Java

```java
package ch.example.ilimap.runtime;

public record DeferredRef(
        String targetObjectClass,
        String targetObjectTid,
        Path targetRefPath,
        String sourceRefClass,
        String targetRefClass,
        String sourceRefTid,
        DslLocation dslLocation
) {}
```

8.2 `RefStmtNode` beim Vormerken

Java

```java
@Override
public void executeVoid(VirtualFrame frame, EvalContext ctx) {
    Object refTidObj = sourceRefTidExpr.executeGeneric(frame, ctx);
    if (refTidObj == null) return;

    String sourceRefTid = String.valueOf(refTidObj);

    ctx.migration().deferredRefs().add(new DeferredRef(
            ctx.target().classQName(),
            ctx.target().tid(),
            targetRefPath,
            sourceRefClass,
            targetRefClass,
            sourceRefTid,
            // DSL-Location aus diesem Node ziehen
            new DslLocation(
                    getSourceSection() != null && getSourceSection().getSource() != null
                            ? getSourceSection().getSource().getName() : "<unknown>",
                    getSourceSection() != null ? getSourceSection().getStartLine() : -1,
                    getSourceSection() != null ? getSourceSection().getStartColumn() : -1
                    getSourceSection() != null ? getSourceSection().getEndLine() : -1,
                    getSourceSection() != null ? getSourceSection().getEndColumn() : -1
            )
    ));
}
```

Eleganter wäre eine kleine Helper-Methode `DslLocations.fromNode(this)`.

8.3 `DeferredRefResolver` meldet mit Location

Wenn du die `MigrationReport` -Issue-Struktur nicht nochmals überladen willst, kannst du im Text anhängen. Besser ist ein eigener Overload.

Minimal pragmatisch:

Java

```
String msg = "Missing reference mapping: " + ref.sourceRefClass() + ":" + ref.sourceRefTi
        + " -> " + ref.targetRefClass()
        + (ref.dslLocation() != null ? " [" + ref.dslLocation() + "]" : "");
```

---

9) `IliMapLanguage.parse(...)` Bind-Fehler schön ausgeben (mit Span)

Du hast schon Bind-Diagnostik. Jetzt kannst du die Meldungen mit Position hochreichen.

Java

```
if (!bind.isOk()) {
    String msg = bind.diagnostics().errors().stream()
            .map(Diagnostic::pretty)
            .reduce((a, b) -> a + "; " + b)
            .orElse("Unknown semantic binding error");
    throw new IllegalArgumentException("Semantic binding failed: " + msg);
}
```

Und Warnings in Report:

Java

```java
bind.diagnostics().warnings().forEach(w ->
        ctx.report().warn("[BIND] " + w.pretty())
);
```

---

10) Testideen für den Durchstich

10.1 Binder-Diagnose enthält Zeile/Spalte

Ein Test mit absichtlich falschem Zielpfad:

Plain text

```
SET target.usaeg = MAP_ENUM(...)
```

Dann prüfen:

- `diagnostic.span() != null`
- `diagnostic.span().startLine() == erwarteteZeile`

10.2 Runtime-ASSERT enthält DSL-Location

Ein Test mit `ASSERT target.area_m2 >= 0` und negativer Fläche:

- Migration läuft
- `report.errorCount() == 1`
- `report.issues().get(0).dslLocation() != null`
- `dslLocation.sourceName == "test.ilimap"`

10.3 Missing Ref in Pass 2 enthält REF-Zeile

Gebäude referenziert Owner `E999`, Owner-Regel erzeugt keinen `B.Owner:E999`.
Dann prüfen:

- Warnung/Fehler für missing ref
- Meldung enthält `dslLocation` der `REF ... RESOLVE ...`-Zeile

---

11) Ein realistischer Stolperstein (damit du nicht suchst)

`Source.createSection(...)` API-Differenzen

Je nach GraalVM/Truffle-Version kann die Signatur leicht variieren. Darum ist der `try/catch`-Fallback in `NodeSourceSections.toSection(...)` hilfreich.

Spans bei rekursiven Exprs

Wenn du bei `BinaryExprAst` den Span des ganzen Ausdrucks setzt (statt nur linker/rechter Teil), ist das **genau richtig** für Reports.

---

12) Was dir das sofort bringt

Mit diesem Durchstich kannst du jetzt:

- **statische Fehler** präzise lokalisieren (Binder)
- **Runtime-Probleme** auf DSL-Regeln zurückführen (Report)
- später Truffle-Instrumentierung bauen, die echte Source-Locations hat

Das ist ein großer Qualitätssprung.

Wenn du willst, gehe ich als Nächstes den **letzten logischen Schritt**:

`InstrumentableNode` **+ Truffle Tags** (`StatementTag` , eigener `AssignmentTag` , `RuleTag`) — dann kannst du auf Basis deiner `SourceSection` s wirklich Tracing/Coverage/Profiler für die Mapping-DSL bauen.